

GUIA DE RECICLAJE COMPLETA

Kit Web Reutilizable - 57 Funcionalidades

Banco completo de componentes, layouts e interfaces interactivas para copiar, pegar e implementar directamente en tus proyectos profesionales.

Edicion Especial 2026

Autor: Carlos Madero

Todos los codigos vigentes y unificados en un solo documento.

TABLA DE CONTENIDO UNIFICADA

Este indice detalla la ubicacion de cada bloque teorico base (PDF original) y de cada anexo de codigo fuente listo para ser reciclado.

Bloque 1: Navegacion y Animaciones (IDs 1 a 10)

- Guia conceptual y teorica (PDF Base)	Pagina 4
* 1. Navbar fija glass (Teoria yCodigo)	Pagina 6
* 2. Sidebar ocultable (Teoria yCodigo)	Pagina 10
* 3. Submenús laterales (Teoria yCodigo)	Pagina 15
* 4. Router de secciones (Teoria yCodigo)	Pagina 19
* 5. Dropdown superior (Teoria yCodigo)	Pagina 23
* 6. Tema claro/oscuro (Teoria yCodigo)	Pagina 27
* 7. Scroll progress (Teoria yCodigo)	Pagina 31
* 8. Back to top (Teoria yCodigo)	Pagina 34
* 9. Reveal on scroll (Teoria yCodigo)	Pagina 38
* 10. Contador animado (Teoria yCodigo)	Pagina 42

Bloque 2: Autenticacion, Formularios y UI (IDs 11 a 25)

- Guia conceptual y teorica (PDF Base)	Pagina 48
* 11. Login demo (Teoria yCodigo)	Pagina 51
* 12. Logout demo (Teoria yCodigo)	Pagina 58
* 13. Recordar usuario (Teoria yCodigo)	Pagina 61
* 14. Password toggle (Teoria yCodigo)	Pagina 64
* 15. Validación formularios (Teoria yCodigo)	Pagina 67
* 16. Mensajes de error (Teoria yCodigo)	Pagina 71
* 17. Character counter (Teoria yCodigo)	Pagina 74
* 18. Range preview (Teoria yCodigo)	Pagina 77
* 19. File info (Teoria yCodigo)	Pagina 80
* 20. Autosave draft (Teoria yCodigo)	Pagina 83
* 21. Stepper (Teoria yCodigo)	Pagina 86
* 22. Skeleton loader (Teoria yCodigo)	Pagina 90
* 23. Select personalizado (Teoria yCodigo)	Pagina 93
* 24. Tabs (Teoria yCodigo)	Pagina 98
* 25. Acordeón (Teoria yCodigo)	Pagina 102

Bloque 3: Datos Dinamicos y Estado Local (IDs 26 a 40)

- Guia conceptual y teorica (PDF Base)	Pagina 107
* 26. Modal (Teoria yCodigo)	Pagina 110
* 27. Toast (Teoria yCodigo)	Pagina 115
* 28. Carrusel (Teoria yCodigo)	Pagina 118
* 29. Tooltip (Teoria yCodigo)	Pagina 123
* 30. Badge (Teoria yCodigo)	Pagina 127
* 31. Copy clipboard (Teoria yCodigo)	Pagina 130
* 32. Cards dinámicas (Teoria yCodigo)	Pagina 133
* 33. Búsqueda (Teoria yCodigo)	Pagina 136
* 34. Filtros chips (Teoria yCodigo)	Pagina 139
* 35. Ordenamiento (Teoria yCodigo)	Pagina 142
* 36. Paginación (Teoria yCodigo)	Pagina 145
* 37. Tabla ordenable (Teoria yCodigo)	Pagina 149
* 38. Filtro de tabla (Teoria yCodigo)	Pagina 152
* 39. LocalStorage notas (Teoria yCodigo)	Pagina 155
* 40. Export JSON (Teoria yCodigo)	Pagina 159

Bloque 4: Utilidades, Actividades y Scrollbar (IDs 41 a 57)

- Guia conceptual y teorica (PDF Base)	Pagina 163
- Anexo de codigos listos para copiar/pegar	Pagina 204
* 41. Todo list (Teoria yCodigo)	Pagina 167
* 42. Drag and drop (Teoria yCodigo)	Pagina 172
* 43. Conversor C/F/K (Teoria yCodigo)	Pagina 176
* 44. Cambio fondo dinámico (Teoria yCodigo)	Pagina 181
* 45. Print page (Teoria yCodigo)	Pagina 184
* 46. Reset demo (Teoria yCodigo)	Pagina 187

* 47. Helpers reutilizables (Teoria y Codigo)	Pagina 190
* 48. Versi�n modular (Teoria y Codigo)	Pagina 193
* 49. Versi�n normal (Teoria y Codigo)	Pagina 196
* 50. Responsive design (Teoria y Codigo)	Pagina 199
* 51. Confirmaci�n visual (Codigo y Guia)	Pagina 205
* 52. Debounce (Codigo y Guia)	Pagina 208
* 53. Tarjeta de presentaci�n Bootstrap (Codigo y Guia)	Pagina 211
* 54. Carrito de compras DOM (Codigo y Guia)	Pagina 220
* 55. Footer reutilizable (Codigo y Guia)	Pagina 230
* 56. Tarjetas de precios Bootstrap (Codigo y Guia)	Pagina 236
* 57. Barra de desplazamiento personalizada (Codigo y Guia)	Pagina 240

Guia de reciclaje de componentes web

Documento 1 - Funcionalidades 1 a 10 en el orden de la tabla de contenido original

Este documento corrige y reorganiza la guía para seguir exactamente el orden de la tabla de contenido del recurso unificado: Navbar fija glass, Sidebar ocultable, Submenus laterales, Router de secciones, Dropdown superior, Tema claro/oscuro, Scroll progress, Back to top, Reveal on scroll y Contador animado.

La idea no es solo ver el código, sino aprender a reciclar cada funcionalidad en cualquier proyecto. Por eso cada componente incluye: para que sirve, que copiar, que cambiar, código HTML, CSS, JavaScript modular, JavaScript normal, pasos de implementación, errores comunes y checklist.

Convenciones para usar esta guía

Convencion	Significado	Ejemplo
ID	Identificador unico. No debe repetirse en la pagina.	id="sidebar"
Clase	Nombre reutilizable para aplicar CSS o seleccionar elementos.	class="glass-navbar"
data-*	Atributo personalizado para guardar datos que JS puede leer.	data-section="inicio"
is-open / is-visible	Clases de estado. JS las agrega o quita para cambiar la interfaz.	.sidebar.is-open
Modular	Código con export/import. Recomendado para proyectos organizados.	export class SidebarToggle {}
Normal	Código sin módulos. Util cuando quieres un solo archivo script-normal.js.	function initSidebarToggle() {}

Orden corregido de funcionalidades 1 a 10

#	Funcionalidad	Para que sirve	Archivo principal que toca
1	Navbar fija glass	Barra superior fija con blur y transparencia.	HTML + CSS + JS
2	Sidebar ocultable	Menu lateral que se abre y cierra.	HTML + CSS + JS
3	Submenus laterales	Opciones internas desplegables.	HTML + CSS + JS
4	Router de secciones	Muestra una seccion diferente en el body.	HTML + JS
5	Dropdown superior	Menu desplegable en navbar.	HTML + CSS + JS
6	Tema claro/oscuro	Cambia variables CSS y guarda preferencia.	CSS + JS
7	Scroll progress	Barra que muestra avance de scroll.	HTML + CSS + JS
8	Back to top	Boton para volver arriba.	HTML + CSS + JS

#	Funcionalidad	Para que sirve	Archivo principal que toca
9	Reveal on scroll	Animacion al aparecer en pantalla.	CSS + JS
10	Contador animado	Cuenta de 0 hasta un valor objetivo.	HTML + JS

Estructura recomendada de archivos

Para reciclar cualquier componente en proyectos nuevos, mantén una estructura simple. Puedes usar modulos o version normal, pero no mezcles las dos al mismo tiempo en la misma pagina para evitar duplicar eventos.

Estructura sugerida

```
mi-proyecto/
├─ index.html
├─ styles.css
├─ funciones.js      // clases y funciones exportadas
├─ script.js        // importa y arranca componentes
└─ script-normal.js  // alternativa sin import/export
```

Regla practica: si usas import/export, en HTML carga `<script type="module" src="script.js"></script>`. Si no usas modulos, carga solo `<script src="script-normal.js"></script>`.

1. Navbar fija glass

La navbar fija glass es una barra superior que permanece visible mientras el usuario hace scroll. Usa transparencia, blur y sombra para verse cristalina sin tapar totalmente el fondo. Es muy usada en landing pages, dashboards, portafolios, ecommerce y aplicaciones web modernas.

Donde se usa en paginas reales: cabeceras de paginas, menus principales, paneles administrativos, sitios de marca, portafolios y single page applications.

Dependencias internas: puede trabajar sola, pero normalmente se conecta con el Router de secciones mediante botones con data-section.

Idea para reciclar: copia solo el bloque HTML que corresponde al componente, copia las reglas CSS indicadas y copia la clase JS modular o la funcion normal. Luego cambia IDs, clases y textos segun tu proyecto.

Parte	Que debes copiar	Que puedes cambiar sin romperlo
HTML	Estructura visible del componente y atributos data-*.	Textos, iconos, rutas, nombres de secciones, opciones y labels.
CSS	Clases del componente y, si aplica, variables de color.	Colores, blur, radios, espacios, sombras, tiempos de transicion.
JS modular	Clase exportada con constructor e inicializacion.	Selectores, nombres de clase activa, claves localStorage y comportamiento.
JS normal	Funcion inicializadora sin import/export.	Selectores y condiciones, igual que en la version modular.

Codigo HTML minimo

HTML que debes llevar a index.html

```
<header class="glass-navbar" id="mainNavbar">
  <a href="#inicio" class="glass-navbar__brand">Mi Proyecto</a>

  <nav class="glass-navbar__links" aria-label="Menu principal">
    <button type="button" data-section="inicio">Inicio</button>
    <button type="button" data-section="servicios">Servicios</button>
    <button type="button" data-section="contacto">Contacto</button>
  </nav>

  <button type="button" class="glass-navbar__menu" id="openSidebar">
    Menu
  </button>
</header>
```

Codigo CSS minimo

CSS que debes llevar a styles.css

```

/* Variable util para que el contenido no quede debajo de la navbar */
:root {
  --navbar-height: 72px;
  --glass-bg: rgba(255, 255, 255, 0.14);
  --glass-border: rgba(255, 255, 255, 0.28);
  --primary: #2563eb;
}

body {
  padding-top: var(--navbar-height);
}

.glass-navbar {
  position: fixed;
  top: 0;
  left: 0;
  z-index: 1000;
  width: 100%;
  min-height: var(--navbar-height);
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 16px;
  padding: 14px 24px;
  background: var(--glass-bg);
  border-bottom: 1px solid var(--glass-border);
  backdrop-filter: blur(18px);
  -webkit-backdrop-filter: blur(18px);
  box-shadow: 0 12px 30px rgba(15, 23, 42, 0.18);
}

.glass-navbar__brand {
  color: #ffffff;
  font-weight: 800;
  text-decoration: none;
}

.glass-navbar__links {
  display: flex;
  gap: 10px;
}

.glass-navbar button {
  border: 0;
  border-radius: 999px;
  padding: 10px 14px;
  color: #ffffff;
  background: rgba(255, 255, 255, 0.12);
  cursor: pointer;
}

.glass-navbar button:hover,
.glass-navbar button.is-active {
  background: var(--primary);
}

@media (max-width: 720px) {
  .glass-navbar__links {
    display: none;
  }
}

```

JavaScript modular con export/import

funciones.js - clase o funcion exportada

```
export class GlassNavbar {
  constructor(navbarId = "mainNavbar") {
    this.navbar = document.getElementById(navbarId);
    this.buttons = this.navbar?.querySelectorAll("[data-section]") || [];
    this.init();
  }

  init() {
    this.buttons.forEach((button) => {
      button.addEventListener("click", () => {
        this.setActive(button);
      });
    });
  }

  setActive(activeButton) {
    this.buttons.forEach((button) => button.classList.remove("is-active"));
    activeButton.classList.add("is-active");
  }
}
```

script.js - como llamarlo

```
// script.js
import { GlassNavbar } from './funciones.js';

new GlassNavbar();
```

JavaScript normal sin modulos

script-normal.js - version directa

```
function initGlassNavbar(navbarId = "mainNavbar") {
  const navbar = document.getElementById(navbarId);
  if (!navbar) return;

  const buttons = navbar.querySelectorAll("[data-section]");

  buttons.forEach((button) => {
    button.addEventListener("click", () => {
      buttons.forEach((item) => item.classList.remove("is-active"));
      button.classList.add("is-active");
    });
  });
}

initGlassNavbar();
```

Variables, clases, IDs y atributos que debes cambiar

Elemento	Donde esta	Como adaptarlo
#mainNavbar	HTML y JS	Cambia el id solo si tambien cambias el argumento new GlassNavbar('nuevold').
.glass-navbar	CSS y HTML	Puedes renombrarla, pero cambia todas sus reglas CSS.
data-section	Botones	Debe coincidir con las secciones si usas router.
--navbar-height	:root	Ajusta la altura si tu navbar es mas alta o mas baja.

Elemento	Donde esta	Como adaptarlo
--glass-bg	:root	Sube o baja la transparencia; 0.10 es mas transparente, 0.30 mas solido.

Paso a paso para implementarlo en otro proyecto

1. Copia el bloque <header> justo despues de abrir <body>.
2. Copia las variables CSS y las reglas de .glass-navbar en tu styles.css.
3. Verifica que el body tenga padding-top para que el contenido no quede escondido debajo de la navbar.
4. Si vas a controlar secciones, pon data-section en cada boton y crea secciones con esos mismos nombres.
5. Importa GlassNavbar en script.js y crea new GlassNavbar().
6. Prueba scroll: la barra debe quedarse fija y el blur debe verse sobre el fondo.

Errores comunes y como corregirlos

Error	Causa probable	Solucion
La navbar tapa el contenido	Falta padding-top en body	Usa body { padding-top: var(--navbar-height); }
No se ve el blur	El fondo detras es plano o el navegador no soporta backdrop-filter	Usa un fondo con degradados y agrega -webkit-backdrop-filter.
Los botones no se activan	El JS no encuentra #mainNavbar	Revisa que el id del HTML coincida con el constructor.

Checklist de prueba

- La navbar queda fija al hacer scroll.
- Los botones cambian a estado activo al hacer clic.
- En celular se ocultan los links si asi lo definiste.
- El contenido no queda debajo de la barra superior.

Adaptaciones profesionales recomendadas

- Agrega aria-label al nav para mejorar accesibilidad.
- Usa una clase .is-scrolled para oscurecer la navbar cuando el usuario baja.
- Conecta el boton Menu con el Sidebar ocultable de la funcionalidad 2.

2. Sidebar ocultable

El sidebar ocultable es un menu lateral que entra y sale de la pantalla cuando el usuario lo pide. Es muy comun en dashboards, paneles administrativos, aplicaciones tipo CRM, proyectos con muchas secciones y paginas responsive.

Donde se usa en paginas reales: dashboards, aplicaciones internas, ecommerce con filtros, sistemas de aprendizaje, paneles con muchas opciones.

Dependencias internas: puede abrirse desde un boton de la navbar; se complementa con overlay y submenus laterales.

Idea para reciclar: copia solo el bloque HTML que corresponde al componente, copia las reglas CSS indicadas y copia la clase JS modular o la funcion normal. Luego cambia IDs, clases y textos segun tu proyecto.

Parte	Que debes copiar	Que puedes cambiar sin romperlo
HTML	Estructura visible del componente y atributos data-*.	Textos, iconos, rutas, nombres de secciones, opciones y labels.
CSS	Clases del componente y, si aplica, variables de color.	Colores, blur, radios, espacios, sombras, tiempos de transicion.
JS modular	Clase exportada con constructor e inicializacion.	Selectores, nombres de clase activa, claves localStorage y comportamiento.
JS normal	Funcion inicializadora sin import/export.	Selectores y condiciones, igual que en la version modular.

Codigo HTML minimo

HTML que debes llevar a index.html

```
<button type="button" id="openSidebar" class="btn-menu">Abrir menu</button>

<aside class="sidebar" id="sidebar" aria-hidden="true">
  <div class="sidebar__header">
    <h2>Panel</h2>
    <button type="button" id="closeSidebar">Cerrar</button>
  </div>

  <nav class="sidebar__nav">
    <button type="button" data-section="inicio">Inicio</button>
    <button type="button" data-section="perfil">Perfil</button>
    <button type="button" data-section="configuracion">Configuracion</button>
  </nav>
</aside>

<div class="sidebar-overlay" id="sidebarOverlay"></div>
```

Codigo CSS minimo

CSS que debes llevar a styles.css

```

body.sidebar-open {
  overflow: hidden;
}

.sidebar {
  position: fixed;
  top: 0;
  left: 0;
  z-index: 1200;
  width: min(320px, 86vw);
  height: 100vh;
  padding: 20px;
  color: #ffffff;
  background: rgba(15, 23, 42, 0.88);
  border-right: 1px solid rgba(255, 255, 255, 0.18);
  backdrop-filter: blur(20px);
  -webkit-backdrop-filter: blur(20px);
  transform: translateX(-105%);
  transition: transform 0.25s ease;
}

.sidebar.is-open {
  transform: translateX(0);
}

.sidebar__header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  gap: 12px;
  margin-bottom: 20px;
}

.sidebar__nav {
  display: grid;
  gap: 8px;
}

.sidebar__nav button,
.sidebar__header button,
.btn-menu {
  border: 0;
  border-radius: 12px;
  padding: 11px 14px;
  color: #ffffff;
  background: rgba(255, 255, 255, 0.12);
  cursor: pointer;
}

.sidebar-overlay {
  position: fixed;
  inset: 0;
  z-index: 1100;
  background: rgba(0, 0, 0, 0.45);
  opacity: 0;
  pointer-events: none;
  transition: opacity 0.25s ease;
}

.sidebar-overlay.is-visible {
  opacity: 1;
  pointer-events: auto;
}

```

JavaScript modular con export/import

funciones.js - clase o funcion exportada

```

export class SidebarToggle {
  constructor(config = {}) {
    this.sidebar = document.getElementById(config.sidebarId || "sidebar");
    this.openButton = document.getElementById(config.openButtonId || "openSidebar");
    this.closeButton = document.getElementById(config.closeButtonId ||
      "closeSidebar");
    this.overlay = document.getElementById(config.overlayId || "sidebarOverlay");
    this.init();
  }

  init() {
    this.openButton?.addEventListener("click", () => this.open());
    this.closeButton?.addEventListener("click", () => this.close());
    this.overlay?.addEventListener("click", () => this.close());
    document.addEventListener("keydown", (event) => {
      if (event.key === "Escape") this.close();
    });
  }

  open() {
    this.sidebar?.classList.add("is-open");
    this.overlay?.classList.add("is-visible");
    document.body.classList.add("sidebar-open");
    this.sidebar?.setAttribute("aria-hidden", "false");
  }

  close() {
    this.sidebar?.classList.remove("is-open");
    this.overlay?.classList.remove("is-visible");
    document.body.classList.remove("sidebar-open");
    this.sidebar?.setAttribute("aria-hidden", "true");
  }
}

```

script.js - como llamarlo

```

// script.js
import { SidebarToggle } from './funciones.js';

new SidebarToggle();

```

JavaScript normal sin modulos

script-normal.js - version directa

```
function initSidebarToggle() {
  const sidebar = document.getElementById("sidebar");
  const openButton = document.getElementById("openSidebar");
  const closeButton = document.getElementById("closeSidebar");
  const overlay = document.getElementById("sidebarOverlay");

  function open() {
    sidebar.classList.add("is-open");
    overlay.classList.add("is-visible");
    document.body.classList.add("sidebar-open");
    sidebar.setAttribute("aria-hidden", "false");
  }

  function close() {
    sidebar.classList.remove("is-open");
    overlay.classList.remove("is-visible");
    document.body.classList.remove("sidebar-open");
    sidebar.setAttribute("aria-hidden", "true");
  }

  openButton?.addEventListener("click", open);
  closeButton?.addEventListener("click", close);
  overlay?.addEventListener("click", close);
  document.addEventListener("keydown", (event) => {
    if (event.key === "Escape") close();
  });
}

initSidebarToggle();
```

Variables, clases, IDs y atributos que debes cambiar

Elemento	Donde esta	Como adaptarlo
#sidebar	HTML/JS	ID del panel lateral. Cambialo en HTML y en config.sidebarId.
#openSidebar	HTML/JS	Boton que abre. Puede estar en navbar o en cualquier parte.
#closeSidebar	HTML/JS	Boton interno que cierra el menu.
#sidebarOverlay	HTML/JS/CSS	Capa oscura para cerrar al hacer clic fuera.
width	CSS .sidebar	Modifica el ancho del panel, por ejemplo 280px o 360px.

Paso a paso para implementarlo en otro proyecto

1. Copia el aside, el boton de abrir y el overlay.
2. Copia CSS de .sidebar, .sidebar-overlay y body.sidebar-open.
3. Copia la clase SidebarToggle en funciones.js.
4. En script.js importa SidebarToggle y crea new SidebarToggle().
5. Prueba abrir, cerrar, cerrar con overlay y cerrar con tecla Escape.

Errores comunes y como corregirlos

Error	Causa probable	Solucion
El sidebar aparece siempre visible	Falta transform translateX(-105%)	Verifica la clase .sidebar y que .is-open solo se agregue con JS.
No cierra con overlay	El overlay no existe o tiene otro id	Revisa #sidebarOverlay y el constructor.
La pagina sigue haciendo scroll	Falta body.sidebar-open	Agrega overflow: hidden cuando el sidebar esta abierto.

Checklist de prueba

- Abre desde el boton principal.
- Cierra con boton Cerrar.
- Cierra con clic en overlay.
- Cierra con Escape.
- En movil no se sale de la pantalla.

Adaptaciones profesionales recomendadas

- Agrega focus trap si el sidebar contiene formularios.
- Usa aria-expanded en el boton de abrir para accesibilidad avanzada.
- Guarda el estado abierto/cerrado si deseas recordar el layout de un dashboard.

3. Submenus laterales

Los submenus laterales permiten agrupar opciones dentro del sidebar. El usuario puede abrir una categoria y ver sus opciones internas. Es clave para organizar proyectos grandes sin saturar la pantalla.

Donde se usa en paginas reales: paneles administrativos, menus de configuracion, rutas de aprendizaje, categorias de tienda, menus de usuario.

Dependencias internas: normalmente va dentro del Sidebar ocultable, pero tambien sirve en menus verticales fijos.

Idea para reciclar: copia solo el bloque HTML que corresponde al componente, copia las reglas CSS indicadas y copia la clase JS modular o la funcion normal. Luego cambia IDs, clases y textos segun tu proyecto.

Parte	Que debes copiar	Que puedes cambiar sin romperlo
HTML	Estructura visible del componente y atributos data-*	Textos, iconos, rutas, nombres de secciones, opciones y labels.
CSS	Clases del componente y, si aplica, variables de color.	Colores, blur, radios, espacios, sombras, tiempos de transicion.
JS modular	Clase exportada con constructor e inicializacion.	Selectores, nombres de clase activa, claves localStorage y comportamiento.
JS normal	Funcion inicializadora sin import/export.	Selectores y condiciones, igual que en la version modular.

Codigo HTML minimo

HTML que debes llevar a index.html

```
<aside class="sidebar" id="sidebar">
  <div class="side-group">
    <button type="button" class="side-group__toggle">
      Productos
    </button>
    <div class="side-group__content">
      <button type="button" data-section="cervezas">Cervezas</button>
      <button type="button" data-section="combos">Combos</button>
      <button type="button" data-section="promos">Promociones</button>
    </div>
  </div>

  <div class="side-group">
    <button type="button" class="side-group__toggle">
      Administracion
    </button>
    <div class="side-group__content">
      <button type="button" data-section="usuarios">Usuarios</button>
      <button type="button" data-section="reportes">Reportes</button>
    </div>
  </div>
</aside>
```

Codigo CSS minimo

CSS que debes llevar a styles.css

```

.side-group {
  display: grid;
  gap: 6px;
  margin-bottom: 10px;
}

.side-group__toggle {
  width: 100%;
  display: flex;
  align-items: center;
  justify-content: space-between;
  border: 0;
  border-radius: 12px;
  padding: 12px 14px;
  color: #ffffff;
  background: rgba(255, 255, 255, 0.14);
  cursor: pointer;
}

.side-group__toggle span {
  transition: transform 0.2s ease;
}

.side-group.is-open .side-group__toggle span {
  transform: rotate(45deg);
}

.side-group__content {
  display: grid;
  gap: 6px;
  max-height: 0;
  overflow: hidden;
  transition: max-height 0.25s ease;
}

.side-group.is-open .side-group__content {
  max-height: 260px;
}

.side-group__content button {
  margin-left: 12px;
  border: 0;
  border-radius: 10px;
  padding: 10px 12px;
  color: #ffffff;
  background: rgba(255, 255, 255, 0.08);
  text-align: left;
  cursor: pointer;
}

```

JavaScript modular con export/import

funciones.js - clase o funcion exportada


```
export class SidebarSubmenus {
  constructor(groupSelector = ".side-group") {
    this.groups = document.querySelectorAll(groupSelector);
    this.init();
  }

  init() {
    this.groups.forEach((group) => {
      const toggle = group.querySelector(".side-group__toggle");
      toggle?.addEventListener("click", () => {
        group.classList.toggle("is-open");
      });
    });
  }
}
```

script.js - como llamarlo

```
// script.js
import { SidebarSubmenus } from './funciones.js';

new SidebarSubmenus();
```

JavaScript normal sin modulos

script-normal.js - version directa

```
function initSidebarSubmenus(groupSelector = ".side-group") {
  const groups = document.querySelectorAll(groupSelector);

  groups.forEach((group) => {
    const toggle = group.querySelector(".side-group__toggle");
    toggle?.addEventListener("click", () => {
      group.classList.toggle("is-open");
    });
  });
}

initSidebarSubmenus();
```

Variables, clases, IDs y atributos que debes cambiar

Elemento	Donde esta	Como adaptarlo
.side-group	HTML/CSS/JS	Cada grupo nuevo debe usar esta clase para que JS lo reconozca.
.side-group__toggle	HTML/JS	Boton que abre y cierra el grupo.
.side-group__content	HTML/CSS	Contenedor de opciones internas.
data-section	Botones internos	Conecta cada opcion con una seccion del Router.
max-height	CSS	Aumentalo si el submenu tiene muchas opciones.

Paso a paso para implementarlo en otro proyecto

1. Inserta uno o varios .side-group dentro del sidebar.
2. Copia las reglas CSS de side-group y sus hijos.

3. Copia SidebarSubmenus a funciones.js.
4. Importa y ejecuta new SidebarSubmenus() en script.js.
5. Duplica un grupo completo para crear nuevas categorias.

Errores comunes y como corregirlos

Error	Causa probable	Solucion
El submenu no abre	Falta clase .side-group__toggle	El boton que abre debe tener exactamente esa clase o debes cambiar el selector.
Se cortan opciones	max-height muy bajo	Sube max-height a 400px o usa calculo dinamico con scrollHeight.
Abren todos a la vez	Se esta usando una clase global mal aplicada	Asegurate de aplicar toggle solo al group actual.

Checklist de prueba

- Cada grupo abre y cierra de forma independiente.
- Las opciones internas se ven alineadas y con margen.
- La animacion no corta textos.
- Los data-section internos funcionan si estan conectados al router.

Adaptaciones profesionales recomendadas

- Agrega modo acordeon: cerrar los otros grupos cuando uno se abre.
- Agrega iconos SVG en lugar del signo +.
- Usa aria-expanded para indicar si el grupo esta abierto.

4. Router de secciones

El router de secciones permite cambiar lo que se muestra en el body sin recargar la pagina. Es una base sencilla para crear una SPA pequeña: al hacer clic en un boton, se ocultan las demas secciones y se muestra solo la solicitada.

Donde se usa en paginas reales: portafolios, demos de componentes, dashboards simples, proyectos academicos, landing pages interactivas.

Dependencias internas: funciona con cualquier boton que tenga data-section y con secciones que tengan data-view-section e id coincidente.

Idea para reciclar: copia solo el bloque HTML que corresponde al componente, copia las reglas CSS indicadas y copia la clase JS modular o la funcion normal. Luego cambia IDs, clases y textos segun tu proyecto.

Parte	Que debes copiar	Que puedes cambiar sin romperlo
HTML	Estructura visible del componente y atributos data-*.	Textos, iconos, rutas, nombres de secciones, opciones y labels.
CSS	Clases del componente y, si aplica, variables de color.	Colores, blur, radios, espacios, sombras, tiempos de transicion.
JS modular	Clase exportada con constructor e inicializacion.	Selectores, nombres de clase activa, claves localStorage y comportamiento.
JS normal	Funcion inicializadora sin import/export.	Selectores y condiciones, igual que en la version modular.

Codigo HTML minimo

HTML que debes llevar a index.html

```
<nav class="section-menu">
  <button type="button" data-section="inicio">Inicio</button>
  <button type="button" data-section="servicios">Servicios</button>
  <button type="button" data-section="contacto">Contacto</button>
</nav>

<main>
  <section id="inicio" data-view-section class="view-section is-visible">
    <h2>Inicio</h2>
    <p>Contenido de inicio.</p>
  </section>

  <section id="servicios" data-view-section class="view-section">
    <h2>Servicios</h2>
    <p>Contenido de servicios.</p>
  </section>

  <section id="contacto" data-view-section class="view-section">
    <h2>Contacto</h2>
    <p>Contenido de contacto.</p>
  </section>
</main>
```

Codigo CSS minimo

CSS que debes llevar a styles.css

```

.view-section {
  display: none;
  animation: fadeIn 0.25s ease;
}

.view-section.is-visible {
  display: block;
}

[data-section].is-active {
  background: #2563eb;
  color: #ffffff;
}

@keyframes fadeIn {
  from {
    opacity: 0;
    transform: translateY(8px);
  }
  to {
    opacity: 1;
    transform: translateY(0);
  }
}

```

JavaScript modular con export/import

funciones.js - clase o funcion exportada

```

export class SectionRouter {
  constructor(config = {}) {
    this.buttonSelector = config.buttonSelector || "[data-section]";
    this.sectionSelector = config.sectionSelector || "[data-view-section]";
    this.buttons = document.querySelectorAll(this.buttonSelector);
    this.sections = document.querySelectorAll(this.sectionSelector);
    this.defaultSection = config.defaultSection || "inicio";
    this.init();
  }

  init() {
    this.buttons.forEach((button) => {
      button.addEventListener("click", () => {
        this.show(button.dataset.section);
      });
    });

    this.show(this.defaultSection);
  }

  show(sectionId) {
    this.sections.forEach((section) => {
      section.classList.toggle("is-visible", section.id === sectionId);
    });

    this.buttons.forEach((button) => {
      button.classList.toggle("is-active", button.dataset.section === sectionId);
    });
  }
}

```

script.js - como llamarlo

```

// script.js
import { SectionRouter } from './funciones.js';

new SectionRouter();

```

JavaScript normal sin modulos

script-normal.js - version directa

```
function initSectionRouter(defaultSection = "inicio") {
  const buttons = document.querySelectorAll("[data-section]");
  const sections = document.querySelectorAll("[data-view-section]");

  function show(sectionId) {
    sections.forEach((section) => {
      section.classList.toggle("is-visible", section.id === sectionId);
    });

    buttons.forEach((button) => {
      button.classList.toggle("is-active", button.dataset.section === sectionId);
    });
  }

  buttons.forEach((button) => {
    button.addEventListener("click", () => show(button.dataset.section));
  });

  show(defaultSection);
}

initSectionRouter();
```

Variables, clases, IDs y atributos que debes cambiar

Elemento	Donde esta	Como adaptarlo
data-section	Botones	Su valor debe ser igual al id de la seccion que quieres mostrar.
data-view-section	Secciones	Marca las secciones que el router puede controlar.
.is-visible	CSS/JS	Clase que muestra la seccion activa.
.is-active	CSS/JS	Clase visual para el boton activo.
defaultSection	JS	Seccion que aparece al cargar la pagina.

Paso a paso para implementarlo en otro proyecto

1. Crea un boton con data-section='nombre'.
2. Crea una seccion con id='nombre' y data-view-section.
3. Copia CSS de .view-section y .is-visible.
4. Copia SectionRouter a funciones.js.
5. En script.js ejecuta new SectionRouter({ defaultSection: 'inicio' }).
6. Prueba que cada boton muestre solo su seccion.

Errores comunes y como corregirlos

Error	Causa probable	Solucion
El boton no muestra nada	data-section no coincide con el id	Iguala exactamente data-section='servicios' con id='servicios'.
Se ven todas las secciones	Falta CSS display none	Agrega .view-section { display: none; }.
La seccion inicial sale vacia	defaultSection no existe	Usa un id real como defaultSection.

Checklist de prueba

- Cada boton muestra una seccion distinta.
- Solo una seccion queda visible a la vez.
- El boton activo cambia de estilo.
- No hay IDs repetidos en HTML.

Adaptaciones profesionales recomendadas

- Agrega location.hash para que la URL cambie con cada seccion.
- Cierra el sidebar automaticamente al cambiar de seccion en movil.
- Carga contenido dinamico desde arrays si quieres evitar mucho HTML repetido.

5. Dropdown superior

El dropdown superior es un menu desplegable que aparece al hacer clic en un boton. Sirve para opciones de usuario, configuraciones rapidas, idioma, perfil, notificaciones o acciones secundarias.

Donde se usa en paginas reales: navbar de usuario, perfiles, acciones de tarjetas, menus de configuracion, dashboards y ecommerce.

Dependencias internas: puede ir dentro de la navbar fija glass o en cualquier contenedor. Usa una clase abierta para mostrar/ocultar.

Idea para reciclar: copia solo el bloque HTML que corresponde al componente, copia las reglas CSS indicadas y copia la clase JS modular o la funcion normal. Luego cambia IDs, clases y textos segun tu proyecto.

Parte	Que debes copiar	Que puedes cambiar sin romperlo
HTML	Estructura visible del componente y atributos data-*	Textos, iconos, rutas, nombres de secciones, opciones y labels.
CSS	Clases del componente y, si aplica, variables de color.	Colores, blur, radios, espacios, sombras, tiempos de transicion.
JS modular	Clase exportada con constructor e inicializacion.	Selectores, nombres de clase activa, claves localStorage y comportamiento.
JS normal	Funcion inicializadora sin import/export.	Selectores y condiciones, igual que en la version modular.

Codigo HTML minimo

HTML que debes llevar a index.html

```
<div class="dropdown" id="userDropdown">
  <button type="button" class="dropdown__button">
    Carlos
    <span>v</span>
  </button>

  <div class="dropdown__menu">
    <button type="button" data-section="perfil">Mi perfil</button>
    <button type="button" data-section="configuracion">Configuracion</button>
    <button type="button" id="logoutButton">Cerrar sesion</button>
  </div>
</div>
```

Codigo CSS minimo

CSS que debes llevar a styles.css

```

.dropdown {
  position: relative;
  display: inline-block;
}

.dropdown__button {
  border: 0;
  border-radius: 999px;
  padding: 10px 14px;
  color: #ffffff;
  background: rgba(255, 255, 255, 0.14);
  cursor: pointer;
}

.dropdown__menu {
  position: absolute;
  top: calc(100% + 10px);
  right: 0;
  min-width: 190px;
  padding: 8px;
  background: rgba(15, 23, 42, 0.88);
  border: 1px solid rgba(255, 255, 255, 0.20);
  border-radius: 16px;
  backdrop-filter: blur(18px);
  -webkit-backdrop-filter: blur(18px);
  box-shadow: 0 18px 45px rgba(0, 0, 0, 0.28);
  opacity: 0;
  transform: translateY(-8px);
  pointer-events: none;
  transition: opacity 0.2s ease, transform 0.2s ease;
}

.dropdown.is-open .dropdown__menu {
  opacity: 1;
  transform: translateY(0);
  pointer-events: auto;
}

.dropdown__menu button {
  width: 100%;
  border: 0;
  border-radius: 10px;
  padding: 10px 12px;
  color: #ffffff;
  background: transparent;
  text-align: left;
  cursor: pointer;
}

.dropdown__menu button:hover {
  background: rgba(255, 255, 255, 0.12);
}

```

JavaScript modular con export/import

funciones.js - clase o funcion exportada


```
export class TopDropdown {
  constructor(dropdownId = "userDropdown") {
    this.dropdown = document.getElementById(dropdownId);
    this.button = this.dropdown?.querySelector(".dropdown__button");
    this.init();
  }

  init() {
    this.button?.addEventListener("click", (event) => {
      event.stopPropagation();
      this.dropdown.classList.toggle("is-open");
    });

    document.addEventListener("click", () => this.close());
    document.addEventListener("keydown", (event) => {
      if (event.key === "Escape") this.close();
    });
  }

  close() {
    this.dropdown?.classList.remove("is-open");
  }
}
```

script.js - como llamarlo

```
// script.js
import { TopDropdown } from './funciones.js';

new TopDropdown();
```

JavaScript normal sin modulos

script-normal.js - version directa

```
function initTopDropdown(dropdownId = "userDropdown") {
  const dropdown = document.getElementById(dropdownId);
  const button = dropdown?.querySelector(".dropdown__button");

  button?.addEventListener("click", (event) => {
    event.stopPropagation();
    dropdown.classList.toggle("is-open");
  });

  document.addEventListener("click", () => {
    dropdown?.classList.remove("is-open");
  });

  document.addEventListener("keydown", (event) => {
    if (event.key === "Escape") dropdown?.classList.remove("is-open");
  });
}

initTopDropdown();
```

Variables, clases, IDs y atributos que debes cambiar

Elemento	Donde esta	Como adaptarlo
#userDropdown	HTML/JS	ID del dropdown. Si cambias uno, cambia el otro.
.dropdown__button	HTML/JS/CSS	Boton que abre/cierra el menu.

Elemento	Donde esta	Como adaptarlo
.dropdown__menu	HTML/CSS	Contenedor de opciones.
right: 0	CSS	Cambia a left: 0 si quieres que abra hacia la izquierda visual.
data-section	Opciones	Puedes usarlo para abrir secciones del router.

Paso a paso para implementarlo en otro proyecto

1. Copia el contenedor .dropdown en la navbar o en una card.
2. Copia el CSS del dropdown.
3. Copia TopDropdown en funciones.js.
4. Importa y ejecuta new TopDropdown('userDropdown').
5. Agrega o elimina botones dentro de .dropdown__menu segun tus opciones.

Errores comunes y como corregirlos

Error	Causa probable	Solucion
Se abre y se cierra al instante	No se uso event.stopPropagation	Mantén event.stopPropagation en el click del boton.
Queda debajo de otros elementos	z-index bajo o padre con overflow hidden	Sube z-index o evita overflow hidden en el contenedor padre.
No cierra al hacer clic fuera	Falta listener en document	Agrega document.addEventListener('click', close).

Checklist de prueba

- Abre al hacer clic en el boton.
- Cierra al hacer clic fuera.
- Cierra con Escape.
- Las opciones internas son clicables.

Adaptaciones profesionales recomendadas

- Convierte cada opcion en enlace <a> si dirige a otra pagina.
- Agrega iconos y separadores para opciones importantes.
- Cierra el dropdown despues de hacer clic en cualquier opcion interna.

6. Tema claro/oscuro

El cambio de tema permite alternar entre apariencia oscura y clara. En proyectos profesionales se hace con variables CSS para no reescribir todo el diseño. El tema elegido puede guardarse en localStorage para que el usuario conserve su preferencia.

Donde se usa en paginas reales: dashboards, blogs, portafolios, aplicaciones web, ecommerce y paneles de configuracion.

Dependencias internas: usa variables CSS, un atributo data-theme en <html> o <body>, un boton y localStorage.

Idea para reciclar: copia solo el bloque HTML que corresponde al componente, copia las reglas CSS indicadas y copia la clase JS modular o la funcion normal. Luego cambia IDs, clases y textos segun tu proyecto.

Parte	Que debes copiar	Que puedes cambiar sin romperlo
HTML	Estructura visible del componente y atributos data-.*.	Textos, iconos, rutas, nombres de secciones, opciones y labels.
CSS	Clases del componente y, si aplica, variables de color.	Colores, blur, radios, espacios, sombras, tiempos de transicion.
JS modular	Clase exportada con constructor e inicializacion.	Selectores, nombres de clase activa, claves localStorage y comportamiento.
JS normal	Funcion inicializadora sin import/export.	Selectores y condiciones, igual que en la version modular.

Codigo HTML minimo

HTML que debes llevar a index.html

```
<button type="button" id="themeToggle" class="theme-toggle">
  Cambiar tema
</button>
```

Codigo CSS minimo

CSS que debes llevar a styles.css

```

:root {
  --page-bg: #0f172a;
  --surface: rgba(255, 255, 255, 0.12);
  --text: #ffffff;
  --muted: rgba(255, 255, 255, 0.72);
  --primary: #2563eb;
}

[data-theme="light"] {
  --page-bg: #f8fafc;
  --surface: rgba(15, 23, 42, 0.06);
  --text: #0f172a;
  --muted: #475569;
  --primary: #1d4ed8;
}

body {
  background: var(--page-bg);
  color: var(--text);
  transition: background 0.25s ease, color 0.25s ease;
}

.card,
.glass-navbar,
.sidebar {
  background: var(--surface);
}

.theme-toggle {
  border: 0;
  border-radius: 999px;
  padding: 10px 14px;
  color: #ffffff;
  background: var(--primary);
  cursor: pointer;
}

```

JavaScript modular con export/import

funciones.js - clase o funcion exportada

```

export class ThemeToggle {
  constructor(config = {}) {
    this.button = document.getElementById(config.buttonId || "themeToggle");
    this.storageKey = config.storageKey || "kit-theme";
    this.root = document.documentElement;
    this.init();
  }

  init() {
    const savedTheme = localStorage.getItem(this.storageKey) || "dark";
    this.apply(savedTheme);

    this.button?.addEventListener("click", () => {
      const current = this.root.dataset.theme === "light" ? "light" : "dark";
      const next = current === "light" ? "dark" : "light";
      this.apply(next);
    });
  }

  apply(theme) {
    this.root.dataset.theme = theme;
    localStorage.setItem(this.storageKey, theme);
    if (this.button) {
      this.button.textContent = theme === "light" ? "Tema oscuro" : "Tema claro";
    }
  }
}

```

script.js - como llamarlo

```
// script.js
import { ThemeToggle } from './funciones.js';

new ThemeToggle();
```

JavaScript normal sin modulos

script-normal.js - version directa

```
function initThemeToggle() {
  const button = document.getElementById("themeToggle");
  const storageKey = "kit-theme";
  const root = document.documentElement;

  function apply(theme) {
    root.dataset.theme = theme;
    localStorage.setItem(storageKey, theme);
    if (button) {
      button.textContent = theme === "light" ? "Tema oscuro" : "Tema claro";
    }
  }

  const savedTheme = localStorage.getItem(storageKey) || "dark";
  apply(savedTheme);

  button?.addEventListener("click", () => {
    const current = root.dataset.theme === "light" ? "light" : "dark";
    apply(current === "light" ? "dark" : "light");
  });
}

initThemeToggle();
```

Variables, clases, IDs y atributos que debes cambiar

Elemento	Donde esta	Como adaptarlo
#themeToggle	HTML/JS	Boton que cambia tema. Cambia el id si actualizas el constructor.
--page-bg	CSS	Color o fondo principal de la pagina.
--surface	CSS	Color de tarjetas/nav/sidebar.
storageKey	JS	Clave de localStorage. Cambiala si mezclas varios proyectos.
data-theme	HTML/JS/CSS	Atributo que activa las variables del tema claro u oscuro.

Paso a paso para implementarlo en otro proyecto

1. Define tus colores base en :root.
2. Define los colores alternativos en [data-theme='light'] o [data-theme='dark'].
3. Usa variables en tus componentes: color: var(--text), background: var(--surface).
4. Copia ThemeToggle a funciones.js.

5. Importa y ejecuta `new ThemeToggle()`.
6. Recarga la pagina y verifica que el tema elegido se conserve.

Errores comunes y como corregirlos

Error	Causa probable	Solucion
Al cambiar tema no cambia nada	Los componentes no usan variables CSS	Reemplaza colores fijos por <code>var(--nombre)</code> .
No recuerda el tema	<code>localStorage</code> bloqueado o clave distinta	Verifica <code>storageKey</code> y el navegador.
Texto ilegible	Mismo color para fondo y texto	Ajusta <code>--text</code> y <code>--surface</code> en ambos temas.

Checklist de prueba

- El boton alterna entre tema claro y oscuro.
- Al recargar se mantiene la preferencia.
- Texto, cards y botones siguen siendo legibles.
- No quedan colores fijos que rompan el tema.

Adaptaciones profesionales recomendadas

- Detecta preferencia del sistema con `window.matchMedia('(prefers-color-scheme: dark)')`.
- Agrega transicion suave a fondo, color y bordes.
- Usa nombres de variables semanticas: `--surface`, `--text`, `--border`, `--primary`.

7. Scroll progress

La barra de progreso de scroll muestra cuanto ha avanzado el usuario dentro de la pagina. Es muy util en blogs largos, documentacion, landing pages y guias extensas.

Donde se usa en paginas reales: articulos, documentacion, cursos, tutoriales, landing pages largas y paginas de lectura.

Dependencias internas: solo necesita un div fijo y escuchar el evento scroll.

Idea para reciclar: copia solo el bloque HTML que corresponde al componente, copia las reglas CSS indicadas y copia la clase JS modular o la funcion normal. Luego cambia IDs, clases y textos segun tu proyecto.

Parte	Que debes copiar	Que puedes cambiar sin romperlo
HTML	Estructura visible del componente y atributos data-.*.	Textos, iconos, rutas, nombres de secciones, opciones y labels.
CSS	Clases del componente y, si aplica, variables de color.	Colores, blur, radios, espacios, sombras, tiempos de transicion.
JS modular	Clase exportada con constructor e inicializacion.	Selectores, nombres de clase activa, claves localStorage y comportamiento.
JS normal	Funcion inicializadora sin import/export.	Selectores y condiciones, igual que en la version modular.

Codigo HTML minimo

HTML que debes llevar a index.html

```
<div class="scroll-progress" id="scrollProgress"></div>
```

Codigo CSS minimo

CSS que debes llevar a styles.css

```
.scroll-progress {  
  position: fixed;  
  top: 0;  
  left: 0;  
  z-index: 2000;  
  width: 0%;  
  height: 4px;  
  background: linear-gradient(90deg, #2563eb, #7c3aed);  
  box-shadow: 0 0 16px rgba(37, 99, 235, 0.55);  
  transition: width 0.08s linear;  
}
```

JavaScript modular con export/import

funciones.js - clase o funcion exportada

```
export class ScrollProgress {
  constructor(progressId = "scrollProgress") {
    this.bar = document.getElementById(progressId);
    this.init();
  }

  init() {
    window.addEventListener("scroll", () => this.update(), { passive: true });
    this.update();
  }

  update() {
    const scrollTop = window.scrollY;
    const documentHeight = document.documentElement.scrollHeight - window.innerHeight;
    const progress = documentHeight > 0 ? (scrollTop / documentHeight) * 100 : 0;

    if (this.bar) {
      this.bar.style.width = `${progress}%`;
    }
  }
}
```

script.js - como llamarlo

```
// script.js
import { ScrollProgress } from './funciones.js';

new ScrollProgress();
```

JavaScript normal sin modulos

script-normal.js - version directa

```
function initScrollProgress(progressId = "scrollProgress") {
  const bar = document.getElementById(progressId);

  function update() {
    const scrollTop = window.scrollY;
    const documentHeight = document.documentElement.scrollHeight - window.innerHeight;
    const progress = documentHeight > 0 ? (scrollTop / documentHeight) * 100 : 0;
    if (bar) bar.style.width = `${progress}%`;
  }

  window.addEventListener("scroll", update, { passive: true });
  update();
}

initScrollProgress();
```

Variables, clases, IDs y atributos que debes cambiar

Elemento	Donde esta	Como adaptarlo
#scrollProgress	HTML/JS	ID de la barra. Cambialo en ambos sitios si lo renombas.
height	CSS	Grosor de la barra. Puedes usar 3px, 4px o 6px.
background	CSS	Color o degradado de la barra.
z-index	CSS	Debe estar por encima de navbar si quieres que se vea siempre.

Elemento	Donde esta	Como adaptarlo
passive: true	JS	Mejora rendimiento en scroll; dejalo si solo lees scroll.

Paso a paso para implementarlo en otro proyecto

1. Coloca el div #scrollProgress al inicio del body.
2. Copia la regla CSS .scroll-progress.
3. Copia ScrollProgress a funciones.js.
4. Ejecuta new ScrollProgress().
5. Haz una pagina suficientemente larga y prueba el avance al hacer scroll.

Errores comunes y como corregirlos

Error	Causa probable	Solucion
La barra no se mueve	La pagina no tiene altura suficiente	Agrega contenido o prueba en pagina larga.
No se ve	z-index bajo o mismo color que fondo	Sube z-index y cambia background.
Llega a mas de 100%	Calculo de altura incorrecto	Usa scrollHeight - innerHeight como en el ejemplo.

Checklist de prueba

- Al inicio la barra esta en 0%.
- Al final de la pagina llega a 100%.
- No tapa botones importantes.
- Funciona con navbar fija.

Adaptaciones profesionales recomendadas

- Oculta la barra si la pagina no tiene scroll.
- Cambia el color de la barra segun seccion activa.
- Combinala con lectura de articulos largos o documentacion.

8. Back to top

El boton Back to top permite volver rapidamente al inicio de la pagina. Mejora la experiencia en paginas largas y se suele mostrar solo cuando el usuario ha bajado cierta distancia.

Donde se usa en paginas reales: blogs, documentacion, portfolios largos, tiendas, landing pages extensas y guias tecnicas.

Dependencias internas: necesita un boton fijo y detectar el scroll para mostrarlo u ocultarlo.

Idea para reciclar: copia solo el bloque HTML que corresponde al componente, copia las reglas CSS indicadas y copia la clase JS modular o la funcion normal. Luego cambia IDs, clases y textos segun tu proyecto.

Parte	Que debes copiar	Que puedes cambiar sin romperlo
HTML	Estructura visible del componente y atributos data-*	Textos, iconos, rutas, nombres de secciones, opciones y labels.
CSS	Clases del componente y, si aplica, variables de color.	Colores, blur, radios, espacios, sombras, tiempos de transicion.
JS modular	Clase exportada con constructor e inicializacion.	Selectores, nombres de clase activa, claves localStorage y comportamiento.
JS normal	Funcion inicializadora sin import/export.	Selectores y condiciones, igual que en la version modular.

Codigo HTML minimo

HTML que debes llevar a index.html

```
<button type="button" class="back-to-top" id="backToTop" aria-label="Volver arriba">
  ↑
</button>
```

Codigo CSS minimo

CSS que debes llevar a styles.css

```

.back-to-top {
  position: fixed;
  right: 22px;
  bottom: 22px;
  z-index: 1300;
  width: 48px;
  height: 48px;
  border: 0;
  border-radius: 50%;
  color: #ffffff;
  background: linear-gradient(135deg, #2563eb, #7c3aed);
  box-shadow: 0 14px 35px rgba(37, 99, 235, 0.35);
  cursor: pointer;
  opacity: 0;
  transform: translateY(12px);
  pointer-events: none;
  transition: opacity 0.2s ease, transform 0.2s ease;
}

.back-to-top.is-visible {
  opacity: 1;
  transform: translateY(0);
  pointer-events: auto;
}

.back-to-top:hover {
  transform: translateY(-3px);
}

```

JavaScript modular con export/import

funciones.js - clase o funcion exportada

```

export class BackToTop {
  constructor(config = {}) {
    this.button = document.getElementById(config.buttonId || "backToTop");
    this.visibleAt = config.visibleAt || 360;
    this.init();
  }

  init() {
    window.addEventListener("scroll", () => this.toggle(), { passive: true });
    this.button?.addEventListener("click", () => {
      window.scrollTo({ top: 0, behavior: "smooth" });
    });
    this.toggle();
  }

  toggle() {
    const shouldShow = window.scrollY > this.visibleAt;
    this.button?.classList.toggle("is-visible", shouldShow);
  }
}

```

script.js - como llamarlo

```

// script.js
import { BackToTop } from './funciones.js';

new BackToTop();

```

JavaScript normal sin modulos

script-normal.js - version directa

```
function initBackToTop(buttonId = "backToTop", visibleAt = 360) {
  const button = document.getElementById(buttonId);

  function toggle() {
    const shouldShow = window.scrollY > visibleAt;
    button?.classList.toggle("is-visible", shouldShow);
  }

  window.addEventListener("scroll", toggle, { passive: true });
  button?.addEventListener("click", () => {
    window.scrollTo({ top: 0, behavior: "smooth" });
  });

  toggle();
}

initBackToTop();
```

Variables, clases, IDs y atributos que debes cambiar

Elemento	Donde esta	Como adaptarlo
#backToTop	HTML/JS	ID del boton. Cambialo junto con el constructor.
visibleAt	JS	Cantidad de pixeles bajados antes de mostrar el boton.
right/bottom	CSS	Ubicacion del boton en pantalla.
behavior: smooth	JS	Hace que el regreso sea suave. Puedes quitarlo si quieres inmediato.
↑	HTML	Puedes reemplazarlo por un icono SVG.

Paso a paso para implementarlo en otro proyecto

1. Coloca el boton al final del body.
2. Copia el CSS .back-to-top.
3. Copia BackToTop a funciones.js.
4. Importa y ejecuta new BackToTop({ visibleAt: 360 }).
5. Baja la pagina y confirma que el boton aparece.

Errores comunes y como corregirlos

Error	Causa probable	Solucion
El boton nunca aparece	Pagina muy corta o visibleAt muy alto	Baja visibleAt a 100 para probar.
No sube al inicio	El listener no carga o id incorrecto	Revisa #backToTop y que script este al final.
Tapa contenido movil	bottom/right mal ubicados	Ajusta posicion con media queries.

Checklist de prueba

- No se ve al cargar arriba.
- Aparece al hacer scroll hacia abajo.

- Al hacer clic vuelve al inicio suavemente.
- No se cruza con botones de chat o footer.

Adaptaciones profesionales recomendadas

- Ocultalo cuando el usuario este en el footer si tapa contenido.
- Agrega aria-label descriptivo para lectores de pantalla.
- Usa CSS prefers-reduced-motion para usuarios que prefieren menos animaciones.

9. Reveal on scroll

Reveal on scroll muestra elementos con animacion cuando aparecen en pantalla. Es muy usado para que cards, secciones, imagenes y textos entren suavemente mientras el usuario navega.

Donde se usa en paginas reales: landing pages, portafolios, secciones de servicios, tarjetas de productos, galerias y storytelling.

Dependencias internas: usa la API IntersectionObserver y una clase .reveal en los elementos que quieres animar.

Idea para reciclar: copia solo el bloque HTML que corresponde al componente, copia las reglas CSS indicadas y copia la clase JS modular o la funcion normal. Luego cambia IDs, clases y textos segun tu proyecto.

Parte	Que debes copiar	Que puedes cambiar sin romperlo
HTML	Estructura visible del componente y atributos data-*	Textos, iconos, rutas, nombres de secciones, opciones y labels.
CSS	Clases del componente y, si aplica, variables de color.	Colores, blur, radios, espacios, sombras, tiempos de transicion.
JS modular	Clase exportada con constructor e inicializacion.	Selectores, nombres de clase activa, claves localStorage y comportamiento.
JS normal	Funcion inicializadora sin import/export.	Selectores y condiciones, igual que en la version modular.

Codigo HTML minimo

HTML que debes llevar a index.html

```
<section class="card reveal">
  <h2>Servicio destacado</h2>
  <p>Este bloque aparecera suavemente cuando entre en pantalla.</p>
</section>

<div class="features">
  <article class="feature-card reveal">Card 1</article>
  <article class="feature-card reveal">Card 2</article>
  <article class="feature-card reveal">Card 3</article>
</div>
```

Codigo CSS minimo

CSS que debes llevar a styles.css

```

.reveal {
  opacity: 0;
  transform: translateY(22px);
  transition: opacity 0.55s ease, transform 0.55s ease;
}

.reveal.is-visible {
  opacity: 1;
  transform: translateY(0);
}

@media (prefers-reduced-motion: reduce) {
  .reveal {
    opacity: 1;
    transform: none;
    transition: none;
  }
}

```

JavaScript modular con export/import

funciones.js - clase o funcion exportada

```

export class RevealOnScroll {
  constructor(selector = ".reveal") {
    this.elements = document.querySelectorAll(selector);
    this.init();
  }

  init() {
    if (!('IntersectionObserver' in window)) {
      this.elements.forEach((el) => el.classList.add("is-visible"));
      return;
    }

    const observer = new IntersectionObserver((entries) => {
      entries.forEach((entry) => {
        if (entry.isIntersecting) {
          entry.target.classList.add("is-visible");
          observer.unobserve(entry.target);
        }
      });
    }, { threshold: 0.18 });

    this.elements.forEach((element) => observer.observe(element));
  }
}

```

script.js - como llamarlo

```

// script.js
import { RevealOnScroll } from './funciones.js';

new RevealOnScroll();

```

JavaScript normal sin modulos

script-normal.js - version directa

```
function initRevealOnScroll(selector = ".reveal") {
  const elements = document.querySelectorAll(selector);

  if (!('IntersectionObserver' in window)) {
    elements.forEach((el) => el.classList.add("is-visible"));
    return;
  }

  const observer = new IntersectionObserver((entries) => {
    entries.forEach((entry) => {
      if (entry.isIntersecting) {
        entry.target.classList.add("is-visible");
        observer.unobserve(entry.target);
      }
    });
  }, { threshold: 0.18 });

  elements.forEach((element) => observer.observe(element));
}

initRevealOnScroll();
```

Variables, clases, IDs y atributos que debes cambiar

Elemento	Donde esta	Como adaptarlo
.reveal	HTML/CSS/JS	Agrega esta clase a cualquier elemento que quieras animar.
.is-visible	CSS/JS	Clase que se agrega cuando el elemento entra en pantalla.
translateY(22px)	CSS	Distancia desde donde entra. Puedes usar 12px o 40px.
threshold	JS	Porcentaje visible necesario para activar la animacion.
unobserve	JS	Evita repetir la animacion. Quitalo si quieres que se repita.

Paso a paso para implementarlo en otro proyecto

1. Agrega class='reveal' a las cards o secciones.
2. Copia CSS de .reveal y .reveal.is-visible.
3. Copia RevealOnScroll en funciones.js.
4. Ejecuta new RevealOnScroll().
5. Haz scroll y verifica que los elementos aparecen al entrar en pantalla.

Errores comunes y como corregirlos

Error	Causa probable	Solucion
Los elementos nunca aparecen	JS no cargo o selector incorrecto	Revisa que tengan clase .reveal y que se ejecute el script.
Aparecen de una vez	Ya tienen .is-visible en HTML	Deja .is-visible solo para elementos que quieras visibles desde el inicio.
Animacion molesta	Transicion exagerada	Reduce translateY y duracion.

Checklist de prueba

- Elementos ocultos aparecen al hacer scroll.
- No quedan textos invisibles permanentemente.
- Funciona en varias secciones.
- Respeta prefers-reduced-motion.

Adaptaciones profesionales recomendadas

- Agrega variantes: .reveal-left, .reveal-right, .reveal-scale.
- Usa delay con style='--delay: 0.2s' para animaciones escalonadas.
- No abuses: anima solo elementos importantes para no saturar la experiencia.

10. Contador animado

El contador animado cuenta desde cero hasta un valor objetivo cuando el elemento aparece en pantalla. Se usa mucho para indicadores, metricas, logros, estadísticas, KPIs y secciones de impacto.

Donde se usa en paginas reales: landing pages, dashboards, secciones de logros, estadísticas de marca, paneles de datos y sitios institucionales.

Dependencias internas: usa atributos data-counter y data-target; puede apoyarse en IntersectionObserver para iniciar solo cuando se vea.

Idea para reciclar: copia solo el bloque HTML que corresponde al componente, copia las reglas CSS indicadas y copia la clase JS modular o la funcion normal. Luego cambia IDs, clases y textos segun tu proyecto.

Parte	Que debes copiar	Que puedes cambiar sin romperlo
HTML	Estructura visible del componente y atributos data-.*.	Textos, iconos, rutas, nombres de secciones, opciones y labels.
CSS	Clases del componente y, si aplica, variables de color.	Colores, blur, radios, espacios, sombras, tiempos de transicion.
JS modular	Clase exportada con constructor e inicializacion.	Selectores, nombres de clase activa, claves localStorage y comportamiento.
JS normal	Funcion inicializadora sin import/export.	Selectores y condiciones, igual que en la version modular.

Codigo HTML minimo

HTML que debes llevar a index.html

```
<section class="stats">
  <article class="stat-card reveal">
    <strong data-counter data-target="1200">0</strong>
    <span>Usuarios registrados</span>
  </article>

  <article class="stat-card reveal">
    <strong data-counter data-target="98">0</strong>
    <span>Porcentaje de satisfaccion</span>
  </article>

  <article class="stat-card reveal">
    <strong data-counter data-target="50">0</strong>
    <span>Componentes reutilizables</span>
  </article>
</section>
```

Codigo CSS minimo

CSS que debes llevar a styles.css

```
.stats {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(180px, 1fr));
  gap: 16px;
}

.stat-card {
  padding: 20px;
  border-radius: 18px;
  background: rgba(255, 255, 255, 0.12);
  border: 1px solid rgba(255, 255, 255, 0.18);
  text-align: center;
}

.stat-card strong {
  display: block;
  font-size: clamp(32px, 6vw, 56px);
  line-height: 1;
  color: #ffffff;
}

.stat-card span {
  display: block;
  margin-top: 8px;
  color: rgba(255, 255, 255, 0.72);
}
```

JavaScript modular con export/import

funciones.js - clase o funcion exportada

```

export class AnimatedCounters {
  constructor(selector = "[data-counter]") {
    this.counters = document.querySelectorAll(selector);
    this.init();
  }

  init() {
    if (!('IntersectionObserver' in window)) {
      this.counters.forEach((counter) => this.animate(counter));
      return;
    }

    const observer = new IntersectionObserver((entries) => {
      entries.forEach((entry) => {
        if (entry.isIntersecting) {
          this.animate(entry.target);
          observer.unobserve(entry.target);
        }
      });
    }, { threshold: 0.35 });

    this.counters.forEach((counter) => observer.observe(counter));
  }

  animate(counter) {
    const target = Number(counter.dataset.target || 0);
    const duration = Number(counter.dataset.duration || 1200);
    const startTime = performance.now();

    const update = (currentTime) => {
      const elapsed = currentTime - startTime;
      const progress = Math.min(elapsed / duration, 1);
      const value = Math.floor(progress * target);
      counter.textContent = value.toLocaleString("es-CO");

      if (progress < 1) {
        requestAnimationFrame(update);
      } else {
        counter.textContent = target.toLocaleString("es-CO");
      }
    };

    requestAnimationFrame(update);
  }
}

```

script.js - como llamarlo

```

// script.js
import { AnimatedCounters } from './funciones.js';

new AnimatedCounters();

```

JavaScript normal sin modulos

script-normal.js - version directa

```
function initAnimatedCounters(selector = "[data-counter]") {
  const counters = document.querySelectorAll(selector);

  function animate(counter) {
    const target = Number(counter.dataset.target || 0);
    const duration = Number(counter.dataset.duration || 1200);
    const startTime = performance.now();

    function update(currentTime) {
      const elapsed = currentTime - startTime;
      const progress = Math.min(elapsed / duration, 1);
      const value = Math.floor(progress * target);
      counter.textContent = value.toLocaleString("es-CO");

      if (progress < 1) {
        requestAnimationFrame(update);
      } else {
        counter.textContent = target.toLocaleString("es-CO");
      }
    }

    requestAnimationFrame(update);
  }

  if (!('IntersectionObserver' in window)) {
    counters.forEach(animate);
    return;
  }

  const observer = new IntersectionObserver((entries) => {
    entries.forEach((entry) => {
      if (entry.isIntersecting) {
        animate(entry.target);
        observer.unobserve(entry.target);
      }
    });
  }, { threshold: 0.35 });

  counters.forEach((counter) => observer.observe(counter));
}

initAnimatedCounters();
```

Variables, clases, IDs y atributos que debes cambiar

Elemento	Donde esta	Como adaptarlo
data-counter	HTML/JS	Marca los numeros que deben animarse.
data-target	HTML/JS	Numero final al que debe llegar el contador.
data-duration	HTML/JS	Duracion opcional en milisegundos; por defecto 1200.
toLocaleString('es-CO')	JS	Formato de miles. Cambia locale si necesitas otro pais.
threshold	JS	Define que tanto debe verse el contador para iniciar.

Paso a paso para implementarlo en otro proyecto

1. Crea una tarjeta o bloque con un elemento numerico.
2. Agrega data-counter y data-target='numero'.
3. Copia el CSS de .stats y .stat-card o adaptalo a tus cards existentes.
4. Copia AnimatedCounters a funciones.js.

5. Ejecuta `new AnimatedCounters()`.
6. Haz scroll hasta la seccion y verifica que cuenta hasta el valor final.

Errores comunes y como corregirlos

Error	Causa probable	Solucion
Muestra NaN	data-target no es numerico	Usa data-target='1200' sin puntos ni comas.
Cuenta demasiado rapido	duration bajo	Agrega data-duration='2200'.
Se repite muchas veces	No se desobserva el elemento	Mantén <code>observer.unobserve(entry.target)</code> .

Checklist de prueba

- Los numeros empiezan en 0.
- Llegan al valor exacto de data-target.
- La animacion inicia al entrar en pantalla.
- No aparece NaN ni valores negativos no deseados.

Adaptaciones profesionales recomendadas

- Agrega sufijos visuales como %, + o K fuera del data-target para no romper el numero.
- Usa easing para que el contador desacelere al final.
- Conecta los valores a datos reales si luego consumes una API.

Resumen final para reciclar las funcionalidades 1 a 10

Antes de copiar código a un proyecto real, revisa esta lista. Te ayuda a evitar los errores más comunes cuando reutilizas componentes.

Pregunta	Respuesta esperada
¿El HTML tiene los IDs correctos?	Cada id usado por JS existe una sola vez en la página.
¿Las clases CSS coinciden?	Las clases del HTML son iguales a las reglas del CSS.
¿Los data-* coinciden?	data-section debe coincidir con el id de la sección.
¿Uso versión modular o normal?	Solo una versión por página para no duplicar eventos.
¿El componente tiene estado?	Clases como is-open, is-visible o is-active deben ser controladas por JS.
¿Se probó en móvil?	Navbar, sidebar y dropdown no deben salirse de pantalla.
¿Hay accesibilidad básica?	Botones reales, aria-label cuando haga falta y cierre con Escape.

Con estas 10 funcionalidades ya puedes construir la base de una interfaz profesional: navegación fija, menú lateral, rutas internas, tema claro/oscuro, indicadores de scroll y animaciones de entrada.

Guía de reciclaje de componentes web - Funcionalidades 10 a 25

Extensión del documento 1. Mantiene el mismo formato: explicación, HTML, CSS, JavaScript modular, JavaScript normal, variables que debes cambiar, pasos, errores comunes y checklist.

Este documento sigue el orden de la tabla de contenido del recurso unificado. En esa tabla, la funcionalidad 10 es Contador animado y la 25 es Acordeón. Por eso esta guía cubre exactamente ese bloque para que puedas continuar después del documento 1 sin perder la numeración original.

#	Funcionalidad	Uso principal
10	Contador animado	Cuenta visualmente desde cero hasta un valor objetivo. Es útil para secciones de métricas, dashboards, estadísticas, resultados, landing pages y tarje...
11	Login demo	Crea una pantalla o tarjeta de inicio de sesión para practicar formularios, validación, cambios de estado y almacenamiento de sesión. En un proyecto r...
12	Logout demo	Cierra la sesión simulada y limpia datos temporales del navegador. Se complementa con el login demo y con componentes que muestran contenido solo cuan...
13	Recordar usuario	Guarda el correo, nombre o preferencia del usuario para rellenar campos automáticamente. Es común en formularios de login, preferencias, temas visuale...
14	Password toggle	Permite mostrar u ocultar una contraseña. Es una funcionalidad muy común en formularios de login, registro y cambio de contraseña.
15	Validación de formulario	Valida campos obligatorios, correo, longitud mínima y reglas personalizadas antes de procesar o enviar datos.
16	Mensajes por campo	Muestra errores o ayudas debajo de cada input. Complementa la validación y hace que el usuario sepa exactamente qué corregir.
17	Character counter	Cuenta los caracteres de un textarea o input. Se usa mucho en comentarios, bios, descripciones, formularios de contacto y publicaciones.
18	Range preview	Muestra en tiempo real el valor de un input type="range". Es útil para filtros de precio, volumen, porcentaje, rating, tamaños, cantidades y configura...
19	File info	Muestra el nombre, tamaño y tipo de archivo seleccionado. Es útil en formularios con carga de CV, imágenes, documentos, facturas o evidencias.
20	Autosave draft	Guarda automáticamente un borrador mientras el usuario escribe. Evita pérdida de información si se cierra la página por accidente.
21	Stepper / Wizard	Divide un proceso en pasos. Es común en formularios largos, registros, compras, onboarding y creación de perfiles.
22	Skeleton loader	Muestra una estructura de carga temporal antes de renderizar datos reales. Es muy usado en páginas modernas para que la espera se sienta más profesio...
23	Select personalizado	Reemplaza el select nativo por un selector bonito, controlable y editable. Permite opciones con estilo, flecha personalizada, blur y mejor integración...

#	Funcionalidad	Uso principal
24	Tabs / pestañas	Permite cambiar entre paneles internos sin recargar la página. Se usa en perfiles, productos, dashboards, documentación y configuraciones.
25	Acordeón / FAQ	Abre y cierra bloques de contenido. Es perfecto para preguntas frecuentes, secciones de ayuda, detalles de producto y menús informativos.

Cómo usar este documento

- Primero identifica la funcionalidad que quieres reutilizar según el número y el nombre.
- Copia el HTML mínimo en tu index.html o en la sección donde lo necesites.
- Copia el CSS en styles.css. Si ya tienes variables globales, adapta colores, radios y espacios.
- Elige solo una versión de JavaScript: modular con funciones.js + script.js, o normal con script-normal.js.
- Cambia los ids, clases y data-* exactamente donde se indique en la tabla de variables.
- Prueba la funcionalidad sola antes de mezclarla con otros componentes.

10. Contador animado

¿Qué hace?

Cuenta visualmente desde cero hasta un valor objetivo. Es útil para secciones de métricas, dashboards, estadísticas, resultados, landing pages y tarjetas tipo "clientes atendidos", "proyectos", "ventas" o "porcentaje de avance".

¿Cuándo usarlo?

Úsalo cuando quieras mostrar un número de forma más atractiva que un texto estático. También sirve para practicar manipulación del DOM, intervalos de tiempo y atributos data-.*.

Estructura que debes copiar

HTML mínimo

```
<section class="stats-grid">
  <article class="stat-card reveal">
    <span class="stat-card__number" data-counter data-target="150"
      data-suffix="+">0</span>
    <p>Proyectos realizados</p>
  </article>

  <article class="stat-card reveal">
    <span class="stat-card__number" data-counter data-target="98"
      data-suffix="%">0</span>
    <p>Satisfacción</p>
  </article>
</section>
```

CSS necesario

```
.stats-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(180px, 1fr));
  gap: 1rem;
}

.stat-card {
  padding: 1.4rem;
  border-radius: 1.2rem;
  background: rgba(255, 255, 255, 0.12);
  border: 1px solid rgba(255, 255, 255, 0.22);
  backdrop-filter: blur(14px);
  text-align: center;
}

.stat-card__number {
  display: block;
  font-size: clamp(2rem, 5vw, 3rem);
  font-weight: 800;
  color: var(--primary, #2563eb);
}
```

funciones.js - versión modular con export

```
export class AnimatedCounters {
  constructor(selector = "[data-counter]", duration = 1200) {
    this.counters = document.querySelectorAll(selector);
    this.duration = duration;
    this.start();
  }

  start() {
    this.counters.forEach((counter) => this.animate(counter));
  }

  animate(counter) {
    const target = Number(counter.dataset.target);
    const suffix = counter.dataset.suffix || "";
    const startTime = performance.now();

    const update = (currentTime) => {
      const elapsed = currentTime - startTime;
      const progress = Math.min(elapsed / this.duration, 1);
      const value = Math.floor(progress * target);

      counter.textContent = `${value}${suffix}`;

      if (progress < 1) {
        requestAnimationFrame(update);
      }
    };

    requestAnimationFrame(update);
  }
}
```

script.js - importación e inicialización

```
import { AnimatedCounters } from "./funciones.js";

new AnimatedCounters("[data-counter]", 1400);
```

script-normal.js - versión normal sin módulos

```
const counters = document.querySelectorAll("[data-counter]");

counters.forEach((counter) => {
  const target = Number(counter.dataset.target);
  const suffix = counter.dataset.suffix || "";
  let current = 0;
  const step = Math.ceil(target / 60);

  const interval = setInterval(() => {
    current += step;
    if (current >= target) {
      current = target;
      clearInterval(interval);
    }
    counter.textContent = `${current}${suffix}`;
  }, 20);
});
```

Variables, clases e IDs que puedes cambiar

Elemento	Para qué sirve	Cómo adaptarlo
data-target	Número final del contador.	Cámbialo por el valor que quieras mostrar.
data-suffix	Símbolo que aparece al final.	Puedes usar +, %, K, M, COP o dejarlo vacío.
.stat-card	Tarjeta visual de la métrica.	Puedes cambiar colores, fondo, blur y bordes.
duration	Duración de la animación.	Aumenta el valor para que cuente más lento.

Paso a paso para reciclarlo en otro proyecto

1. Copia el HTML del contador dentro de la sección donde mostrarás las métricas.
2. Ajusta data-target según el número final que necesitas.
3. Agrega o quita data-suffix según quieras mostrar %, + u otro símbolo.
4. Copia el CSS de .stats-grid y .stat-card en tu archivo styles.css.
5. En funciones.js exporta la clase AnimatedCounters.
6. En script.js importa la clase y crea una instancia con new AnimatedCounters().
7. Prueba que los números avancen al cargar la página.

Qué debes revisar al integrarlo

- Que los ids del HTML coincidan con los ids usados en JavaScript.
- Que no estés usando el mismo id dos veces en la misma página.
- Que las clases activas como is-active, is-open, is-valid o is-invalid estén escritas igual en HTML, CSS y JS.
- Que si usas módulos el script se cargue con type="module".
- Que si usas la versión normal no dejes imports ni exports en el archivo.

Errores comunes

- Poner data-target con texto en vez de número.
- Olvidar importar la clase si usas módulos.
- Usar demasiados contadores con intervalos pesados; requestAnimationFrame es más fluido.

Cómo cambiar nombres sin romper la funcionalidad

Paso	Qué hacer	Recomendación
1	Cambia primero el id en el HTML.	Ejemplo: loginForm por formAcceso.
2	Busca ese mismo id en script.js o script-normal.js.	Debe quedar exactamente igual.
3	Si el CSS usa una clase, cambia la clase en HTML y CSS al mismo tiempo.	No cambies solo uno de los dos archivos.
4	Si hay data-* como data-tab, data-step o data-value, revisa qué lo lee en JS.	Los atributos data-* suelen ser el puente entre HTML y JS.
5	Prueba la funcionalidad aislada antes de mezclarla con otros componentes.	Así sabes si el error viene del componente o de otra parte.

Adaptaciones comunes en proyectos reales

- Para una landing page: usa Contador animado dentro de una sección con clase .section y manten el diseño centrado.
- Para un dashboard: coloca Contador animado dentro de una card reutilizable y controla el estado con clases como is-active o is-open.
- Para un formulario: valida ids, required, minlength y data-error antes de enviar datos.
- Para un proyecto con muchos componentes: inicializa esta funcionalidad desde una función init10() o desde initApp().
- Para trabajo en equipo: deja comentarios cortos indicando qué HTML, CSS y JS pertenecen a Contador animado.

Mini guía de depuración si no funciona

1. Abre la consola del navegador con F12 y revisa si aparece un error rojo.
2. Si el error dice null, significa que JavaScript no encontró un elemento del DOM; revisa ids y clases.
3. Si el diseño no cambia, revisa que la clase que agrega JavaScript exista también en CSS.

4. Si usas import/export, abre el proyecto con Live Server; no lo abras solo con doble clic.
5. Si el componente depende de localStorage o sessionStorage, limpia el almacenamiento del navegador y prueba de nuevo.
6. Comenta temporalmente otros componentes para confirmar que no hay conflicto de nombres.

Patrón mental para reutilizarlo

Cada funcionalidad de esta guía se puede entender como una combinación de tres capas: la capa HTML define la estructura, la capa CSS define el diseño visual y la capa JavaScript controla el comportamiento. Para reciclarla bien, no copies solo una capa: copia la estructura mínima, los estilos necesarios y la inicialización correspondiente. Después, cambia nombres y valores de forma ordenada.

Consejo de adaptación profesional

Si quieres que el contador se active solo cuando aparece en pantalla, combínalo con IntersectionObserver o con la funcionalidad Reveal on scroll del documento anterior.

11. Login demo

¿Qué hace?

Crea una pantalla o tarjeta de inicio de sesión para practicar formularios, validación, cambios de estado y almacenamiento de sesión. En un proyecto real se conecta con backend, API o Firebase.

¿Cuándo usarlo?

Úsalo en portafolios, dashboards, intranets de práctica, paneles administrativos y proyectos donde quieras simular ingreso de usuario.

Estructura que debes copiar

HTML mínimo

```
<section class="auth-card" id="loginSection">
  <h2>Iniciar sesión</h2>

  <form id="loginForm" novalidate>
    <label for="loginEmail">Correo</label>
    <input id="loginEmail" type="email" placeholder="correo@ejemplo.com" required>
    <small class="field-error" data-error="loginEmail"></small>

    <label for="loginPassword">Contraseña</label>
    <input id="loginPassword" type="password" placeholder="Mínimo 6 caracteres"
      required minlength="6">
    <small class="field-error" data-error="loginPassword"></small>

    <button type="submit">Entrar</button>
  </form>

  <p id="loginMessage" class="form-message"></p>
</section>
```

CSS necesario

```
.auth-card {
  max-width: 420px;
  margin: 2rem auto;
  padding: 2rem;
  border-radius: 1.5rem;
  background: rgba(255, 255, 255, 0.12);
  border: 1px solid rgba(255, 255, 255, 0.2);
  backdrop-filter: blur(16px);
}

.auth-card label {
  display: block;
  margin-top: 1rem;
  margin-bottom: .4rem;
  font-weight: 700;
}

.auth-card input {
  width: 100%;
  padding: .85rem 1rem;
  border-radius: .9rem;
  border: 1px solid rgba(255, 255, 255, 0.28);
  background: rgba(255, 255, 255, 0.12);
  color: inherit;
}

.field-error {
  display: block;
  min-height: 1rem;
  margin-top: .3rem;
  color: #fca5a5;
  font-size: .82rem;
}

.form-message.is-success { color: #86efac; }
.form-message.is-error { color: #fca5a5; }
```

funciones.js - versión modular con export

```
export class LoginDemo {
  constructor(formId, messageId) {
    this.form = document.getElementById(formId);
    this.message = document.getElementById(messageId);
    this.init();
  }

  init() {
    this.form.addEventListener("submit", (event) => {
      event.preventDefault();
      this.login();
    });
  }

  login() {
    const email = this.form.querySelector("#loginEmail").value.trim();
    const password = this.form.querySelector("#loginPassword").value.trim();

    if (!email || !password) {
      this.showMessage("Completa todos los campos.", "error");
      return;
    }

    if (password.length < 6) {
      this.showMessage("La contraseña debe tener mínimo 6 caracteres.",
        "error");
      return;
    }

    const user = { email, loggedAt: new Date().toISOString() };
    sessionStorage.setItem("kit-session-user", JSON.stringify(user));
    this.showMessage("Ingreso correcto. Sesión demo guardada.", "success");
  }

  showMessage(text, type) {
    this.message.textContent = text;
    this.message.className = `form-message is-${type}`;
  }
}
```

script.js - importación e inicialización

```
import { LoginDemo } from "./funciones.js";

new LoginDemo("loginForm", "loginMessage");
```

script-normal.js - versión normal sin módulos

```
const loginForm = document.getElementById("loginForm");
const loginMessage = document.getElementById("loginMessage");

loginForm.addEventListener("submit", (event) => {
  event.preventDefault();

  const email = document.getElementById("loginEmail").value.trim();
  const password = document.getElementById("loginPassword").value.trim();

  if (!email || !password) {
    loginMessage.textContent = "Completa todos los campos.";
    loginMessage.className = "form-message is-error";
    return;
  }

  sessionStorage.setItem("kit-session-user", JSON.stringify({ email }));
  loginMessage.textContent = "Ingreso correcto.";
  loginMessage.className = "form-message is-success";
});
```

Variables, clases e IDs que puedes cambiar

Elemento	Para qué sirve	Cómo adaptarlo
#loginForm	Formulario que escucha el submit.	Cambia el id si tu formulario tiene otro nombre.
#loginEmail	Campo de correo.	Debe coincidir con el selector usado en JS.
#loginPassword	Campo de contraseña.	Puedes cambiar minlength según tu regla.
kit-session-user	Clave de sessionStorage.	Cámbiala si usas varios proyectos.

Paso a paso para reciclarlo en otro proyecto

1. Copia la tarjeta de login en la página donde quieres simular el acceso.
2. Verifica que el formulario tenga id="loginForm" o ajusta el id en JavaScript.
3. Conecta el CSS de .auth-card, .field-error y .form-message.
4. Si usas módulos, exporta LoginDemo en funciones.js e impórtalo en script.js.
5. Prueba enviar el formulario vacío, con contraseña corta y con datos válidos.
6. Para producción reemplaza la simulación por una llamada fetch a tu backend.

Qué debes revisar al integrarlo

- Que los ids del HTML coincidan con los ids usados en JavaScript.
- Que no estés usando el mismo id dos veces en la misma página.
- Que las clases activas como is-active, is-open, is-valid o is-invalid estén escritas igual en HTML, CSS y JS.
- Que si usas módulos el script se cargue con type="module".
- Que si usas la versión normal no dejes imports ni exports en el archivo.

Errores comunes

- Creer que este login es seguridad real.
- No usar event.preventDefault() y que la página se recargue.
- Repetir ids si hay más de un login.

Cómo cambiar nombres sin romper la funcionalidad

Paso	Qué hacer	Recomendación
1	Cambia primero el id en el HTML.	Ejemplo: loginForm por formAcceso.
2	Busca ese mismo id en script.js o script-normal.js.	Debe quedar exactamente igual.
3	Si el CSS usa una clase, cambia la clase en HTML y CSS al mismo tiempo.	No cambies solo uno de los dos archivos.
4	Si hay data-* como data-tab, data-step o data-value, revisa qué lo lee en JS.	Los atributos data-* suelen ser el puente entre HTML y JS.
5	Prueba la funcionalidad aislada antes de mezclarla con otros componentes.	Así sabes si el error viene del componente o de otra parte.

Adaptaciones comunes en proyectos reales

- Para una landing page: usa Login demo dentro de una sección con clase .section y manten el diseño centrado.
- Para un dashboard: coloca Login demo dentro de una card reutilizable y controla el estado con clases como is-active o is-open.

- Para un formulario: valida ids, required, minlength y data-error antes de enviar datos.
- Para un proyecto con muchos componentes: inicializa esta funcionalidad desde una función `init11()` o desde `initApp()`.
- Para trabajo en equipo: deja comentarios cortos indicando qué HTML, CSS y JS pertenecen a Login demo.

Mini guía de depuración si no funciona

1. Abre la consola del navegador con F12 y revisa si aparece un error rojo.
2. Si el error dice null, significa que JavaScript no encontró un elemento del DOM; revisa ids y clases.
3. Si el diseño no cambia, revisa que la clase que agrega JavaScript exista también en CSS.
4. Si usas import/export, abre el proyecto con Live Server; no lo abras solo con doble clic.
5. Si el componente depende de localStorage o sessionStorage, limpia el almacenamiento del navegador y prueba de nuevo.
6. Comenta temporalmente otros componentes para confirmar que no hay conflicto de nombres.

Patrón mental para reutilizarlo

Cada funcionalidad de esta guía se puede entender como una combinación de tres capas: la capa HTML define la estructura, la capa CSS define el diseño visual y la capa JavaScript controla el comportamiento. Para reciclarla bien, no copies solo una capa: copia la estructura mínima, los estilos necesarios y la inicialización correspondiente. Después, cambia nombres y valores de forma ordenada.

Consejo de adaptación profesional

No guardes contraseñas en localStorage ni sessionStorage. Para producción usa backend, tokens seguros, HTTPS y validación del servidor.

12. Logout demo

¿Qué hace?

Cierra la sesión simulada y limpia datos temporales del navegador. Se complementa con el login demo y con componentes que muestran contenido solo cuando existe usuario.

¿Cuándo usarlo?

Úsalo en dashboards de práctica, paneles demo y proyectos donde quieras simular la entrada y salida de un usuario.

Estructura que debes copiar

HTML mínimo

```
<div class="user-panel">
  <span id="userStatus">Sin sesión activa</span>
  <button id="logoutButton" type="button">Cerrar sesión</button>
</div>
```

CSS necesario

```
.user-panel {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 1rem;
  padding: 1rem;
  border-radius: 1rem;
  background: rgba(255, 255, 255, 0.1);
}

#logoutButton {
  border: 0;
  border-radius: .8rem;
  padding: .75rem 1rem;
  cursor: pointer;
  color: #fff;
  background: linear-gradient(135deg, #ef4444, #f97316);
}
```

funciones.js - versión modular con export

```
export class LogoutDemo {
  constructor(buttonId, statusId, storageKey = "kit-session-user") {
    this.button = document.getElementById(buttonId);
    this.status = document.getElementById(statusId);
    this.storageKey = storageKey;
    this.init();
  }

  init() {
    this.renderStatus();
    this.button.addEventListener("click", () => this.logout());
  }

  renderStatus() {
    const user = JSON.parse(sessionStorage.getItem(this.storageKey));
    this.status.textContent = user ? `Sesión: ${user.email}` : "Sin sesión activa";
  }

  logout() {
    sessionStorage.removeItem(this.storageKey);
    this.renderStatus();
  }
}
```

script.js - importación e inicialización

```
import { LogoutDemo } from "../funciones.js";

new LogoutDemo("logoutButton", "userStatus", "kit-session-user");
```

script-normal.js - versión normal sin módulos

```
const logoutButton = document.getElementById("logoutButton");
const userStatus = document.getElementById("userStatus");

function renderUserStatus() {
  const user = JSON.parse(sessionStorage.getItem("kit-session-user"));
  userStatus.textContent = user ? `Sesión: ${user.email}` : "Sin sesión activa";
}

logoutButton.addEventListener("click", () => {
  sessionStorage.removeItem("kit-session-user");
  renderUserStatus();
});

renderUserStatus();
```

Variables, clases e IDs que puedes cambiar

Elemento	Para qué sirve	Cómo adaptarlo
storageKey	Clave usada para guardar la sesión.	Debe ser la misma del login.
#logoutButton	Botón que cierra sesión.	Cambia el id si usas otro botón.
#userStatus	Texto donde se muestra el estado.	Puedes ponerlo en navbar o panel lateral.

Paso a paso para reciclarlo en otro proyecto

1. Agrega el panel o solo el botón de cerrar sesión donde lo necesites.
2. Asegúrate de usar la misma clave de sessionStorage que el login.
3. Copia el CSS o adapta el botón a tu diseño.
4. Inicializa la clase LogoutDemo después de LoginDemo.
5. Inicia sesión, recarga y verifica que el usuario aparezca.
6. Presiona cerrar sesión y revisa que el estado se actualice.

Qué debes revisar al integrarlo

- Que los ids del HTML coincidan con los ids usados en JavaScript.
- Que no estés usando el mismo id dos veces en la misma página.
- Que las clases activas como is-active, is-open, is-valid o is-invalid estén escritas igual en HTML, CSS y JS.
- Que si usas módulos el script se cargue con type="module".
- Que si usas la versión normal no dejes imports ni exports en el archivo.

Errores comunes

- Usar localStorage y sessionStorage mezclados sin control.
- Eliminar una clave diferente a la que guarda el login.
- No actualizar visualmente el estado después de cerrar sesión.

Cómo cambiar nombres sin romper la funcionalidad

Paso	Qué hacer	Recomendación
1	Cambia primero el id en el HTML.	Ejemplo: loginForm por formAcceso.

Paso	Qué hacer	Recomendación
2	Busca ese mismo id en script.js o script-normal.js.	Debe quedar exactamente igual.
3	Si el CSS usa una clase, cambia la clase en HTML y CSS al mismo tiempo.	No cambies solo uno de los dos archivos.
4	Si hay data-* como data-tab, data-step o data-value, revisa qué lo lee en JS.	Los atributos data-* suelen ser el puente entre HTML y JS.
5	Prueba la funcionalidad aislada antes de mezclarla con otros componentes.	Así sabes si el error viene del componente o de otra parte.

Adaptaciones comunes en proyectos reales

- Para una landing page: usa Logout demo dentro de una sección con clase .section y manten el diseño centrado.
- Para un dashboard: coloca Logout demo dentro de una card reutilizable y controla el estado con clases como is-active o is-open.
- Para un formulario: valida ids, required, minlength y data-error antes de enviar datos.
- Para un proyecto con muchos componentes: inicializa esta funcionalidad desde una función init12() o desde initApp().
- Para trabajo en equipo: deja comentarios cortos indicando qué HTML, CSS y JS pertenecen a Logout demo.

Mini guía de depuración si no funciona

1. Abre la consola del navegador con F12 y revisa si aparece un error rojo.
2. Si el error dice null, significa que JavaScript no encontró un elemento del DOM; revisa ids y clases.
3. Si el diseño no cambia, revisa que la clase que agrega JavaScript exista también en CSS.
4. Si usas import/export, abre el proyecto con Live Server; no lo abras solo con doble clic.
5. Si el componente depende de localStorage o sessionStorage, limpia el almacenamiento del navegador y prueba de nuevo.
6. Comenta temporalmente otros componentes para confirmar que no hay conflicto de nombres.

Patrón mental para reutilizarlo

Cada funcionalidad de esta guía se puede entender como una combinación de tres capas: la capa HTML define la estructura, la capa CSS define el diseño visual y la capa JavaScript controla el comportamiento. Para reciclarla bien, no copies solo una capa: copia la estructura mínima, los estilos necesarios y la inicialización correspondiente. Después, cambia nombres y valores de forma ordenada.

Consejo de adaptación profesional

Después de cerrar sesión en producción también debes invalidar el token del servidor o borrar cookies seguras si las usas.

13. Recordar usuario

¿Qué hace?

Guarda el correo, nombre o preferencia del usuario para rellenar campos automáticamente. Es común en formularios de login, preferencias, temas visuales y configuraciones.

¿Cuándo usarlo?

Úsalo cuando quieras mejorar experiencia de usuario sin pedir los mismos datos cada vez.

Estructura que debes copiar

HTML mínimo

```
<label class="remember-row">
  <input type="checkbox" id="rememberUser">
  Recordar mi correo
</label>
```

CSS necesario

```
.remember-row {
  display: flex;
  align-items: center;
  gap: .6rem;
  margin-top: .8rem;
  font-size: .9rem;
  color: var(--text-muted, #cbd5e1);
}

.remember-row input {
  width: 18px;
  height: 18px;
  accent-color: var(--primary, #2563eb);
}
```

funciones.js - versión modular con export

```
export class RememberUser {
  constructor(emailInputId, checkboxId, storageKey = "kit-remember-email") {
    this.emailInput = document.getElementById(emailInputId);
    this.checkbox = document.getElementById(checkboxId);
    this.storageKey = storageKey;
    this.init();
  }

  init() {
    const savedEmail = localStorage.getItem(this.storageKey);

    if (savedEmail) {
      this.emailInput.value = savedEmail;
      this.checkbox.checked = true;
    }
  }

  saveIfChecked() {
    if (this.checkbox.checked) {
      localStorage.setItem(this.storageKey, this.emailInput.value.trim());
    } else {
      localStorage.removeItem(this.storageKey);
    }
  }
}
```

script.js - importación e inicialización

```
import { RememberUser } from "../funciones.js";

const rememberUser = new RememberUser("loginEmail", "rememberUser");

// Dentro del submit del login, después de validar:
rememberUser.saveIfChecked();
```

script-normal.js - versión normal sin módulos

```
const rememberCheckbox = document.getElementById("rememberUser");
const emailInput = document.getElementById("loginEmail");
const savedEmail = localStorage.getItem("kit-remember-email");

if (savedEmail) {
  emailInput.value = savedEmail;
  rememberCheckbox.checked = true;
}

function saveRememberedEmail() {
  if (rememberCheckbox.checked) {
    localStorage.setItem("kit-remember-email", emailInput.value.trim());
  } else {
    localStorage.removeItem("kit-remember-email");
  }
}
```

Variables, clases e IDs que puedes cambiar

Elemento	Para qué sirve	Cómo adaptarlo
#rememberUser	Checkbox de recordar usuario.	Cambia el id si tienes varios formularios.
#loginEmail	Input que se rellena automáticamente.	Debe ser el campo de correo real.
kit-remember-email	Clave de localStorage.	Personalízala por proyecto.

Paso a paso para reciclarlo en otro proyecto

1. Agrega el checkbox debajo del correo en el login.
2. Copia el CSS para que el checkbox se alinee bien.
3. Crea la instancia de RememberUser en script.js.
4. Dentro del submit, llama saveIfChecked() después de validar los datos.
5. Prueba marcar el checkbox, iniciar sesión y recargar la página.
6. Prueba desmarcarlo y verificar que el correo ya no se guarde.

Qué debes revisar al integrarlo

- Que los ids del HTML coincidan con los ids usados en JavaScript.
- Que no estés usando el mismo id dos veces en la misma página.
- Que las clases activas como is-active, is-open, is-valid o is-invalid estén escritas igual en HTML, CSS y JS.
- Que si usas módulos el script se cargue con type="module".
- Que si usas la versión normal no dejes imports ni exports en el archivo.

Errores comunes

- Guardar contraseñas en localStorage: no se debe hacer.
- Llamar saveIfChecked antes de validar el correo.
- No cambiar la clave si mezclas varios proyectos.

Cómo cambiar nombres sin romper la funcionalidad

Paso	Qué hacer	Recomendación
1	Cambia primero el id en el HTML.	Ejemplo: loginForm por formAcceso.
2	Busca ese mismo id en script.js o script-normal.js.	Debe quedar exactamente igual.
3	Si el CSS usa una clase, cambia la clase en HTML y CSS al mismo tiempo.	No cambies solo uno de los dos archivos.
4	Si hay data-* como data-tab, data-step o data-value, revisa qué lo lee en JS.	Los atributos data-* suelen ser el puente entre HTML y JS.
5	Prueba la funcionalidad aislada antes de mezclarla con otros componentes.	Así sabes si el error viene del componente o de otra parte.

Adaptaciones comunes en proyectos reales

- Para una landing page: usa Recordar usuario dentro de una sección con clase .section y manten el diseño centrado.
- Para un dashboard: coloca Recordar usuario dentro de una card reutilizable y controla el estado con clases como is-active o is-open.
- Para un formulario: valida ids, required, minlength y data-error antes de enviar datos.
- Para un proyecto con muchos componentes: inicializa esta funcionalidad desde una función init13() o desde initApp().
- Para trabajo en equipo: deja comentarios cortos indicando qué HTML, CSS y JS pertenecen a Recordar usuario.

Mini guía de depuración si no funciona

1. Abre la consola del navegador con F12 y revisa si aparece un error rojo.
2. Si el error dice null, significa que JavaScript no encontró un elemento del DOM; revisa ids y clases.
3. Si el diseño no cambia, revisa que la clase que agrega JavaScript exista también en CSS.
4. Si usas import/export, abre el proyecto con Live Server; no lo abras solo con doble clic.
5. Si el componente depende de localStorage o sessionStorage, limpia el almacenamiento del navegador y prueba de nuevo.
6. Comenta temporalmente otros componentes para confirmar que no hay conflicto de nombres.

Patrón mental para reutilizarlo

Cada funcionalidad de esta guía se puede entender como una combinación de tres capas: la capa HTML define la estructura, la capa CSS define el diseño visual y la capa JavaScript controla el comportamiento. Para reciclarla bien, no copies solo una capa: copia la estructura mínima, los estilos necesarios y la inicialización correspondiente. Después, cambia nombres y valores de forma ordenada.

Consejo de adaptación profesional

Solo recuerda datos no sensibles. Recordar un correo es aceptable; recordar contraseñas en el navegador con JS no lo es.

14. Password toggle

¿Qué hace?

Permite mostrar u ocultar una contraseña. Es una funcionalidad muy común en formularios de login, registro y cambio de contraseña.

¿Cuándo usarlo?

Úsalo cuando tengas inputs type="password" y quieras mejorar la experiencia del usuario.

Estructura que debes copiar

HTML mínimo

```
<div class="password-field">
  <input id="loginPassword" type="password" placeholder="Contraseña">
  <button type="button" data-toggle-password="loginPassword">Mostrar</button>
</div>
```

CSS necesario

```
.password-field {
  display: flex;
  align-items: center;
  gap: .5rem;
}

.password-field input {
  flex: 1;
}

.password-field button {
  width: auto;
  min-height: auto;
  padding: .75rem .9rem;
  border-radius: .8rem;
  border: 1px solid rgba(255, 255, 255, 0.2);
  background: rgba(255, 255, 255, 0.12);
  color: inherit;
}
```

funciones.js - versión modular con export

```
export class PasswordToggle {
  constructor(selector = "[data-toggle-password]") {
    this.buttons = document.querySelectorAll(selector);
    this.init();
  }

  init() {
    this.buttons.forEach((button) => {
      button.addEventListener("click", () => this.toggle(button));
    });
  }

  toggle(button) {
    const inputId = button.dataset.togglePassword;
    const input = document.getElementById(inputId);
    const isHidden = input.type === "password";

    input.type = isHidden ? "text" : "password";
    button.textContent = isHidden ? "Ocultar" : "Mostrar";
  }
}
```

script.js - importación e inicialización

```
import { PasswordToggle } from "./funciones.js";

new PasswordToggle("[data-toggle-password]");
```


script-normal.js - versión normal sin módulos

```
document.querySelectorAll("[data-toggle-password]").forEach((button) => {
  button.addEventListener("click", () => {
    const input = document.getElementById(button.dataset.togglePassword);
    const isHidden = input.type === "password";

    input.type = isHidden ? "text" : "password";
    button.textContent = isHidden ? "Ocultar" : "Mostrar";
  });
});
```

Variables, clases e IDs que puedes cambiar

Elemento	Para qué sirve	Cómo adaptarlo
data-toggle-password	Debe contener el id del input password.	Si el input cambia de id, actualiza este valor.
#loginPassword	Input que cambia entre password y text.	Puedes usarlo en cualquier formulario.
.password-field	Contenedor visual.	Puedes modificar flex, gap y colores.

Paso a paso para reciclarlo en otro proyecto

1. Encierra el input y el botón dentro de .password-field.
2. El botón debe tener data-toggle-password con el id del input.
3. Copia el CSS para alinear el botón con el input.
4. Exporta PasswordToggle o copia la versión normal.
5. Prueba que el botón cambie de Mostrar a Ocultar.
6. Verifica que el valor de la contraseña no se borre al alternar.

Qué debes revisar al integrarlo

- Que los ids del HTML coincidan con los ids usados en JavaScript.
- Que no estés usando el mismo id dos veces en la misma página.
- Que las clases activas como is-active, is-open, is-valid o is-invalid estén escritas igual en HTML, CSS y JS.
- Que si usas módulos el script se cargue con type="module".
- Que si usas la versión normal no dejes imports ni exports en el archivo.

Errores comunes

- Poner data-toggle-password sin valor.
- Usar un id que no existe.
- Crear varios inputs con el mismo id.

Cómo cambiar nombres sin romper la funcionalidad

Paso	Qué hacer	Recomendación
1	Cambia primero el id en el HTML.	Ejemplo: loginForm por formAcceso.
2	Busca ese mismo id en script.js o script-normal.js.	Debe quedar exactamente igual.
3	Si el CSS usa una clase, cambia la clase en HTML y CSS al mismo tiempo.	No cambies solo uno de los dos archivos.
4	Si hay data-* como data-tab, data-step o data-value, revisa qué lo lee en JS.	Los atributos data-* suelen ser el puente entre HTML y JS.

Paso	Qué hacer	Recomendación
5	Prueba la funcionalidad aislada antes de mezclarla con otros componentes.	Así sabes si el error viene del componente o de otra parte.

Adaptaciones comunes en proyectos reales

- Para una landing page: usa Password toggle dentro de una sección con clase `.section` y manten el diseño centrado.
- Para un dashboard: coloca Password toggle dentro de una card reutilizable y controla el estado con clases como `is-active` o `is-open`.
- Para un formulario: valida `ids`, `required`, `minlength` y `data-error` antes de enviar datos.
- Para un proyecto con muchos componentes: inicializa esta funcionalidad desde una función `init14()` o desde `initApp()`.
- Para trabajo en equipo: deja comentarios cortos indicando qué HTML, CSS y JS pertenecen a Password toggle.

Mini guía de depuración si no funciona

1. Abre la consola del navegador con F12 y revisa si aparece un error rojo.
2. Si el error dice `null`, significa que JavaScript no encontró un elemento del DOM; revisa `ids` y clases.
3. Si el diseño no cambia, revisa que la clase que agrega JavaScript exista también en CSS.
4. Si usas `import/export`, abre el proyecto con Live Server; no lo abras solo con doble clic.
5. Si el componente depende de `localStorage` o `sessionStorage`, limpia el almacenamiento del navegador y prueba de nuevo.
6. Comenta temporalmente otros componentes para confirmar que no hay conflicto de nombres.

Patrón mental para reutilizarlo

Cada funcionalidad de esta guía se puede entender como una combinación de tres capas: la capa HTML define la estructura, la capa CSS define el diseño visual y la capa JavaScript controla el comportamiento. Para reciclarla bien, no copies solo una capa: copia la estructura mínima, los estilos necesarios y la inicialización correspondiente. Después, cambia nombres y valores de forma ordenada.

Consejo de adaptación profesional

En móviles, este patrón mejora mucho la experiencia porque reduce errores al escribir contraseñas.

15. Validación de formulario

¿Qué hace?

Valida campos obligatorios, correo, longitud mínima y reglas personalizadas antes de procesar o enviar datos.

¿Cuándo usarlo?

Úsalo en login, registro, contacto, cotizaciones, compras, formularios de soporte y paneles administrativos.

Estructura que debes copiar

HTML mínimo

```
<form id="contactForm" novalidate>
  <label for="contactName">Nombre</label>
  <input id="contactName" type="text" required minlength="3">
  <small class="field-error" data-error="contactName"></small>

  <label for="contactEmail">Correo</label>
  <input id="contactEmail" type="email" required>
  <small class="field-error" data-error="contactEmail"></small>

  <button type="submit">Enviar</button>
</form>
```

CSS necesario

```
input.is-invalid,
textarea.is-invalid {
  border-color: #ef4444;
  box-shadow: 0 0 4px rgba(239, 68, 68, .16);
}

input.is-valid,
textarea.is-valid {
  border-color: #22c55e;
  box-shadow: 0 0 4px rgba(34, 197, 94, .14);
}

.field-error {
  color: #fca5a5;
  font-size: .82rem;
  min-height: 1rem;
}
```

funciones.js - versión modular con export

```

export class FormValidator {
  constructor(formId) {
    this.form = document.getElementById(formId);
    this.fields = this.form.querySelectorAll("input, textarea, select");
    this.init();
  }

  init() {
    this.form.addEventListener("submit", (event) => {
      event.preventDefault();

      if (this.validateAll()) {
        console.log("Formulario válido");
      }
    });

    this.fields.forEach((field) => {
      field.addEventListener("input", () => this.validateField(field));
    });
  }

  validateAll() {
    return [...this.fields].every((field) => this.validateField(field));
  }

  validateField(field) {
    let message = "";

    if (field.hasAttribute("required") && field.value.trim() === "") {
      message = "Este campo es obligatorio.";
    } else if (field.type === "email" && !field.value.includes("@")) {
      message = "Ingresa un correo válido.";
    } else if (field.minLength > 0 && field.value.length < field.minLength) {
      message = `Mínimo ${field.minLength} caracteres.`;
    }

    this.showError(field, message);
    return message === "";
  }

  showError(field, message) {
    const error = this.form.querySelector(`[data-error="${field.id}"]`);
    field.classList.toggle("is-invalid", Boolean(message));
    field.classList.toggle("is-valid", !message && field.value.trim() !== "");
    if (error) error.textContent = message;
  }
}

```

script.js - importación e inicialización

```

import { FormValidator } from "./funciones.js";

new FormValidator("contactForm");

```

script-normal.js - versión normal sin módulos

```
const contactForm = document.getElementById("contactForm");
const fields = contactForm.querySelectorAll("input, textarea, select");

function validateField(field) {
  let message = "";

  if (field.required && field.value.trim() === "") message = "Este campo es obligatorio.";
  if (field.type === "email" && !field.value.includes("@")) message = "Correo inválido.";

  const error = contactForm.querySelector(`[data-error="${field.id}"]`);
  field.classList.toggle("is-invalid", Boolean(message));
  if (error) error.textContent = message;

  return message === "";
}

contactForm.addEventListener("submit", (event) => {
  event.preventDefault();
  const valid = [...fields].every(validateField);
  if (valid) console.log("Formulario válido");
});
```

Variables, clases e IDs que puedes cambiar

Elemento	Para qué sirve	Cómo adaptarlo
required	Indica que el campo no puede ir vacío.	Agrégallo solo a campos obligatorios.
minlength	Longitud mínima.	Cambia el número según tu necesidad.
data-error	Relaciona el mensaje con el id del input.	Debe coincidir exactamente con el id.
.is-invalid / .is-valid	Clases visuales.	Cambia colores y sombra en CSS.

Paso a paso para reciclarlo en otro proyecto

1. Agrega required, type="email" o minlength donde corresponda.
2. Debajo de cada input agrega un small con data-error igual al id del campo.
3. Copia los estilos de campos válidos e inválidos.
4. Exporta FormValidator en funciones.js.
5. Inicializa new FormValidator("idDelFormulario") en script.js.
6. Prueba enviar vacío, con email incorrecto y con datos correctos.

Qué debes revisar al integrarlo

- Que los ids del HTML coincidan con los ids usados en JavaScript.
- Que no estés usando el mismo id dos veces en la misma página.
- Que las clases activas como is-active, is-open, is-valid o is-invalid estén escritas igual en HTML, CSS y JS.
- Que si usas módulos el script se cargue con type="module".
- Que si usas la versión normal no dejes imports ni exports en el archivo.

Errores comunes

- Olvidar novalidate si quieres controlar mensajes con JS.
- Usar data-error diferente al id del campo.
- Validar solo al submit y no en input; la experiencia se siente peor.

Cómo cambiar nombres sin romper la funcionalidad

Paso	Qué hacer	Recomendación
1	Cambia primero el id en el HTML.	Ejemplo: loginForm por formAcceso.
2	Busca ese mismo id en script.js o script-normal.js.	Debe quedar exactamente igual.
3	Si el CSS usa una clase, cambia la clase en HTML y CSS al mismo tiempo.	No cambies solo uno de los dos archivos.
4	Si hay data-* como data-tab, data-step o data-value, revisa qué lo lee en JS.	Los atributos data-* suelen ser el puente entre HTML y JS.
5	Prueba la funcionalidad aislada antes de mezclarla con otros componentes.	Así sabes si el error viene del componente o de otra parte.

Adaptaciones comunes en proyectos reales

- Para una landing page: usa Validación de formulario dentro de una sección con clase .section y manten el diseño centrado.
- Para un dashboard: coloca Validación de formulario dentro de una card reutilizable y controla el estado con clases como is-active o is-open.
- Para un formulario: valida ids, required, minlength y data-error antes de enviar datos.
- Para un proyecto con muchos componentes: inicializa esta funcionalidad desde una función init15() o desde initApp().
- Para trabajo en equipo: deja comentarios cortos indicando qué HTML, CSS y JS pertenecen a Validación de formulario.

Mini guía de depuración si no funciona

1. Abre la consola del navegador con F12 y revisa si aparece un error rojo.
2. Si el error dice null, significa que JavaScript no encontró un elemento del DOM; revisa ids y clases.
3. Si el diseño no cambia, revisa que la clase que agrega JavaScript exista también en CSS.
4. Si usas import/export, abre el proyecto con Live Server; no lo abras solo con doble clic.
5. Si el componente depende de localStorage o sessionStorage, limpia el almacenamiento del navegador y prueba de nuevo.
6. Comenta temporalmente otros componentes para confirmar que no hay conflicto de nombres.

Patrón mental para reutilizarlo

Cada funcionalidad de esta guía se puede entender como una combinación de tres capas: la capa HTML define la estructura, la capa CSS define el diseño visual y la capa JavaScript controla el comportamiento. Para reciclarla bien, no copies solo una capa: copia la estructura mínima, los estilos necesarios y la inicialización correspondiente. Después, cambia nombres y valores de forma ordenada.

Consejo de adaptación profesional

La validación del frontend mejora la experiencia, pero no reemplaza la validación del backend.

16. Mensajes por campo

¿Qué hace?

Muestra errores o ayudas debajo de cada input. Complementa la validación y hace que el usuario sepa exactamente qué corregir.

¿Cuándo usarlo?

Úsalo en formularios profesionales donde cada campo debe tener retroalimentación individual.

Estructura que debes copiar

HTML mínimo

```
<div class="field">
  <label for="username">Usuario</label>
  <input id="username" type="text" required>
  <small class="field-help">Usa letras y números.</small>
  <small class="field-error" data-error="username"></small>
</div>
```

CSS necesario

```
.field {
  margin-bottom: 1rem;
}

.field-help {
  display: block;
  margin-top: .3rem;
  color: var(--text-muted, #94a3b8);
  font-size: .78rem;
}

.field-error {
  display: block;
  min-height: 1rem;
  margin-top: .25rem;
  color: #fb7185;
  font-size: .8rem;
}
```

funciones.js - versión modular con export

```
export function setFieldMessage(form, fieldId, message = "") {
  const errorBox = form.querySelector(`[data-error="${fieldId}"]`);
  const field = form.querySelector(`#${fieldId}`);

  if (errorBox) errorBox.textContent = message;
  if (field) field.classList.toggle("is-invalid", Boolean(message));
}

export function clearFieldMessages(form) {
  form.querySelectorAll("[data-error]").forEach((errorBox) => {
    errorBox.textContent = "";
  });

  form.querySelectorAll(".is-invalid").forEach((field) => {
    field.classList.remove("is-invalid");
  });
}
```

script.js - importación e inicialización

```
import { setFieldMessage, clearFieldMessages } from "./funciones.js";

const form = document.getElementById("contactForm");
setFieldMessage(form, "username", "El usuario es obligatorio.");
// clearFieldMessages(form);
```

script-normal.js - versión normal sin módulos

```
function setFieldMessage(form, fieldId, message = "") {
  const errorBox = form.querySelector(`[data-error="${fieldId}"]`);
  const field = form.querySelector(`#${fieldId}`);

  if (errorBox) errorBox.textContent = message;
  if (field) field.classList.toggle("is-invalid", Boolean(message));
}
```

Variables, clases e IDs que puedes cambiar

Elemento	Para qué sirve	Cómo adaptarlo
data-error	Conecta el mensaje con un input.	Debe tener el mismo valor que el id del input.
.field-help	Texto de ayuda normal.	Úsalo para explicar reglas antes de que haya error.
.field-error	Texto de error.	Solo se llena cuando algo está mal.

Paso a paso para reciclarlo en otro proyecto

1. Agrupa cada input dentro de un div.field.
2. Agrega field-help si quieres instrucciones permanentes.
3. Agrega field-error con data-error igual al id del input.
4. Usa setFieldMessage(form, id, mensaje) desde tu validación.
5. Usa clearFieldMessages(form) antes de validar todo si quieres limpiar errores previos.

Qué debes revisar al integrarlo

- Que los ids del HTML coincidan con los ids usados en JavaScript.
- Que no estés usando el mismo id dos veces en la misma página.
- Que las clases activas como is-active, is-open, is-valid o is-invalid estén escritas igual en HTML, CSS y JS.
- Que si usas módulos el script se cargue con type="module".
- Que si usas la versión normal no dejes imports ni exports en el archivo.

Errores comunes

- Mostrar todos los errores en un solo párrafo genérico.
- No dejar espacio mínimo y que el diseño salte cuando aparece error.
- Usar mensajes muy largos que rompen la tarjeta.

Cómo cambiar nombres sin romper la funcionalidad

Paso	Qué hacer	Recomendación
1	Cambia primero el id en el HTML.	Ejemplo: loginForm por formAcceso.
2	Busca ese mismo id en script.js o script-normal.js.	Debe quedar exactamente igual.
3	Si el CSS usa una clase, cambia la clase en HTML y CSS al mismo tiempo.	No cambies solo uno de los dos archivos.
4	Si hay data-* como data-tab, data-step o data-value, revisa qué lo lee en JS.	Los atributos data-* suelen ser el puente entre HTML y JS.
5	Prueba la funcionalidad aislada antes de mezclarla con otros componentes.	Así sabes si el error viene del componente o de otra parte.

Adaptaciones comunes en proyectos reales

- Para una landing page: usa Mensajes por campo dentro de una sección con clase `.section` y manten el diseño centrado.
- Para un dashboard: coloca Mensajes por campo dentro de una card reutilizable y controla el estado con clases como `is-active` o `is-open`.
- Para un formulario: valida `ids`, `required`, `minlength` y `data-error` antes de enviar datos.
- Para un proyecto con muchos componentes: inicializa esta funcionalidad desde una función `init16()` o desde `initApp()`.
- Para trabajo en equipo: deja comentarios cortos indicando qué HTML, CSS y JS pertenecen a Mensajes por campo.

Mini guía de depuración si no funciona

1. Abre la consola del navegador con F12 y revisa si aparece un error rojo.
2. Si el error dice `null`, significa que JavaScript no encontró un elemento del DOM; revisa `ids` y clases.
3. Si el diseño no cambia, revisa que la clase que agrega JavaScript exista también en CSS.
4. Si usas `import/export`, abre el proyecto con Live Server; no lo abras solo con doble clic.
5. Si el componente depende de `localStorage` o `sessionStorage`, limpia el almacenamiento del navegador y prueba de nuevo.
6. Comenta temporalmente otros componentes para confirmar que no hay conflicto de nombres.

Patrón mental para reutilizarlo

Cada funcionalidad de esta guía se puede entender como una combinación de tres capas: la capa HTML define la estructura, la capa CSS define el diseño visual y la capa JavaScript controla el comportamiento. Para reciclarla bien, no copies solo una capa: copia la estructura mínima, los estilos necesarios y la inicialización correspondiente. Después, cambia nombres y valores de forma ordenada.

Consejo de adaptación profesional

Los mensajes por campo deben ser cortos y específicos. Evita textos genéricos como "error".

17. Character counter

¿Qué hace?

Cuenta los caracteres de un textarea o input. Se usa mucho en comentarios, bios, descripciones, formularios de contacto y publicaciones.

¿Cuándo usarlo?

Úsalo cuando haya un límite de caracteres y el usuario necesite verlo en tiempo real.

Estructura que debes copiar

HTML mínimo

```
<label for="bio">Descripción</label>
<textarea id="bio" maxlength="160" placeholder="Escribe una descripción
  corta"></textarea>
<small id="bioCounter">0 / 160</small>
```

CSS necesario

```
textarea {
  width: 100%;
  min-height: 120px;
  resize: vertical;
  padding: 1rem;
  border-radius: 1rem;
}

#bioCounter {
  display: block;
  margin-top: .3rem;
  text-align: right;
  color: var(--text-muted, #94a3b8);
}

#bioCounter.is-limit {
  color: #f97316;
  font-weight: 700;
}
```

funciones.js - versión modular con export

```
export class CharacterCounter {
  constructor(inputId, counterId) {
    this.input = document.getElementById(inputId);
    this.counter = document.getElementById(counterId);
    this.max = Number(this.input.getAttribute("maxlength")) || 0;
    this.init();
  }

  init() {
    this.update();
    this.input.addEventListener("input", () => this.update());
  }

  update() {
    const length = this.input.value.length;
    this.counter.textContent = this.max ? `${length} / ${this.max}` : `${length}`;
    this.counter.classList.toggle("is-limit", this.max && length >= this.max *
      0.9);
  }
}
```

script.js - importación e inicialización

```
import { CharacterCounter } from "./funciones.js";

new CharacterCounter("bio", "bioCounter");
```

script-normal.js - versión normal sin módulos

```
const bio = document.getElementById("bio");
const bioCounter = document.getElementById("bioCounter");
const max = Number(bio.getAttribute("maxLength"));

function updateCounter() {
  const length = bio.value.length;
  bioCounter.textContent = `${length} / ${max}`;
  bioCounter.classList.toggle("is-limit", length >= max * 0.9);
}

bio.addEventListener("input", updateCounter);
updateCounter();
```

Variables, clases e IDs que puedes cambiar

Elemento	Para qué sirve	Cómo adaptarlo
maxlength	Límite real del textarea.	Cámbialo según el máximo permitido.
#bioCounter	Elemento donde se muestra el conteo.	Cambia el id si tienes varios contadores.
.is-limit	Estado cercano al límite.	Puedes cambiarlo a rojo o naranja.

Paso a paso para reciclarlo en otro proyecto

1. Agrega maxlength al input o textarea.
2. Crea un small donde se verá el contador.
3. Copia el CSS de contador y estado is-limit.
4. Inicializa CharacterCounter con el id del campo y el id del contador.
5. Escribe hasta llegar cerca del límite y revisa que cambie el color.

Qué debes revisar al integrarlo

- Que los ids del HTML coincidan con los ids usados en JavaScript.
- Que no estés usando el mismo id dos veces en la misma página.
- Que las clases activas como is-active, is-open, is-valid o is-invalid estén escritas igual en HTML, CSS y JS.
- Que si usas módulos el script se cargue con type="module".
- Que si usas la versión normal no dejes imports ni exports en el archivo.

Errores comunes

- Olvidar maxlength y no tener límite real.
- Usar innerHTML para un texto que no necesita HTML.
- No llamar update() al cargar y que aparezca vacío.

Cómo cambiar nombres sin romper la funcionalidad

Paso	Qué hacer	Recomendación
1	Cambia primero el id en el HTML.	Ejemplo: loginForm por formAcceso.
2	Busca ese mismo id en script.js o script-normal.js.	Debe quedar exactamente igual.
3	Si el CSS usa una clase, cambia la clase en HTML y CSS al mismo tiempo.	No cambies solo uno de los dos archivos.
4	Si hay data-* como data-tab, data-step o data-value, revisa qué lo lee en JS.	Los atributos data-* suelen ser el puente entre HTML y JS.

Paso	Qué hacer	Recomendación
5	Prueba la funcionalidad aislada antes de mezclarla con otros componentes.	Así sabes si el error viene del componente o de otra parte.

Adaptaciones comunes en proyectos reales

- Para una landing page: usa Character counter dentro de una sección con clase `.section` y manten el diseño centrado.
- Para un dashboard: coloca Character counter dentro de una card reutilizable y controla el estado con clases como `is-active` o `is-open`.
- Para un formulario: valida `ids`, `required`, `minlength` y `data-error` antes de enviar datos.
- Para un proyecto con muchos componentes: inicializa esta funcionalidad desde una función `init17()` o desde `initApp()`.
- Para trabajo en equipo: deja comentarios cortos indicando qué HTML, CSS y JS pertenecen a Character counter.

Mini guía de depuración si no funciona

1. Abre la consola del navegador con F12 y revisa si aparece un error rojo.
2. Si el error dice `null`, significa que JavaScript no encontró un elemento del DOM; revisa `ids` y clases.
3. Si el diseño no cambia, revisa que la clase que agrega JavaScript exista también en CSS.
4. Si usas `import/export`, abre el proyecto con Live Server; no lo abras solo con doble clic.
5. Si el componente depende de `localStorage` o `sessionStorage`, limpia el almacenamiento del navegador y prueba de nuevo.
6. Comenta temporalmente otros componentes para confirmar que no hay conflicto de nombres.

Patrón mental para reutilizarlo

Cada funcionalidad de esta guía se puede entender como una combinación de tres capas: la capa HTML define la estructura, la capa CSS define el diseño visual y la capa JavaScript controla el comportamiento. Para reciclarla bien, no copies solo una capa: copia la estructura mínima, los estilos necesarios y la inicialización correspondiente. Después, cambia nombres y valores de forma ordenada.

Consejo de adaptación profesional

Combina el contador con `maxlength` para que el límite sea real, no solo visual.

18. Range preview

¿Qué hace?

Muestra en tiempo real el valor de un input type="range". Es útil para filtros de precio, volumen, porcentaje, rating, tamaños, cantidades y configuraciones.

¿Cuándo usarlo?

Úsalo cuando un slider controle un valor que el usuario necesita leer claramente.

Estructura que debes copiar

HTML mínimo

```
<label for="priceRange">Precio máximo</label>
<input id="priceRange" type="range" min="0" max="100000" step="5000" value="50000">
<p>Valor seleccionado: <strong id="pricePreview">50000</strong></p>
```

CSS necesario

```
input[type="range"] {
  width: 100%;
  accent-color: var(--primary, #2563eb);
}

#pricePreview {
  color: var(--primary, #2563eb);
  font-size: 1.1rem;
}
```

funciones.js - versión modular con export

```
export class RangePreview {
  constructor(rangeId, previewId, formatter = null) {
    this.range = document.getElementById(rangeId);
    this.preview = document.getElementById(previewId);
    this.formatter = formatter || ((value) => value);
    this.init();
  }

  init() {
    this.update();
    this.range.addEventListener("input", () => this.update());
  }

  update() {
    this.preview.textContent = this.formatter(this.range.value);
  }
}
```

script.js - importación e inicialización

```
import { RangePreview } from "./funciones.js";

new RangePreview("priceRange", "pricePreview", (value) =>
  `$$${Number(value).toLocaleString("es-CO")}`);
```

script-normal.js - versión normal sin módulos

```
const priceRange = document.getElementById("priceRange");
const pricePreview = document.getElementById("pricePreview");

function updatePricePreview() {
  pricePreview.textContent = `$$${Number(priceRange.value).toLocaleString("es-CO")}`;
}

priceRange.addEventListener("input", updatePricePreview);
updatePricePreview();
```

Variables, clases e IDs que puedes cambiar

Elemento	Para qué sirve	Cómo adaptarlo
min / max / step	Rango y salto del slider.	Ajusta según precio, porcentaje o cantidad.
value	Valor inicial.	Debe estar entre min y max.
formatter	Función para formatear el resultado.	Úsala para COP, %, kg, L, etc.

Paso a paso para reciclarlo en otro proyecto

1. Copia el input type="range" y el elemento preview.
2. Ajusta min, max, step y value.
3. Inicializa RangePreview con los ids correctos.
4. Agrega formatter si quieres moneda o unidad.
5. Mueve el slider y revisa que el valor cambie sin recargar.

Qué debes revisar al integrarlo

- Que los ids del HTML coincidan con los ids usados en JavaScript.
- Que no estés usando el mismo id dos veces en la misma página.
- Que las clases activas como is-active, is-open, is-valid o is-invalid estén escritas igual en HTML, CSS y JS.
- Que si usas módulos el script se cargue con type="module".
- Que si usas la versión normal no dejes imports ni exports en el archivo.

Errores comunes

- No actualizar el preview al cargar.
- Dejar step demasiado grande o demasiado pequeño.
- Mostrar número sin formato cuando es precio.

Cómo cambiar nombres sin romper la funcionalidad

Paso	Qué hacer	Recomendación
1	Cambia primero el id en el HTML.	Ejemplo: loginForm por formAcceso.
2	Busca ese mismo id en script.js o script-normal.js.	Debe quedar exactamente igual.
3	Si el CSS usa una clase, cambia la clase en HTML y CSS al mismo tiempo.	No cambies solo uno de los dos archivos.
4	Si hay data-* como data-tab, data-step o data-value, revisa qué lo lee en JS.	Los atributos data-* suelen ser el puente entre HTML y JS.
5	Prueba la funcionalidad aislada antes de mezclarla con otros componentes.	Así sabes si el error viene del componente o de otra parte.

Adaptaciones comunes en proyectos reales

- Para una landing page: usa Range preview dentro de una sección con clase .section y manten el diseño centrado.
- Para un dashboard: coloca Range preview dentro de una card reutilizable y controla el estado con clases como is-active o is-open.
- Para un formulario: valida ids, required, minlength y data-error antes de enviar datos.
- Para un proyecto con muchos componentes: inicializa esta funcionalidad desde una función init18() o desde initApp().

- Para trabajo en equipo: deja comentarios cortos indicando qué HTML, CSS y JS pertenecen a Range preview.

Mini guía de depuración si no funciona

1. Abre la consola del navegador con F12 y revisa si aparece un error rojo.
2. Si el error dice null, significa que JavaScript no encontró un elemento del DOM; revisa ids y clases.
3. Si el diseño no cambia, revisa que la clase que agrega JavaScript exista también en CSS.
4. Si usas import/export, abre el proyecto con Live Server; no lo abras solo con doble clic.
5. Si el componente depende de localStorage o sessionStorage, limpia el almacenamiento del navegador y prueba de nuevo.
6. Comenta temporalmente otros componentes para confirmar que no hay conflicto de nombres.

Patrón mental para reutilizarlo

Cada funcionalidad de esta guía se puede entender como una combinación de tres capas: la capa HTML define la estructura, la capa CSS define el diseño visual y la capa JavaScript controla el comportamiento. Para reciclarla bien, no copies solo una capa: copia la estructura mínima, los estilos necesarios y la inicialización correspondiente. Después, cambia nombres y valores de forma ordenada.

Consejo de adaptación profesional

Siempre formatea el valor según contexto: moneda, porcentaje, litros, kilogramos o estrellas.

19. File info

¿Qué hace?

Muestra el nombre, tamaño y tipo de archivo seleccionado. Es útil en formularios con carga de CV, imágenes, documentos, facturas o evidencias.

¿Cuándo usarlo?

Úsalo siempre que tengas input type="file" y quieras que el usuario confirme qué archivo eligió.

Estructura que debes copiar

HTML mínimo

```
<label for="fileInput">Subir archivo</label>
<input id="fileInput" type="file" accept=".pdf,.png,.jpg,.jpeg">
<p id="fileInfo" class="file-info">Ningún archivo seleccionado.</p>
```

CSS necesario

```
.file-info {
  margin-top: .6rem;
  padding: .75rem 1rem;
  border-radius: .9rem;
  background: rgba(255, 255, 255, 0.1);
  color: var(--text-muted, #cbd5e1);
}
```

funciones.js - versión modular con export

```
export class FileInfo {
  constructor(inputId, infoId) {
    this.input = document.getElementById(inputId);
    this.info = document.getElementById(infoId);
    this.init();
  }

  init() {
    this.input.addEventListener("change", () => this.update());
  }

  update() {
    const file = this.input.files[0];

    if (!file) {
      this.info.textContent = "Ningún archivo seleccionado.";
      return;
    }

    const sizeKb = (file.size / 1024).toFixed(1);
    this.info.textContent = `${file.name} - ${sizeKb} KB - ${file.type || "tipo desconocido"}`;
  }
}
```

script.js - importación e inicialización

```
import { FileInfo } from "./funciones.js";

new FileInfo("fileInput", "fileInfo");
```

script-normal.js - versión normal sin módulos


```
const fileInput = document.getElementById("fileInput");
const fileInfo = document.getElementById("fileInfo");

fileInput.addEventListener("change", () => {
  const file = fileInput.files[0];

  if (!file) {
    fileInfo.textContent = "Ningún archivo seleccionado.";
    return;
  }

  fileInfo.textContent = `${file.name} - ${((file.size / 1024).toFixed(1))} KB`;
});
```

Variables, clases e IDs que puedes cambiar

Elemento	Para qué sirve	Cómo adaptarlo
accept	Tipos permitidos.	Cambia extensiones según tu caso.
file.size	Peso en bytes.	Puedes convertir a KB o MB.
file.type	Tipo MIME.	Sirve para validar imágenes o PDF.

Paso a paso para reciclarlo en otro proyecto

1. Agrega input type="file" y un contenedor de texto para la información.
2. Configura accept para limitar tipos de archivo.
3. Copia la clase FileInfo o la versión normal.
4. Prueba elegir archivo y cancelar selección.
5. Agrega validaciones de tamaño si el proyecto lo requiere.

Qué debes revisar al integrarlo

- Que los ids del HTML coincidan con los ids usados en JavaScript.
- Que no estés usando el mismo id dos veces en la misma página.
- Que las clases activas como is-active, is-open, is-valid o is-invalid estén escritas igual en HTML, CSS y JS.
- Que si usas módulos el script se cargue con type="module".
- Que si usas la versión normal no dejes imports ni exports en el archivo.

Errores comunes

- Creer que accept es seguridad total; también valida en backend.
- No manejar el caso cuando el usuario cancela.
- Mostrar bytes sin convertir y que sea difícil de leer.

Cómo cambiar nombres sin romper la funcionalidad

Paso	Qué hacer	Recomendación
1	Cambia primero el id en el HTML.	Ejemplo: loginForm por formAcceso.
2	Busca ese mismo id en script.js o script-normal.js.	Debe quedar exactamente igual.
3	Si el CSS usa una clase, cambia la clase en HTML y CSS al mismo tiempo.	No cambies solo uno de los dos archivos.
4	Si hay data-* como data-tab, data-step o data-value, revisa qué lo lee en JS.	Los atributos data-* suelen ser el puente entre HTML y JS.

Paso	Qué hacer	Recomendación
5	Prueba la funcionalidad aislada antes de mezclarla con otros componentes.	Así sabes si el error viene del componente o de otra parte.

Adaptaciones comunes en proyectos reales

- Para una landing page: usa File info dentro de una sección con clase .section y manten el diseño centrado.
- Para un dashboard: coloca File info dentro de una card reutilizable y controla el estado con clases como is-active o is-open.
- Para un formulario: valida ids, required, minlength y data-error antes de enviar datos.
- Para un proyecto con muchos componentes: inicializa esta funcionalidad desde una función init19() o desde initApp().
- Para trabajo en equipo: deja comentarios cortos indicando qué HTML, CSS y JS pertenecen a File info.

Mini guía de depuración si no funciona

1. Abre la consola del navegador con F12 y revisa si aparece un error rojo.
2. Si el error dice null, significa que JavaScript no encontró un elemento del DOM; revisa ids y clases.
3. Si el diseño no cambia, revisa que la clase que agrega JavaScript exista también en CSS.
4. Si usas import/export, abre el proyecto con Live Server; no lo abras solo con doble clic.
5. Si el componente depende de localStorage o sessionStorage, limpia el almacenamiento del navegador y prueba de nuevo.
6. Comenta temporalmente otros componentes para confirmar que no hay conflicto de nombres.

Patrón mental para reutilizarlo

Cada funcionalidad de esta guía se puede entender como una combinación de tres capas: la capa HTML define la estructura, la capa CSS define el diseño visual y la capa JavaScript controla el comportamiento. Para reciclarla bien, no copies solo una capa: copia la estructura mínima, los estilos necesarios y la inicialización correspondiente. Después, cambia nombres y valores de forma ordenada.

Consejo de adaptación profesional

Para cargas reales, valida también en backend el tipo y tamaño del archivo.

20. Autosave draft

¿Qué hace?

Guarda automáticamente un borrador mientras el usuario escribe. Evita pérdida de información si se cierra la página por accidente.

¿Cuándo usarlo?

Úsalo en comentarios largos, publicaciones, formularios, notas, mensajes y editores de contenido.

Estructura que debes copiar

HTML mínimo

```
<label for="draftText">Borrador</label>
<textarea id="draftText" placeholder="Escribe aquí..."></textarea>
<small id="draftStatus">Sin guardar todavía</small>
```

CSS necesario

```
#draftStatus {
  display: block;
  margin-top: .4rem;
  color: var(--text-muted, #94a3b8);
  font-size: .82rem;
}

#draftStatus.is-saved {
  color: #86efac;
}
```

funciones.js - versión modular con export

```
export class AutosaveDraft {
  constructor(inputId, statusId, storageKey = "kit-draft") {
    this.input = document.getElementById(inputId);
    this.status = document.getElementById(statusId);
    this.storageKey = storageKey;
    this.timeout = null;
    this.init();
  }

  init() {
    this.input.value = localStorage.getItem(this.storageKey) || "";
    this.input.addEventListener("input", () => this.scheduleSave());
  }

  scheduleSave() {
    this.status.textContent = "Guardando...";
    this.status.classList.remove("is-saved");
    clearTimeout(this.timeout);

    this.timeout = setTimeout(() => {
      localStorage.setItem(this.storageKey, this.input.value);
      this.status.textContent = "Borrador guardado automáticamente.";
      this.status.classList.add("is-saved");
    }, 600);
  }
}
```

script.js - importación e inicialización

```
import { AutosaveDraft } from "./funciones.js";

new AutosaveDraft("draftText", "draftStatus", "contact-draft");
```

script-normal.js - versión normal sin módulos

```
const draftText = document.getElementById("draftText");
const draftStatus = document.getElementById("draftStatus");
let draftTimeout;

draftText.value = localStorage.getItem("contact-draft") || "";

draftText.addEventListener("input", () => {
  draftStatus.textContent = "Guardando...";
  clearTimeout(draftTimeout);

  draftTimeout = setTimeout(() => {
    localStorage.setItem("contact-draft", draftText.value);
    draftStatus.textContent = "Borrador guardado.";
  }, 600);
});
```

Variables, clases e IDs que puedes cambiar

Elemento	Para qué sirve	Cómo adaptarlo
storageKey	Clave donde se guarda el borrador.	Cámbiala por formulario o página.
600	Tiempo de espera antes de guardar.	Aumenta si quieres guardar menos seguido.
#draftText	Textarea que se guarda.	Puedes usar input o textarea.

Paso a paso para reciclarlo en otro proyecto

1. Agrega textarea y un status visual.
2. Define una clave única de localStorage para ese borrador.
3. Copia AutosaveDraft e inicialízalo.
4. Escribe texto, espera y recarga la página.
5. Confirma que el texto se restaura automáticamente.
6. Agrega un botón de limpiar borrador si el flujo lo necesita.

Qué debes revisar al integrarlo

- Que los ids del HTML coincidan con los ids usados en JavaScript.
- Que no estés usando el mismo id dos veces en la misma página.
- Que las clases activas como is-active, is-open, is-valid o is-invalid estén escritas igual en HTML, CSS y JS.
- Que si usas módulos el script se cargue con type="module".
- Que si usas la versión normal no dejes imports ni exports en el archivo.

Errores comunes

- Usar la misma clave para varios formularios.
- Guardar información sensible sin avisar.
- Guardar en cada tecla sin debounce o setTimeout.

Cómo cambiar nombres sin romper la funcionalidad

Paso	Qué hacer	Recomendación
1	Cambia primero el id en el HTML.	Ejemplo: loginForm por formAcceso.
2	Busca ese mismo id en script.js o script-normal.js.	Debe quedar exactamente igual.
3	Si el CSS usa una clase, cambia la clase en HTML y CSS al mismo tiempo.	No cambies solo uno de los dos archivos.

Paso	Qué hacer	Recomendación
4	Si hay data-* como data-tab, data-step o data-value, revisa qué lo lee en JS.	Los atributos data-* suelen ser el puente entre HTML y JS.
5	Prueba la funcionalidad aislada antes de mezclarla con otros componentes.	Así sabes si el error viene del componente o de otra parte.

Adaptaciones comunes en proyectos reales

- Para una landing page: usa Autosave draft dentro de una sección con clase .section y manten el diseño centrado.
- Para un dashboard: coloca Autosave draft dentro de una card reutilizable y controla el estado con clases como is-active o is-open.
- Para un formulario: valida ids, required, minlength y data-error antes de enviar datos.
- Para un proyecto con muchos componentes: inicializa esta funcionalidad desde una función init20() o desde initApp().
- Para trabajo en equipo: deja comentarios cortos indicando qué HTML, CSS y JS pertenecen a Autosave draft.

Mini guía de depuración si no funciona

1. Abre la consola del navegador con F12 y revisa si aparece un error rojo.
2. Si el error dice null, significa que JavaScript no encontró un elemento del DOM; revisa ids y clases.
3. Si el diseño no cambia, revisa que la clase que agrega JavaScript exista también en CSS.
4. Si usas import/export, abre el proyecto con Live Server; no lo abras solo con doble clic.
5. Si el componente depende de localStorage o sessionStorage, limpia el almacenamiento del navegador y prueba de nuevo.
6. Comenta temporalmente otros componentes para confirmar que no hay conflicto de nombres.

Patrón mental para reutilizarlo

Cada funcionalidad de esta guía se puede entender como una combinación de tres capas: la capa HTML define la estructura, la capa CSS define el diseño visual y la capa JavaScript controla el comportamiento. Para reciclarla bien, no copies solo una capa: copia la estructura mínima, los estilos necesarios y la inicialización correspondiente. Después, cambia nombres y valores de forma ordenada.

Consejo de adaptación profesional

Agrega una opción de "descartar borrador" para que el usuario pueda limpiar el contenido guardado.

21. Stepper / Wizard

¿Qué hace?

Divide un proceso en pasos. Es común en formularios largos, registros, compras, onboarding y creación de perfiles.

¿Cuándo usarlo?

Úsalo cuando una tarea sea demasiado larga para una sola pantalla.

Estructura que debes copiar

HTML mínimo

```
<section class="stepper" id="projectStepper">
  <div class="stepper__indicators">
    <span class="is-active">1</span>
    <span>2</span>
    <span>3</span>
  </div>

  <article class="stepper__panel is-active" data-step="0">
    <h3>Paso 1: Datos básicos</h3>
    <input placeholder="Nombre del proyecto">
  </article>

  <article class="stepper__panel" data-step="1">
    <h3>Paso 2: Detalles</h3>
    <textarea placeholder="Descripción"></textarea>
  </article>

  <article class="stepper__panel" data-step="2">
    <h3>Paso 3: Confirmación</h3>
    <p>Revisa la información antes de enviar.</p>
  </article>

  <div class="stepper__actions">
    <button type="button" data-step-prev>Anterior</button>
    <button type="button" data-step-next>Siguiente</button>
  </div>
</section>
```

CSS necesario

```
.stepper__indicators {
  display: flex;
  gap: .5rem;
  margin-bottom: 1rem;
}

.stepper__indicators span {
  width: 32px;
  height: 32px;
  display: grid;
  place-items: center;
  border-radius: 999px;
  background: rgba(255, 255, 255, .12);
}

.stepper__indicators span.is-active {
  background: var(--primary, #2563eb);
  color: #fff;
}

.stepper__panel { display: none; }
.stepper__panel.is-active { display: block; }
.stepper__actions { display: flex; gap: .8rem; margin-top: 1rem; }
```

funciones.js - versión modular con export

```
export class Stepper {
  constructor(rootId) {
    this.root = document.getElementById(rootId);
    this.panels = this.root.querySelectorAll(".stepper__panel");
    this.indicators = this.root.querySelectorAll(".stepper__indicators span");
    this.prev = this.root.querySelector("[data-step-prev]");
    this.next = this.root.querySelector("[data-step-next]");
    this.current = 0;
    this.init();
  }

  init() {
    this.prev.addEventListener("click", () => this.goTo(this.current - 1));
    this.next.addEventListener("click", () => this.goTo(this.current + 1));
    this.render();
  }

  goTo(index) {
    if (index < 0 || index >= this.panels.length) return;
    this.current = index;
    this.render();
  }

  render() {
    this.panels.forEach((panel, index) => {
      panel.classList.toggle("is-active", index === this.current);
    });

    this.indicators.forEach((indicator, index) => {
      indicator.classList.toggle("is-active", index === this.current);
    });

    this.prev.disabled = this.current === 0;
    this.next.textContent = this.current === this.panels.length - 1 ? "Finalizar"
      : "Siguiente";
  }
}
```

script.js - importación e inicialización

```
import { Stepper } from "../funciones.js";

new Stepper("projectStepper");
```

script-normal.js - versión normal sin módulos

```
const stepper = document.getElementById("projectStepper");
const panels = stepper.querySelectorAll(".stepper__panel");
const indicators = stepper.querySelectorAll(".stepper__indicators span");
const prevButton = stepper.querySelector("[data-step-prev]");
const nextButton = stepper.querySelector("[data-step-next]");
let currentStep = 0;

function renderStepper() {
  panels.forEach((panel, index) => panel.classList.toggle("is-active", index ===
    currentStep));
  indicators.forEach((item, index) => item.classList.toggle("is-active", index ===
    currentStep));
}

prevButton.addEventListener("click", () => { currentStep = Math.max(0, currentStep -
  1); renderStepper(); });
nextButton.addEventListener("click", () => { currentStep = Math.min(panels.length - 1,
  currentStep + 1); renderStepper(); });
renderStepper();
```

Variables, clases e IDs que puedes cambiar

Elemento	Para qué sirve	Cómo adaptarlo
#projectStepper	Contenedor principal.	Cambia el id si tienes varios steppers.
.stepper__panel	Cada paso.	Duplica o elimina panels para más o menos pasos.

Elemento	Para qué sirve	Cómo adaptarlo
data-step-prev / data-step-next	Botones de navegación.	Deben estar dentro del stepper.

Paso a paso para reciclarlo en otro proyecto

1. Copia el contenedor .stepper completo.
2. Cambia títulos y campos dentro de cada .stepper__panel.
3. Agrega un indicador por cada panel.
4. Copia estilos de indicadores, panels y acciones.
5. Inicializa new Stepper("projectStepper").
6. Prueba avanzar, retroceder y llegar al último paso.

Qué debes revisar al integrarlo

- Que los ids del HTML coincidan con los ids usados en JavaScript.
- Que no estés usando el mismo id dos veces en la misma página.
- Que las clases activas como is-active, is-open, is-valid o is-invalid estén escritas igual en HTML, CSS y JS.
- Que si usas módulos el script se cargue con type="module".
- Que si usas la versión normal no dejes imports ni exports en el archivo.

Errores comunes

- Tener más indicadores que panels o al contrario.
- Poner los botones fuera del root y que querySelector no los encuentre.
- No validar cada paso antes de avanzar si el formulario es real.

Cómo cambiar nombres sin romper la funcionalidad

Paso	Qué hacer	Recomendación
1	Cambia primero el id en el HTML.	Ejemplo: loginForm por formAcceso.
2	Busca ese mismo id en script.js o script-normal.js.	Debe quedar exactamente igual.
3	Si el CSS usa una clase, cambia la clase en HTML y CSS al mismo tiempo.	No cambies solo uno de los dos archivos.
4	Si hay data-* como data-tab, data-step o data-value, revisa qué lo lee en JS.	Los atributos data-* suelen ser el puente entre HTML y JS.
5	Prueba la funcionalidad aislada antes de mezclarla con otros componentes.	Así sabes si el error viene del componente o de otra parte.

Adaptaciones comunes en proyectos reales

- Para una landing page: usa Stepper / Wizard dentro de una sección con clase .section y manten el diseño centrado.
- Para un dashboard: coloca Stepper / Wizard dentro de una card reutilizable y controla el estado con clases como is-active o is-open.
- Para un formulario: valida ids, required, minlength y data-error antes de enviar datos.
- Para un proyecto con muchos componentes: inicializa esta funcionalidad desde una función init21() o desde initApp().
- Para trabajo en equipo: deja comentarios cortos indicando qué HTML, CSS y JS pertenecen a Stepper / Wizard.

Mini guía de depuración si no funciona

1. Abre la consola del navegador con F12 y revisa si aparece un error rojo.
2. Si el error dice null, significa que JavaScript no encontró un elemento del DOM; revisa ids y clases.
3. Si el diseño no cambia, revisa que la clase que agrega JavaScript exista también en CSS.
4. Si usas import/export, abre el proyecto con Live Server; no lo abras solo con doble clic.
5. Si el componente depende de localStorage o sessionStorage, limpia el almacenamiento del navegador y prueba de nuevo.
6. Comenta temporalmente otros componentes para confirmar que no hay conflicto de nombres.

Patrón mental para reutilizarlo

Cada funcionalidad de esta guía se puede entender como una combinación de tres capas: la capa HTML define la estructura, la capa CSS define el diseño visual y la capa JavaScript controla el comportamiento. Para reciclarla bien, no copies solo una capa: copia la estructura mínima, los estilos necesarios y la inicialización correspondiente. Después, cambia nombres y valores de forma ordenada.

Consejo de adaptación profesional

En un wizard real, valida cada paso antes de permitir avanzar al siguiente.

22. Skeleton loader

¿Qué hace?

Muestra una estructura de carga temporal antes de renderizar datos reales. Es muy usado en páginas modernas para que la espera se sienta más profesional.

¿Cuándo usarlo?

Úsalo mientras esperas datos de una API, una búsqueda, cards de productos, listas o tablas.

Estructura que debes copiar

HTML mínimo

```
<section id="contentLoader" class="skeleton-grid">
  <article class="skeleton-card"></article>
  <article class="skeleton-card"></article>
  <article class="skeleton-card"></article>
</section>
```

CSS necesario

```
.skeleton-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(180px, 1fr));
  gap: 1rem;
}

.skeleton-card {
  height: 140px;
  border-radius: 1rem;
  background: linear-gradient(90deg, #e2e8f0 25%, #f8fafc 37%, #e2e8f0 63%);
  background-size: 400% 100%;
  animation: skeleton-loading 1.2s infinite;
}

@keyframes skeleton-loading {
  0% { background-position: 100% 0; }
  100% { background-position: 0 0; }
}
```

funciones.js - versión modular con export

```
export class SkeletonLoader {
  constructor(containerId) {
    this.container = document.getElementById(containerId);
  }

  show(count = 3) {
    this.container.innerHTML = Array.from({ length: count }, () => {
      return `<article class="skeleton-card"></article>`;
    }).join("");
  }

  hideWithContent(html) {
    this.container.innerHTML = html;
  }
}
```

script.js - importación e inicialización

```
import { SkeletonLoader } from "../funciones.js";

const loader = new SkeletonLoader("contentLoader");
loader.show(3);

setTimeout(() => {
  loader.hideWithContent(`
    <article class="card">Contenido cargado</article>
    <article class="card">Otro contenido</article>
  `);
}, 1500);
```

script-normal.js - versión normal sin módulos

```
const contentLoader = document.getElementById("contentLoader");

function showSkeleton(count = 3) {
  contentLoader.innerHTML = Array.from({ length: count }, () => {
    return `<article class="skeleton-card"></article>`;
  }).join("");
}

showSkeleton(3);
setTimeout(() => {
  contentLoader.innerHTML = `<article class="card">Contenido cargado</article>`;
}, 1500);
```

Variables, clases e IDs que puedes cambiar

Elemento	Para qué sirve	Cómo adaptarlo
count	Cantidad de placeholders.	Hazlo igual al número aproximado de cards reales.
.skeleton-card	Diseño del placeholder.	Cambia alto, radio o animación.
hideWithContent	Reemplaza skeleton por contenido real.	Pásale HTML seguro o creado con DOM.

Paso a paso para reciclarlo en otro proyecto

1. Agrega un contenedor donde irá la carga.
2. Copia el CSS del skeleton y keyframes.
3. Muestra skeleton antes de traer datos.
4. Cuando lleguen los datos, reemplaza con contenido real.
5. Prueba con setTimeout y luego cámbialo por fetch real.

Qué debes revisar al integrarlo

- Que los ids del HTML coincidan con los ids usados en JavaScript.
- Que no estés usando el mismo id dos veces en la misma página.
- Que las clases activas como is-active, is-open, is-valid o is-invalid estén escritas igual en HTML, CSS y JS.
- Que si usas módulos el script se cargue con type="module".
- Que si usas la versión normal no dejes imports ni exports en el archivo.

Errores comunes

- Dejar skeleton infinito si falla la API.
- Usar demasiados skeletons y saturar la pantalla.
- Renderizar HTML no confiable sin sanitizar.

Cómo cambiar nombres sin romper la funcionalidad

Paso	Qué hacer	Recomendación
1	Cambia primero el id en el HTML.	Ejemplo: loginForm por formAcceso.
2	Busca ese mismo id en script.js o script-normal.js.	Debe quedar exactamente igual.
3	Si el CSS usa una clase, cambia la clase en HTML y CSS al mismo tiempo.	No cambies solo uno de los dos archivos.
4	Si hay data-* como data-tab, data-step o data-value, revisa qué lo lee en JS.	Los atributos data-* suelen ser el puente entre HTML y JS.
5	Prueba la funcionalidad aislada antes de mezclarla con otros componentes.	Así sabes si el error viene del componente o de otra parte.

Adaptaciones comunes en proyectos reales

- Para una landing page: usa Skeleton loader dentro de una sección con clase .section y manten el diseño centrado.
- Para un dashboard: coloca Skeleton loader dentro de una card reutilizable y controla el estado con clases como is-active o is-open.
- Para un formulario: valida ids, required, minlength y data-error antes de enviar datos.
- Para un proyecto con muchos componentes: inicializa esta funcionalidad desde una función init22() o desde initApp().
- Para trabajo en equipo: deja comentarios cortos indicando qué HTML, CSS y JS pertenecen a Skeleton loader.

Mini guía de depuración si no funciona

1. Abre la consola del navegador con F12 y revisa si aparece un error rojo.
2. Si el error dice null, significa que JavaScript no encontró un elemento del DOM; revisa ids y clases.
3. Si el diseño no cambia, revisa que la clase que agrega JavaScript exista también en CSS.
4. Si usas import/export, abre el proyecto con Live Server; no lo abras solo con doble clic.
5. Si el componente depende de localStorage o sessionStorage, limpia el almacenamiento del navegador y prueba de nuevo.
6. Comenta temporalmente otros componentes para confirmar que no hay conflicto de nombres.

Patrón mental para reutilizarlo

Cada funcionalidad de esta guía se puede entender como una combinación de tres capas: la capa HTML define la estructura, la capa CSS define el diseño visual y la capa JavaScript controla el comportamiento. Para reciclarla bien, no copies solo una capa: copia la estructura mínima, los estilos necesarios y la inicialización correspondiente. Después, cambia nombres y valores de forma ordenada.

Consejo de adaptación profesional

Incluye estado de error si la API falla; no dejes el skeleton cargando para siempre.

23. Select personalizado

¿Qué hace?

Reemplaza el select nativo por un selector bonito, controlable y editable. Permite opciones con estilo, flecha personalizada, blur y mejor integración visual.

¿Cuándo usarlo?

Úsalo cuando el select nativo se vea mal o no combine con el diseño, como en tarjetas glassmorphism.

Estructura que debes copiar

HTML mínimo

```
<div class="custom-select" id="statusSelect">
  <input type="hidden" id="statusValue" value="pendiente">

  <button type="button" class="custom-select__button">
    <span class="custom-select__value">Pendiente</span>
    <span class="custom-select__arrow">▼</span>
  </button>

  <ul class="custom-select__options">
    <li class="custom-select__option is-selected"
      data-value="pendiente">Pendiente</li>
    <li class="custom-select__option" data-value="proceso">En proceso</li>
    <li class="custom-select__option" data-value="finalizado">Finalizado</li>
  </ul>
</div>
```

CSS necesario

```
.custom-select {
  position: relative;
  width: 100%;
  z-index: 20;
}

.custom-select__button {
  width: 100%;
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: .9rem 1rem;
  border-radius: 1rem;
  border: 1px solid rgba(255, 255, 255, .25);
  background: rgba(255, 255, 255, .14);
  color: inherit;
  cursor: pointer;
}

.custom-select__options {
  position: absolute;
  top: calc(100% + .5rem);
  left: 0;
  width: 100%;
  padding: .5rem;
  margin: 0;
  list-style: none;
  border-radius: 1rem;
  background: rgba(15, 23, 42, .88);
  backdrop-filter: blur(18px);
  opacity: 0;
  transform: translateY(-8px);
  pointer-events: none;
  transition: .2s ease;
}

.custom-select.is-open .custom-select__options {
  opacity: 1;
  transform: translateY(0);
  pointer-events: auto;
}

.custom-select__option {
  padding: .75rem .9rem;
  border-radius: .8rem;
  cursor: pointer;
}

.custom-select__option:hover,
.custom-select__option.is-selected {
  background: linear-gradient(135deg, #2563eb, #7c3aed);
}
```

funciones.js - versión modular con export

```
export class CustomSelect {
  constructor(rootId) {
    this.root = document.getElementById(rootId);
    this.hiddenInput = this.root.querySelector("input[type='hidden']");
    this.button = this.root.querySelector(".custom-select__button");
    this.valueText = this.root.querySelector(".custom-select__value");
    this.options = this.root.querySelectorAll(".custom-select__option");
    this.init();
  }

  init() {
    this.button.addEventListener("click", () => {
      this.root.classList.toggle("is-open");

      this.options.forEach((option) => {
        option.addEventListener("click", () => this.select(option));
      });

      document.addEventListener("click", (event) => {
        if (!this.root.contains(event.target))
          this.root.classList.remove("is-open");
      });
    });
  }

  select(option) {
    this.hiddenInput.value = option.dataset.value;
    this.valueText.textContent = option.textContent;
    this.options.forEach((item) => item.classList.remove("is-selected"));
    option.classList.add("is-selected");
    this.root.classList.remove("is-open");
    this.hiddenInput.dispatchEvent(new Event("change"));
  }
}
```

script.js - importación e inicialización

```
import { CustomSelect } from "./funciones.js";

new CustomSelect("statusSelect");

// Para leer el valor:
const status = document.getElementById("statusValue").value;
```

script-normal.js - versión normal sin módulos

```
const customSelect = document.getElementById("statusSelect");
const customButton = customSelect.querySelector(".custom-select__button");
const customValue = customSelect.querySelector(".custom-select__value");
const hiddenInput = customSelect.querySelector("input[type='hidden']");
const customOptions = customSelect.querySelectorAll(".custom-select__option");

customButton.addEventListener("click", () => {
  customSelect.classList.toggle("is-open");
  customOptions.forEach((option) => {
    option.addEventListener("click", () => {
      hiddenInput.value = option.dataset.value;
      customValue.textContent = option.textContent;
      customSelect.classList.remove("is-open");
    });
  });
});
```

Variables, clases e IDs que puedes cambiar

Elemento	Para qué sirve	Cómo adaptarlo
#statusSelect	Contenedor del selector.	Cámbialo por unidad, categoría, ciudad, etc.
#statusValue	Input oculto con el valor real.	Tu JS debe leer este value.
data-value	Valor lógico de cada opción.	Cámbialo según lo que tu app necesite.
.custom-select__value	Texto visible actual.	Debe coincidir con la opción inicial.

Paso a paso para reciclarlo en otro proyecto

1. Copia el bloque HTML completo del custom select.
2. Cambia el id del contenedor y del input hidden.
3. Agrega, elimina o edita `li.custom-select__option`.
4. Cada `li` debe tener `data-value` y texto visible.
5. Copia el CSS del selector.
6. Inicializa `new CustomSelect("idDelSelector")`.
7. Lee el valor desde el input hidden con `.value`.

Qué debes revisar al integrarlo

- Que los ids del HTML coincidan con los ids usados en JavaScript.
- Que no estés usando el mismo id dos veces en la misma página.
- Que las clases activas como `is-active`, `is-open`, `is-valid` o `is-invalid` estén escritas igual en HTML, CSS y JS.
- Que si usas módulos el script se cargue con `type="module"`.
- Que si usas la versión normal no dejes `imports` ni `exports` en el archivo.

Errores comunes

- Leer el texto del botón en vez del input oculto.
- Olvidar `data-value` en las opciones.
- No subir `z-index` y que el menú quede detrás de otra tarjeta.

Cómo cambiar nombres sin romper la funcionalidad

Paso	Qué hacer	Recomendación
1	Cambia primero el id en el HTML.	Ejemplo: <code>loginForm</code> por <code>formAcceso</code> .
2	Busca ese mismo id en <code>script.js</code> o <code>script-normal.js</code> .	Debe quedar exactamente igual.
3	Si el CSS usa una clase, cambia la clase en HTML y CSS al mismo tiempo.	No cambies solo uno de los dos archivos.
4	Si hay <code>data-*</code> como <code>data-tab</code> , <code>data-step</code> o <code>data-value</code> , revisa qué lo lee en JS.	Los atributos <code>data-*</code> suelen ser el puente entre HTML y JS.
5	Prueba la funcionalidad aislada antes de mezclarla con otros componentes.	Así sabes si el error viene del componente o de otra parte.

Adaptaciones comunes en proyectos reales

- Para una landing page: usa Select personalizado dentro de una sección con clase `.section` y manten el diseño centrado.
- Para un dashboard: coloca Select personalizado dentro de una card reutilizable y controla el estado con clases como `is-active` o `is-open`.
- Para un formulario: valida ids, `required`, `minlength` y `data-error` antes de enviar datos.
- Para un proyecto con muchos componentes: inicializa esta funcionalidad desde una función `init23()` o desde `initApp()`.
- Para trabajo en equipo: deja comentarios cortos indicando qué HTML, CSS y JS pertenecen a Select personalizado.

Mini guía de depuración si no funciona

1. Abre la consola del navegador con F12 y revisa si aparece un error rojo.
2. Si el error dice `null`, significa que JavaScript no encontró un elemento del DOM; revisa ids y clases.
3. Si el diseño no cambia, revisa que la clase que agrega JavaScript exista también en CSS.

4. Si usas import/export, abre el proyecto con Live Server; no lo abras solo con doble clic.
5. Si el componente depende de localStorage o sessionStorage, limpia el almacenamiento del navegador y prueba de nuevo.
6. Comenta temporalmente otros componentes para confirmar que no hay conflicto de nombres.

Patrón mental para reutilizarlo

Cada funcionalidad de esta guía se puede entender como una combinación de tres capas: la capa HTML define la estructura, la capa CSS define el diseño visual y la capa JavaScript controla el comportamiento. Para reciclarla bien, no copies solo una capa: copia la estructura mínima, los estilos necesarios y la inicialización correspondiente. Después, cambia nombres y valores de forma ordenada.

Consejo de adaptación profesional

Mantén un input hidden porque facilita leer el valor desde formularios y desde JavaScript.

24. Tabs / pestañas

¿Qué hace?

Permite cambiar entre paneles internos sin recargar la página. Se usa en perfiles, productos, dashboards, documentación y configuraciones.

¿Cuándo usarlo?

Úsalo cuando varias secciones están relacionadas pero no deben verse todas al mismo tiempo.

Estructura que debes copiar

HTML mínimo

```
<div class="tabs" id="productTabs">
  <div class="tabs__buttons">
    <button class="is-active" data-tab="description">Descripción</button>
    <button data-tab="details">Detalles</button>
    <button data-tab="reviews">Reseñas</button>
  </div>

  <section class="tab-panel is-active" id="description">Contenido de
    descripción</section>
  <section class="tab-panel" id="details">Contenido de detalles</section>
  <section class="tab-panel" id="reviews">Contenido de reseñas</section>
</div>
```

CSS necesario

```
.tabs__buttons {
  display: flex;
  flex-wrap: wrap;
  gap: .5rem;
  margin-bottom: 1rem;
}

.tabs__buttons button {
  width: auto;
  padding: .75rem 1rem;
  border-radius: 999px;
  background: rgba(255, 255, 255, .12);
  color: inherit;
}

.tabs__buttons button.is-active {
  background: linear-gradient(135deg, #2563eb, #7c3aed);
  color: #fff;
}

.tab-panel {
  display: none;
  padding: 1rem;
  border-radius: 1rem;
  background: rgba(255, 255, 255, .08);
}

.tab-panel.is-active { display: block; }
```

funciones.js - versión modular con export

```
export class Tabs {
  constructor(rootId) {
    this.root = document.getElementById(rootId);
    this.buttons = this.root.querySelectorAll("[data-tab]");
    this.panels = this.root.querySelectorAll(".tab-panel");
    this.init();
  }

  init() {
    this.buttons.forEach((button) => {
      button.addEventListener("click", () => this.activate(button.dataset.tab));
    });
  }

  activate(tabId) {
    this.buttons.forEach((button) => {
      button.classList.toggle("is-active", button.dataset.tab === tabId);
    });

    this.panels.forEach((panel) => {
      panel.classList.toggle("is-active", panel.id === tabId);
    });
  }
}
```

script.js - importación e inicialización

```
import { Tabs } from "../funciones.js";

new Tabs("productTabs");
```

script-normal.js - versión normal sin módulos

```
const tabs = document.getElementById("productTabs");
const tabButtons = tabs.querySelectorAll("[data-tab]");
const tabPanels = tabs.querySelectorAll(".tab-panel");

tabButtons.forEach((button) => {
  button.addEventListener("click", () => {
    const tabId = button.dataset.tab;
    tabButtons.forEach((btn) => btn.classList.toggle("is-active", btn ===
      button));
    tabPanels.forEach((panel) => panel.classList.toggle("is-active", panel.id ===
      tabId));
  });
});
```

Variables, clases e IDs que puedes cambiar

Elemento	Para qué sirve	Cómo adaptarlo
data-tab	Nombre del panel que se activará.	Debe coincidir con el id del panel.
.is-active	Clase que muestra botón y panel activos.	No cambies el nombre si no ajustas JS/CSS.
#productTabs	Contenedor raíz.	Permite tener varias zonas de tabs independientes.

Paso a paso para reciclarlo en otro proyecto

1. Copia la estructura .tabs completa.
2. Cambia textos de botones y contenido de paneles.
3. Asegúrate de que data-tab="x" coincida con id="x".
4. Copia los estilos de botones y paneles.
5. Inicializa new Tabs("productTabs").
6. Prueba todos los botones y verifica que solo haya un panel activo.

Qué debes revisar al integrarlo

- Que los ids del HTML coincidan con los ids usados en JavaScript.
- Que no estés usando el mismo id dos veces en la misma página.
- Que las clases activas como is-active, is-open, is-valid o is-invalid estén escritas igual en HTML, CSS y JS.
- Que si usas módulos el script se cargue con type="module".
- Que si usas la versión normal no dejes imports ni exports en el archivo.

Errores comunes

- data-tab no coincide con ningún id.
- No poner is-active inicial y que no se vea nada.
- Usar ids repetidos en varias secciones de tabs.

Cómo cambiar nombres sin romper la funcionalidad

Paso	Qué hacer	Recomendación
1	Cambia primero el id en el HTML.	Ejemplo: loginForm por formAcceso.
2	Busca ese mismo id en script.js o script-normal.js.	Debe quedar exactamente igual.
3	Si el CSS usa una clase, cambia la clase en HTML y CSS al mismo tiempo.	No cambies solo uno de los dos archivos.
4	Si hay data-* como data-tab, data-step o data-value, revisa qué lo lee en JS.	Los atributos data-* suelen ser el puente entre HTML y JS.
5	Prueba la funcionalidad aislada antes de mezclarla con otros componentes.	Así sabes si el error viene del componente o de otra parte.

Adaptaciones comunes en proyectos reales

- Para una landing page: usa Tabs / pestañas dentro de una sección con clase .section y manten el diseño centrado.
- Para un dashboard: coloca Tabs / pestañas dentro de una card reutilizable y controla el estado con clases como is-active o is-open.
- Para un formulario: valida ids, required, minlength y data-error antes de enviar datos.
- Para un proyecto con muchos componentes: inicializa esta funcionalidad desde una función init24() o desde initApp().
- Para trabajo en equipo: deja comentarios cortos indicando qué HTML, CSS y JS pertenecen a Tabs / pestañas.

Mini guía de depuración si no funciona

1. Abre la consola del navegador con F12 y revisa si aparece un error rojo.
2. Si el error dice null, significa que JavaScript no encontró un elemento del DOM; revisa ids y clases.
3. Si el diseño no cambia, revisa que la clase que agrega JavaScript exista también en CSS.
4. Si usas import/export, abre el proyecto con Live Server; no lo abras solo con doble clic.
5. Si el componente depende de localStorage o sessionStorage, limpia el almacenamiento del navegador y prueba de nuevo.
6. Comenta temporalmente otros componentes para confirmar que no hay conflicto de nombres.

Patrón mental para reutilizarlo

Cada funcionalidad de esta guía se puede entender como una combinación de tres capas: la capa HTML define la estructura, la capa CSS define el diseño visual y la capa JavaScript controla el comportamiento. Para reciclarla bien, no copies solo una capa: copia la estructura mínima, los estilos necesarios y la inicialización correspondiente. Después, cambia nombres y valores de forma ordenada.

Consejo de adaptación profesional

Si las tabs cambian contenido importante, considera actualizar la URL con hash para compartir el estado.

25. Acordeón / FAQ

¿Qué hace?

Abre y cierra bloques de contenido. Es perfecto para preguntas frecuentes, secciones de ayuda, detalles de producto y menús informativos.

¿Cuándo usarlo?

Úsalo cuando tienes mucho contenido textual y quieres ahorrar espacio visual.

Estructura que debes copiar

HTML mínimo

```
<section class="accordion" id="faqAccordion">
  <article class="accordion_item">
    <button type="button" class="accordion_button">¿Qué incluye el
      servicio?</button>
    <div class="accordion_content">
      <p>Incluye diseño, desarrollo y soporte inicial.</p>
    </div>
  </article>

  <article class="accordion_item">
    <button type="button" class="accordion_button">¿Cuánto tarda?</button>
    <div class="accordion_content">
      <p>Depende del alcance del proyecto.</p>
    </div>
  </article>
</section>
```

CSS necesario

```
.accordion_item {
  border-bottom: 1px solid rgba(255, 255, 255, .15);
}

.accordion_button {
  width: 100%;
  padding: 1rem;
  border: 0;
  background: transparent;
  color: inherit;
  text-align: left;
  font-weight: 700;
  cursor: pointer;
}

.accordion_content {
  max-height: 0;
  overflow: hidden;
  transition: max-height .25s ease;
}

.accordion_item.is-open .accordion_content {
  max-height: 220px;
}

.accordion_content p {
  padding: 0 1rem 1rem;
  margin: 0;
  color: var(--text-muted, #cbd5e1);
}
```

funciones.js - versión modular con export

```
export class Accordion {
  constructor(rootId, allowMultiple = false) {
    this.root = document.getElementById(rootId);
    this.items = this.root.querySelectorAll(".accordion__item");
    this.allowMultiple = allowMultiple;
    this.init();
  }

  init() {
    this.items.forEach((item) => {
      const button = item.querySelector(".accordion__button");
      button.addEventListener("click", () => this.toggle(item));
    });
  }

  toggle(selectedItem) {
    if (!this.allowMultiple) {
      this.items.forEach((item) => {
        if (item !== selectedItem) item.classList.remove("is-open");
      });
    }

    selectedItem.classList.toggle("is-open");
  }
}
```

script.js - importación e inicialización

```
import { Accordion } from "./funciones.js";

new Accordion("faqAccordion", false);
```

script-normal.js - versión normal sin módulos

```
const faqAccordion = document.getElementById("faqAccordion");
const accordionItems = faqAccordion.querySelectorAll(".accordion__item");

accordionItems.forEach((item) => {
  const button = item.querySelector(".accordion__button");
  button.addEventListener("click", () => {
    accordionItems.forEach((otherItem) => {
      if (otherItem !== item) otherItem.classList.remove("is-open");
    });
    item.classList.toggle("is-open");
  });
});
```

Variables, clases e IDs que puedes cambiar

Elemento	Para qué sirve	Cómo adaptarlo
#faqAccordion	Contenedor del acordeón.	Cambia el id si tienes varios acordeones.
.accordion__item	Cada bloque desplegable.	Duplica este article para más preguntas.
allowMultiple	Permite abrir varios bloques a la vez.	true abre varios; false solo uno.
max-height	Altura máxima del contenido abierto.	Auméntala si el contenido es largo.

Paso a paso para reciclarlo en otro proyecto

1. Copia el section.accordion completo.
2. Duplica article.accordion__item para cada pregunta o bloque.
3. Cambia el texto del botón y del contenido.
4. Copia el CSS y ajusta max-height si hace falta.
5. Inicializa new Accordion("faqAccordion", false).
6. Prueba abrir una pregunta y confirmar que las demás se cierran.

Qué debes revisar al integrarlo

- Que los ids del HTML coincidan con los ids usados en JavaScript.
- Que no estés usando el mismo id dos veces en la misma página.
- Que las clases activas como is-active, is-open, is-valid o is-invalid estén escritas igual en HTML, CSS y JS.
- Que si usas módulos el script se cargue con type="module".
- Que si usas la versión normal no dejes imports ni exports en el archivo.

Errores comunes

- Poner contenido muy largo y que se corte por max-height.
- No usar button y perder accesibilidad.
- Olvidar que allowMultiple cambia el comportamiento.

Cómo cambiar nombres sin romper la funcionalidad

Paso	Qué hacer	Recomendación
1	Cambia primero el id en el HTML.	Ejemplo: loginForm por formAcceso.
2	Busca ese mismo id en script.js o script-normal.js.	Debe quedar exactamente igual.
3	Si el CSS usa una clase, cambia la clase en HTML y CSS al mismo tiempo.	No cambies solo uno de los dos archivos.
4	Si hay data-* como data-tab, data-step o data-value, revisa qué lo lee en JS.	Los atributos data-* suelen ser el puente entre HTML y JS.
5	Prueba la funcionalidad aislada antes de mezclarla con otros componentes.	Así sabes si el error viene del componente o de otra parte.

Adaptaciones comunes en proyectos reales

- Para una landing page: usa Acordeón / FAQ dentro de una sección con clase .section y manten el diseño centrado.
- Para un dashboard: coloca Acordeón / FAQ dentro de una card reutilizable y controla el estado con clases como is-active o is-open.
- Para un formulario: valida ids, required, minlength y data-error antes de enviar datos.
- Para un proyecto con muchos componentes: inicializa esta funcionalidad desde una función init25() o desde initApp().
- Para trabajo en equipo: deja comentarios cortos indicando qué HTML, CSS y JS pertenecen a Acordeón / FAQ.

Mini guía de depuración si no funciona

1. Abre la consola del navegador con F12 y revisa si aparece un error rojo.
2. Si el error dice null, significa que JavaScript no encontró un elemento del DOM; revisa ids y clases.
3. Si el diseño no cambia, revisa que la clase que agrega JavaScript exista también en CSS.
4. Si usas import/export, abre el proyecto con Live Server; no lo abras solo con doble clic.
5. Si el componente depende de localStorage o sessionStorage, limpia el almacenamiento del navegador y prueba de nuevo.
6. Comenta temporalmente otros componentes para confirmar que no hay conflicto de nombres.

Patrón mental para reutilizarlo

Cada funcionalidad de esta guía se puede entender como una combinación de tres capas: la capa HTML define la estructura, la capa CSS define el diseño visual y la capa JavaScript controla el comportamiento. Para reciclarla bien, no copies solo una capa: copia la estructura mínima, los estilos necesarios y la inicialización correspondiente. Después, cambia nombres y valores de forma ordenada.

Consejo de adaptación profesional

Para accesibilidad avanzada, puedes agregar aria-expanded al botón y actualizarlo con JavaScript.

Apéndice: mapa rápido de integración

#	Funcionalidad	Archivos	Versión JS	Revisión clave
10	Contador animado	HTML + CSS + JS	Modular o normal	Revisar ids, clases y data-*
11	Login demo	HTML + CSS + JS	Modular o normal	Revisar ids, clases y data-*
12	Logout demo	HTML + CSS + JS	Modular o normal	Revisar ids, clases y data-*
13	Recordar usuario	HTML + CSS + JS	Modular o normal	Revisar ids, clases y data-*
14	Password toggle	HTML + CSS + JS	Modular o normal	Revisar ids, clases y data-*
15	Validación de formulario	HTML + CSS + JS	Modular o normal	Revisar ids, clases y data-*
16	Mensajes por campo	HTML + CSS + JS	Modular o normal	Revisar ids, clases y data-*
17	Character counter	HTML + CSS + JS	Modular o normal	Revisar ids, clases y data-*
18	Range preview	HTML + CSS + JS	Modular o normal	Revisar ids, clases y data-*
19	File info	HTML + CSS + JS	Modular o normal	Revisar ids, clases y data-*
20	Autosave draft	HTML + CSS + JS	Modular o normal	Revisar ids, clases y data-*
21	Stepper / Wizard	HTML + CSS + JS	Modular o normal	Revisar ids, clases y data-*
22	Skeleton loader	HTML + CSS + JS	Modular o normal	Revisar ids, clases y data-*
23	Select personalizado	HTML + CSS + JS	Modular o normal	Revisar ids, clases y data-*
24	Tabs / pestañas	HTML + CSS + JS	Modular o normal	Revisar ids, clases y data-*
25	Acordeón / FAQ	HTML + CSS + JS	Modular o normal	Revisar ids, clases y data-*

Checklist final antes de copiar a otro proyecto

- Copiar solo la funcionalidad necesaria para no llenar el proyecto de código que no vas a usar.
- Mantener una sola estrategia de JavaScript: modular o normal.
- Si usas módulos, index.html debe tener script type="module".
- Si usas script normal, elimina export, import y clases que no inicialices.
- Revisar en móvil, tablet y escritorio.
- Probar estados vacíos, errores, valores válidos y valores extremos.
- Cambiar nombres de localStorage para evitar choques entre proyectos.
- Usar nombres claros para ids y clases: loginForm, productTabs, faqAccordion, etc.
- Comentar el código si lo vas a entregar como tarea o compartir con el equipo.

Guia de reciclaje de componentes web

Funcionalidades 26 a 40 - continuacion del recurso unificado

HTML + CSS + JavaScript modular y version normal

Este documento mantiene el mismo enfoque de la guia anterior: cada funcionalidad se explica como una pieza reciclable. La idea es que puedas copiar el bloque HTML, el bloque CSS y el bloque JavaScript, cambiar los ids, clases, data-attributes o variables necesarias, y reutilizarlo en cualquier proyecto web.

El orden sigue la tabla de contenido del recurso unificado: desde la funcionalidad 26, Modal, hasta la 40, Exportar JSON.

Rango	Funcionalidades cubiertas	Objetivo de esta guia
26 a 40	Modal, Toast, Carrusel, Tooltip, Badge, Copy clipboard, Cards dinamicas, Busqueda debounce, Filtros chips, Ordenamiento, Paginacion, Tabla ordenable, Filtro de tabla, Notas localStorage y Exportar JSON.	Aprender a reciclar cada componente en otros proyectos con instrucciones claras, variables a cambiar y ejemplos en codigo modular y normal.

Tabla de contenido

- 26. Modal reutilizable
- 27. Toast / notificaciones flotantes
- 28. Carrusel / slider
- 29. Tooltip / ayuda contextual
- 30. Badge / indicador numerico
- 31. Copy clipboard / copiar al portapapeles
- 32. Cards dinamicas desde un array
- 33. Búsqueda con debounce
- 34. Filtros chips por categoria
- 35. Ordenamiento de resultados
- 36. Paginacion
- 37. Tabla ordenable por columnas
- 38. Filtro de tabla por texto
- 39. Notas con localStorage
- 40. Exportar JSON

Como usar esta guia

- 1 Elige la funcionalidad que necesitas reutilizar.
- 2 Copia primero el HTML minimo del componente.
- 3 Copia el CSS asociado y modifica colores, tamaños o variables visuales.
- 4 Escoge una sola version de JavaScript: modular con export/import o normal sin modulos.
- 5 Cambia ids, clases y data-attributes segun la tabla de variables de cada funcionalidad.
- 6 Prueba la funcionalidad con el checklist antes de integrarla con otros componentes.

Regla importante: no mezcles la version modular y la version normal al mismo tiempo. Si usas import/export, carga el HTML con type="module". Si usas JS normal, usa un script comun sin import/export.

Convenciones que se repiten en las funcionalidades 26 a 40

Convencion	Para que sirve	Como adaptarla
data-*	Guarda informacion en HTML para que JavaScript la lea sin depender del texto visible.	Cambia valores como data-copy, data-category, data-page o data-open-modal segun tu proyecto.
is-active / is-open / is-hidden	Clases de estado agregadas por JavaScript para cambiar la vista.	No las escribas manualmente salvo estado inicial; JS debe agregarlas o quitarlas.
containerId	Id del contenedor donde se renderiza o controla el componente.	Debe coincidir con el id del HTML.
callback / onResults / onPageChange	Funcion que se ejecuta cuando un componente produce resultados.	Usala para conectar filtros, busqueda, paginacion o render de cards.
localStorage key	Nombre con el que guardas datos en el navegador.	Cambia claves como kit-notes si vas a mezclar varios proyectos.

Ejemplo de conexion entre componentes: Cards dinamicas + Busqueda debounce + Filtros chips + Ordenamiento + Paginacion. En proyectos reales, lo mas limpio es tener un estado global con los datos originales, filtros activos, texto de busqueda, orden elegido y pagina actual.

26. Modal reutilizable

Aspecto	Explicacion
Que hace	Ventana emergente para confirmar acciones, mostrar formularios, avisos, detalles o contenido que no debe ocupar toda la pagina.
Donde se usa en industria	Confirmar eliminaciones, login rapido, politicas, vista previa de producto, formulario de contacto, mensajes de exito o errores importantes.
Como se personaliza	Cambia el id del modal, los botones data-open-modal y data-close-modal, el titulo, el contenido y el comportamiento de cierre.

Mapa rapido de reciclaje

- 1 Ubica el lugar exacto del HTML donde quieres insertar el componente.
- 2 Copia el HTML minimo y cambia textos visibles, ids y data-attributes.
- 3 Copia el CSS y ajusta colores, separaciones, radios, sombras y responsive si hace falta.
- 4 Escoge JS modular o JS normal, no ambos al mismo tiempo.
- 5 Inicializa el componente con el id correcto y prueba la interaccion principal.

HTML minimo que debes copiar

index.html

```
<button class="btn" data-open-modal="modalContacto">Abrir modal</button>

<div class="modal" id="modalContacto" aria-hidden="true">
  <div class="modal__overlay" data-close-modal></div>

  <article class="modal__content" role="dialog" aria-modal="true" aria-labelledby="modalTitulo">
    <button class="modal__close" type="button" data-close-modal aria-label="Cerrar modal">x</button>
    <h2 id="modalTitulo">Contactar asesor</h2>
    <p>Completa tus datos y nos comunicaremos contigo.</p>
    <form class="form">
      <input type="text" placeholder="Nombre" required>
      <input type="email" placeholder="Correo" required>
      <button class="btn" type="submit">Enviar</button>
    </form>
  </article>
</div>
```

CSS del componente

styles.css

```

.modal {
  position: fixed;
  inset: 0;
  display: grid;
  place-items: center;
  padding: 24px;
  opacity: 0;
  visibility: hidden;
  pointer-events: none;
  z-index: 1000;
  transition: opacity .25s ease, visibility .25s ease;
}
.modal.is-open {
  opacity: 1;
  visibility: visible;
  pointer-events: auto;
}
.modal__overlay {
  position: absolute;
  inset: 0;
  background: rgba(15, 23, 42, .65);
  backdrop-filter: blur(10px);
}
.modal__content {
  position: relative;
  width: min(520px, 100%);
  padding: 28px;
  border-radius: 22px;
  background: rgba(255,255,255,.92);
  color: #0f172a;
  box-shadow: 0 30px 80px rgba(0,0,0,.35);
  transform: translateY(12px) scale(.98);
  transition: transform .25s ease;
}
.modal.is-open .modal__content {
  transform: translateY(0) scale(1);
}
.modal__close {
  position: absolute;
  top: 12px;
  right: 12px;
  border: 0;
  border-radius: 999px;
  padding: 8px 11px;
  cursor: pointer;
}

```

JavaScript modular con export/import

Usa esta version si tu HTML carga `<script type="module" src="script.js"></script>`. La clase o funcion vive en `funciones.js` y la instancia se hace en `script.js`.

funciones.js + script.js

```

export class ModalManager {
  constructor() {
    this.openButtons = document.querySelectorAll('[data-open-modal]');
    this.closeButtons = document.querySelectorAll('[data-close-modal]');
    this.activeModal = null;
    this.init();
  }

  init() {
    this.openButtons.forEach((button) => {
      button.addEventListener('click', () => this.open(button.dataset.openModal));
    });

    this.closeButtons.forEach((button) => {
      button.addEventListener('click', () => this.closeActive());
    });

    document.addEventListener('keydown', (event) => {
      if (event.key === 'Escape') this.closeActive();
    });
  }

  open(modalId) {
    const modal = document.getElementById(modalId);
    if (!modal) return;
    modal.classList.add('is-open');
    modal.setAttribute('aria-hidden', 'false');
    document.body.classList.add('no-scroll');
    this.activeModal = modal;
  }

  closeActive() {
    if (!this.activeModal) return;
    this.activeModal.classList.remove('is-open');
    this.activeModal.setAttribute('aria-hidden', 'true');
    document.body.classList.remove('no-scroll');
    this.activeModal = null;
  }
}

// script.js
import { ModalManager } from './funciones.js';
new ModalManager();

```

JavaScript normal sin modulos

Usa esta version si prefieres tener todo en un solo archivo como script-normal.js y cargarlo con `<script src="script-normal.js"></script>`.

script-normal.js


```

class ModalManager {
  constructor() {
    this.openButtons = document.querySelectorAll('[data-open-modal]');
    this.closeButtons = document.querySelectorAll('[data-close-modal]');
    this.activeModal = null;
    this.init();
  }
  init() {
    this.openButtons.forEach((button) => {
      button.addEventListener('click', () => this.open(button.dataset.openModal));
    });
    this.closeButtons.forEach((button) => {
      button.addEventListener('click', () => this.closeActive());
    });
    document.addEventListener('keydown', (event) => {
      if (event.key === 'Escape') this.closeActive();
    });
  }
  open(modalId) {
    const modal = document.getElementById(modalId);
    if (!modal) return;
    modal.classList.add('is-open');
    modal.setAttribute('aria-hidden', 'false');
    document.body.classList.add('no-scroll');
    this.activeModal = modal;
  }
  closeActive() {
    if (!this.activeModal) return;
    this.activeModal.classList.remove('is-open');
    this.activeModal.setAttribute('aria-hidden', 'true');
    document.body.classList.remove('no-scroll');
    this.activeModal = null;
  }
}
new ModalManager();

```

Variables, clases, ids y atributos que debes cambiar

Elemento	Para que sirve	Que debes cambiar
id="modalContacto"	Identificador unico del modal.	Debe coincidir con data-open-modal="modalContacto".
data-open-modal	Indica que boton abre un modal.	Cambia el valor por el id de tu modal.
data-close-modal	Marca elementos que cierran el modal.	Ponlo en la X y en el overlay si quieres cerrar al hacer clic afuera.
.modal.is-open	Clase de estado abierto.	No la escribes en HTML; JS la agrega y la quita.

Implementacion paso a paso

- 1 Copia el bloque HTML del modal al final del body para que quede por encima del resto del contenido.
- 2 Copia el CSS del modal en styles.css. Si ya tienes un z-index alto en navbar, sube el z-index del modal.
- 3 Copia la clase ModalManager en funciones.js si usas modulos, o en script-normal.js si no usas modulos.
- 4 Crea un boton con data-open-modal y asegúrate de que el valor coincida exactamente con el id del modal.
- 5 Prueba abrir, cerrar con la X, cerrar con el fondo y cerrar con la tecla Escape.

Errores comunes y como solucionarlos

- El modal no abre: revisa que el id del modal y data-open-modal sean iguales.
- El modal queda detras de otros elementos: aumenta z-index en .modal.
- La pagina se sigue moviendo al abrir: agrega .no-scroll { overflow: hidden; } al CSS.

- El boton dentro del modal recarga la pagina: si es un formulario, controla el submit con `preventDefault` cuando sea demo.

Checklist de prueba antes de reutilizar

- El componente existe en HTML y el id coincide con el usado en JavaScript.
- No hay ids duplicados en la pagina.
- La clase visual de estado se agrega y se quita correctamente.
- No aparecen errores en la consola del navegador.
- La funcionalidad sigue funcionando en pantalla pequena.
- El componente no rompe otros elementos del proyecto.

Mejoras opcionales

- Conectar el componente con `ToastManager` para mostrar mensajes de exito o error.
- Agregar estados de carga si depende de datos externos.
- Guardar preferencias en `localStorage` si el usuario puede cambiar configuraciones.
- Mejorar accesibilidad con `aria-label`, `aria-hidden`, `focus-visible` y navegacion por teclado cuando aplique.

27. Toast / notificaciones flotantes

Aspecto	Explicacion
Que hace	Muestra mensajes temporales sin interrumpir al usuario. Es ideal para confirmaciones rapidas.
Donde se usa en industria	Guardar cambios, copiar texto, iniciar sesion, mostrar error de API, confirmar envio de formulario o aviso de carrito.
Como se personaliza	Cambia posicion, duracion, colores por tipo y mensaje.

Mapa rapido de reciclaje

- 1 Ubica el lugar exacto del HTML donde quieres insertar el componente.
- 2 Copia el HTML minimo y cambia textos visibles, ids y data-attributes.
- 3 Copia el CSS y ajusta colores, separaciones, radios, sombras y responsive si hace falta.
- 4 Escoge JS modular o JS normal, no ambos al mismo tiempo.
- 5 Inicializa el componente con el id correcto y prueba la interaccion principal.

HTML minimo que debes copiar

index.html

```
<button class="btn" data-toast="Datos guardados correctamente" data-toast-type="success">
  Mostrar toast
</button>

<div class="toast-container" id="toastContainer" aria-live="polite"></div>
```

CSS del componente

styles.css

```
.toast-container {
  position: fixed;
  right: 20px;
  bottom: 20px;
  display: grid;
  gap: 12px;
  z-index: 1200;
}
.toast {
  min-width: 260px;
  max-width: 380px;
  padding: 14px 16px;
  border-radius: 16px;
  color: #fff;
  background: rgba(15, 23, 42, .9);
  border: 1px solid rgba(255,255,255,.18);
  box-shadow: 0 16px 45px rgba(0,0,0,.28);
  backdrop-filter: blur(12px);
  animation: toastIn .25s ease both;
}
.toast--success { background: rgba(22, 163, 74, .92); }
.toast--error { background: rgba(220, 38, 38, .92); }
.toast--info { background: rgba(37, 99, 235, .92); }
.toast.is-leaving { animation: toastOut .25s ease both; }
@keyframes toastIn { from { transform: translateY(12px); opacity: 0; } to { transform: translateY(0); opacity: 1; } }
@keyframes toastOut { to { transform: translateY(12px); opacity: 0; } }
```

JavaScript modular con export/import

Usa esta version si tu HTML carga `<script type="module" src="script.js"></script>`. La clase o funcion vive en funciones.js y la instancia se hace en script.js.

funciones.js + script.js

```
export class ToastManager {
  constructor(containerId = 'toastContainer') {
    this.container = document.getElementById(containerId);
  }

  show(message, type = 'info', duration = 3000) {
    if (!this.container) return;

    const toast = document.createElement('div');
    toast.className = `toast toast--${type}`;
    toast.textContent = message;
    this.container.appendChild(toast);

    setTimeout(() => {
      toast.classList.add('is-leaving');
      toast.addEventListener('animationend', () => toast.remove(), { once: true });
    }, duration);
  }

  bindButtons() {
    document.querySelectorAll('[data-toast]').forEach((button) => {
      button.addEventListener('click', () => {
        this.show(button.dataset.toast, button.dataset.toastType || 'info');
      });
    });
  }
}

// script.js
import { ToastManager } from './funciones.js';
const toast = new ToastManager();
toast.bindButtons();
// toast.show('Producto agregado', 'success');
```

JavaScript normal sin modulos

Usa esta version si prefieres tener todo en un solo archivo como script-normal.js y cargarlo con `<script src="script-normal.js"></script>`.

script-normal.js

```
class ToastManager {
  constructor(containerId = 'toastContainer') {
    this.container = document.getElementById(containerId);
  }
  show(message, type = 'info', duration = 3000) {
    if (!this.container) return;
    const toast = document.createElement('div');
    toast.className = `toast toast--${type}`;
    toast.textContent = message;
    this.container.appendChild(toast);
    setTimeout(() => {
      toast.classList.add('is-leaving');
      toast.addEventListener('animationend', () => toast.remove(), { once: true });
    }, duration);
  }
  bindButtons() {
    document.querySelectorAll('[data-toast]').forEach((button) => {
      button.addEventListener('click', () => {
        this.show(button.dataset.toast, button.dataset.toastType || 'info');
      });
    });
  }
}

const toast = new ToastManager();
toast.bindButtons();
```

Variables, clases, ids y atributos que debes cambiar

Elemento	Para que sirve	Que debes cambiar
toastContainer	Contenedor donde se insertan los mensajes.	Cambia el id si quieres varios contenedores.
data-toast	Texto del mensaje.	Personaliza el texto visible.
data-toast-type	Tipo visual: success, error, info.	Crea mas clases CSS si necesitas warning u otros.
duration	Tiempo antes de desaparecer.	Aumenta a 5000 si el mensaje es largo.

Implementacion paso a paso

1. Agrega el contenedor toastContainer al final del body.
2. Copia los estilos del toast y ajusta right/bottom si lo quieres arriba o a la izquierda.
3. Importa ToastManager o pega la version normal.
4. Usa botones con data-toast o llama toast.show() desde otros eventos.
5. Prueba varios toasts seguidos para confirmar que se apilan bien.

Errores comunes y como solucionarlos

- No aparece nada: falta el div con id toastContainer.
- Sale sin color: revisa que el tipo coincida con una clase .toast--tipo.
- No se elimina: revisa que el navegador ejecute la animacion o reemplaza animationend por toast.remove en setTimeout.

Checklist de prueba antes de reutilizar

- El componente existe en HTML y el id coincide con el usado en JavaScript.
- No hay ids duplicados en la pagina.
- La clase visual de estado se agrega y se quita correctamente.
- No aparecen errores en la consola del navegador.
- La funcionalidad sigue funcionando en pantalla pequena.
- El componente no rompe otros elementos del proyecto.

Mejoras opcionales

- Conectar el componente con ToastManager para mostrar mensajes de exito o error.
- Agregar estados de carga si depende de datos externos.
- Guardar preferencias en localStorage si el usuario puede cambiar configuraciones.
- Mejorar accesibilidad con aria-label, aria-hidden, focus-visible y navegacion por teclado cuando aplique.

28. Carrusel / slider

Aspecto	Explicacion
Que hace	Permite mostrar varios bloques de contenido, imagenes o promociones en un mismo espacio con botones anterior/siguiente.
Donde se usa en industria	Hero de home, testimonios, productos destacados, galeria de imagenes, banners comerciales.
Como se personaliza	Duplica slides, cambia imagenes, textos, velocidad, autoplay y controles.

Mapa rapido de reciclaje

- 1 Ubica el lugar exacto del HTML donde quieres insertar el componente.
- 2 Copia el HTML minimo y cambia textos visibles, ids y data-attributes.
- 3 Copia el CSS y ajusta colores, separaciones, radios, sombras y responsive si hace falta.
- 4 Escoge JS modular o JS normal, no ambos al mismo tiempo.
- 5 Inicializa el componente con el id correcto y prueba la interaccion principal.

HTML minimo que debes copiar

index.html

```
<section class="carousel" id="mainCarousel">
  <div class="carousel__track">
    <article class="carousel__slide is-active">Slide 1 - Promocion principal</article>
    <article class="carousel__slide">Slide 2 - Producto destacado</article>
    <article class="carousel__slide">Slide 3 - Testimonio</article>
  </div>

  <button class="carousel__btn carousel__btn--prev" type="button" data-carousel-prev>Anterior</button>
  <button class="carousel__btn carousel__btn--next" type="button" data-carousel-next>Siguiente</button>

  <div class="carousel__dots" data-carousel-dots></div>
</section>
```

CSS del componente

styles.css

```

.carousel {
  position: relative;
  overflow: hidden;
  border-radius: 24px;
  background: rgba(255,255,255,.12);
  border: 1px solid rgba(255,255,255,.18);
}
.carousel__track { position: relative; min-height: 240px; }
.carousel__slide {
  position: absolute;
  inset: 0;
  display: grid;
  place-items: center;
  padding: 32px;
  opacity: 0;
  transform: translateX(20px);
  transition: opacity .35s ease, transform .35s ease;
}
.carousel__slide.is-active {
  opacity: 1;
  transform: translateX(0);
  position: relative;
}
.carousel__btn {
  position: absolute;
  top: 50%;
  transform: translateY(-50%);
  border: 0;
  border-radius: 999px;
  padding: 10px 14px;
  cursor: pointer;
}
.carousel__btn--prev { left: 12px; }
.carousel__btn--next { right: 12px; }
.carousel__dots {
  display: flex;
  justify-content: center;
  gap: 8px;
  padding: 12px;
}
.carousel__dot {
  width: 10px;
  height: 10px;
  border-radius: 999px;
  border: 0;
  background: rgba(255,255,255,.35);
  cursor: pointer;
}
.carousel__dot.is-active { background: #2563eb; }

```

JavaScript modular con export/import

Usa esta version si tu HTML carga `<script type="module" src="script.js"></script>`. La clase o funcion vive en `funciones.js` y la instancia se hace en `script.js`.

funciones.js + script.js

```

export class Carousel {
  constructor(carouselId, { autoplay = false, interval = 4000 } = {}) {
    this.carousel = document.getElementById(carouselId);
    if (!this.carousel) return;
    this.slides = [...this.carousel.querySelectorAll('.carousel__slide')];
    this.prev = this.carousel.querySelector('[data-carousel-prev]');
    this.next = this.carousel.querySelector('[data-carousel-next]');
    this.dotsContainer = this.carousel.querySelector('[data-carousel-dots]');
    this.index = 0;
    this.autoplay = autoplay;
    this.interval = interval;
    this.timer = null;
    this.init();
  }

  init() {
    this.renderDots();
    this.prev?.addEventListener('click', () => this.goTo(this.index - 1));
    this.next?.addEventListener('click', () => this.goTo(this.index + 1));
    if (this.autoplay) this.start();
  }

  renderDots() {
    this.dotsContainer.innerHTML = this.slides.map((_, i) =>
      `<button class="carousel__dot" data-dot="${i}" aria-label="Ir al slide ${i + 1}"></button>`
    ).join('');
    this.dots = [...this.dotsContainer.querySelectorAll('.carousel__dot')];
    this.dots.forEach((dot) => dot.addEventListener('click', () => this.goTo(Number(dot.dataset.
dot))));
    this.update();
  }

  goTo(newIndex) {
    this.index = (newIndex + this.slides.length) % this.slides.length;
    this.update();
  }

  update() {
    this.slides.forEach((slide, i) => slide.classList.toggle('is-active', i === this.index));
    this.dots?.forEach((dot, i) => dot.classList.toggle('is-active', i === this.index));
  }

  start() { this.timer = setInterval(() => this.goTo(this.index + 1), this.interval); }
  stop() { clearInterval(this.timer); }
}

// script.js
import { Carousel } from './funciones.js';
new Carousel('mainCarousel', { autoplay: true, interval: 5000 });

```

JavaScript normal sin modulos

Usa esta version si prefieres tener todo en un solo archivo como script-normal.js y cargarlo con `<script src="script-normal.js"></script>`.

script-normal.js


```

class Carousel {
  constructor(carouselId, options = {}) {
    this.carousel = document.getElementById(carouselId);
    if (!this.carousel) return;
    this.slides = [...this.carousel.querySelectorAll('.carousel__slide')];
    this.prev = this.carousel.querySelector('[data-carousel-prev]');
    this.next = this.carousel.querySelector('[data-carousel-next]');
    this.dotsContainer = this.carousel.querySelector('[data-carousel-dots]');
    this.index = 0;
    this.autoplay = options.autoplay || false;
    this.interval = options.interval || 4000;
    this.init();
  }
  init() {
    this.renderDots();
    this.prev?.addEventListener('click', () => this.goTo(this.index - 1));
    this.next?.addEventListener('click', () => this.goTo(this.index + 1));
    if (this.autoplay) setInterval(() => this.goTo(this.index + 1), this.interval);
  }
  renderDots() {
    this.dotsContainer.innerHTML = this.slides.map((_, i) => `<button class="carousel__dot" data-
dot="${i}"></button>`).join('');
    this.dots = [...this.dotsContainer.querySelectorAll('.carousel__dot')];
    this.dots.forEach((dot) => dot.addEventListener('click', () => this.goTo(Number(dot.dataset.
dot))));
    this.update();
  }
  goTo(newIndex) {
    this.index = (newIndex + this.slides.length) % this.slides.length;
    this.update();
  }
  update() {
    this.slides.forEach((slide, i) => slide.classList.toggle('is-active', i === this.index));
    this.dots.forEach((dot, i) => dot.classList.toggle('is-active', i === this.index));
  }
}
new Carousel('mainCarousel', { autoplay: true });

```

Variables, clases, ids y atributos que debes cambiar

Elemento	Para que sirve	Que debes cambiar
mainCarousel	Id del carrusel.	Cambia el id si tienes mas de un carrusel.
.carousel__slide	Cada elemento visible del carrusel.	Duplica esta etiqueta para agregar mas slides.
autoplay	Activa cambio automatico.	Pon false para que solo cambie con botones.
interval	Tiempo entre slides.	Valor en milisegundos: 5000 = 5 segundos.

Implementacion paso a paso

- 1 Copia el HTML del carrusel donde quieras mostrarlo.
- 2 Duplica o elimina elementos .carousel__slide segun el numero de contenidos.
- 3 Copia los estilos y ajusta min-height si tus imagenes son mas grandes.
- 4 Inicializa la clase con el id correspondiente.
- 5 Prueba botones, dots y autoplay.

Errores comunes y como solucionarlos

- Solo se ve un slide siempre: revisa que la clase is-active cambie en DevTools.
- Los botones no aparecen: puede estar faltando position: relative en .carousel.
- Los dots no se generan: falta el contenedor data-carousel-dots.

Checklist de prueba antes de reutilizar

- El componente existe en HTML y el id coincide con el usado en JavaScript.
- No hay ids duplicados en la pagina.
- La clase visual de estado se agrega y se quita correctamente.
- No aparecen errores en la consola del navegador.
- La funcionalidad sigue funcionando en pantalla pequena.
- El componente no rompe otros elementos del proyecto.

Mejoras opcionales

- Conectar el componente con ToastManager para mostrar mensajes de exito o error.
- Agregar estados de carga si depende de datos externos.
- Guardar preferencias en localStorage si el usuario puede cambiar configuraciones.
- Mejorar accesibilidad con aria-label, aria-hidden, focus-visible y navegacion por teclado cuando aplique.

29. Tooltip / ayuda contextual

Aspecto	Explicacion
Que hace	Muestra una ayuda corta al pasar el mouse o enfocar un elemento.
Donde se usa en industria	Iconos de ayuda, botones con acciones, formularios, tablas con estados, paneles administrativos.
Como se personaliza	Cambia el texto de data-tooltip, la posicion y el estilo visual.

Mapa rapido de reciclaje

- 1 Ubica el lugar exacto del HTML donde quieres insertar el componente.
- 2 Copia el HTML minimo y cambia textos visibles, ids y data-attributes.
- 3 Copia el CSS y ajusta colores, separaciones, radios, sombras y responsive si hace falta.
- 4 Escoge JS modular o JS normal, no ambos al mismo tiempo.
- 5 Inicializa el componente con el id correcto y prueba la interaccion principal.

HTML minimo que debes copiar

index.html

```
<button class="icon-button" data-tooltip="Editar producto" aria-label="Editar producto">
  Editar
</button>

<span class="info" data-tooltip="Este dato se guarda automaticamente">?</span>
```

CSS del componente

styles.css

```

[data-tooltip] {
  position: relative;
}
[data-tooltip]::before,
[data-tooltip]::after {
  position: absolute;
  left: 50%;
  opacity: 0;
  visibility: hidden;
  pointer-events: none;
  transition: opacity .2s ease, transform .2s ease;
  z-index: 900;
}
[data-tooltip]::before {
  content: attr(data-tooltip);
  bottom: calc(100% + 10px);
  transform: translateX(-50%) translateY(6px);
  width: max-content;
  max-width: 220px;
  padding: 8px 10px;
  border-radius: 10px;
  background: #0f172a;
  color: #fff;
  font-size: 13px;
  box-shadow: 0 10px 25px rgba(0,0,0,.22);
}
[data-tooltip]::after {
  content: "";
  bottom: calc(100% + 4px);
  transform: translateX(-50%) translateY(6px);
  border: 6px solid transparent;
  border-top-color: #0f172a;
}
[data-tooltip]:hover::before,
[data-tooltip]:hover::after,
[data-tooltip]:focus-visible::before,
[data-tooltip]:focus-visible::after {
  opacity: 1;
  visibility: visible;
  transform: translateX(-50%) translateY(0);
}

```

JavaScript modular con export/import

Usa esta version si tu HTML carga `<script type="module" src="script.js"></script>`. La clase o funcion vive en funciones.js y la instancia se hace en script.js.

funciones.js + script.js

```

// Version modular opcional para actualizar tooltips dinamicamente.
export function setTooltip(selector, text) {
  const element = document.querySelector(selector);
  if (!element) return;
  element.setAttribute('data-tooltip', text);
}

export function removeTooltip(selector) {
  const element = document.querySelector(selector);
  if (!element) return;
  element.removeAttribute('data-tooltip');
}

// script.js
import { setTooltip } from './funciones.js';
setTooltip('#botonGuardar', 'Guarda los cambios del formulario');

```

JavaScript normal sin modulos

Usa esta version si prefieres tener todo en un solo archivo como script-normal.js y cargarlo con `<script src="script-normal.js"></script>`.

script-normal.js

```
function setTooltip(selector, text) {
  const element = document.querySelector(selector);
  if (!element) return;
  element.setAttribute('data-tooltip', text);
}

function removeTooltip(selector) {
  const element = document.querySelector(selector);
  if (!element) return;
  element.removeAttribute('data-tooltip');
}

setTooltip('#botonGuardar', 'Guarda los cambios del formulario');
```

Variables, clases, ids y atributos que debes cambiar

Elemento	Para que sirve	Que debes cambiar
data-tooltip	Texto del tooltip.	Cambia el texto directamente en HTML.
bottom	Ubicacion vertical del tooltip.	Cambia a top/left/right si quieres otra posicion.
max-width	Ancho maximo del tooltip.	Sube a 280px si el texto es largo.
z-index	Prioridad visual.	Aumenta si el tooltip queda detras de una card.

Implementacion paso a paso

1. Agrega data-tooltip al elemento que necesita ayuda contextual.
2. Asegurate de que el elemento pueda recibir foco si es interactivo.
3. Copia el CSS y prueba hover y focus-visible.
4. Si el texto es largo, acortalo o aumenta max-width.
5. Usa setTooltip solo si necesitas cambiar el texto desde JavaScript.

Errores comunes y como solucionarlos

- No se ve el tooltip: puede estar recortado por un contenedor con overflow: hidden.
- Se ve muy ancho: usa max-width o divide el texto.
- No funciona en celular: para mobile conviene mostrar ayuda con clic o modal.

Checklist de prueba antes de reutilizar

- El componente existe en HTML y el id coincide con el usado en JavaScript.
- No hay ids duplicados en la pagina.
- La clase visual de estado se agrega y se quita correctamente.
- No aparecen errores en la consola del navegador.
- La funcionalidad sigue funcionando en pantalla pequena.
- El componente no rompe otros elementos del proyecto.

Mejoras opcionales

- Conectar el componente con ToastManager para mostrar mensajes de exito o error.
- Agregar estados de carga si depende de datos externos.

- Guardar preferencias en localStorage si el usuario puede cambiar configuraciones.
- Mejorar accesibilidad con aria-label, aria-hidden, focus-visible y navegacion por teclado cuando aplique.

30. Badge / indicador numerico

Aspecto	Explicacion
Que hace	Muestra un numero o estado pequeño sobre un boton, icono o texto.
Donde se usa en industria	Notificaciones, carrito de compras, mensajes pendientes, tareas abiertas, alertas de panel administrativo.
Como se personaliza	Cambia el id del badge, el numero, los colores y la condicion para ocultarlo.

Mapa rapido de reciclaje

- 1 Ubica el lugar exacto del HTML donde quieres insertar el componente.
- 2 Copia el HTML minimo y cambia textos visibles, ids y data-attributes.
- 3 Copia el CSS y ajusta colores, separaciones, radios, sombras y responsive si hace falta.
- 4 Escoge JS modular o JS normal, no ambos al mismo tiempo.
- 5 Inicializa el componente con el id correcto y prueba la interaccion principal.

HTML minimo que debes copiar

index.html

```
<button class="notification-button" type="button">
  Notificaciones
  <span class="badge" id="notificationBadge">3</span>
</button>

<button class="btn" id="addNotification">Agregar</button>
<button class="btn" id="clearNotifications">Limpiar</button>
```

CSS del componente

styles.css

```
.notification-button {
  position: relative;
  display: inline-flex;
  align-items: center;
  gap: 8px;
  padding: 10px 16px;
  border-radius: 999px;
}
.badge {
  min-width: 22px;
  height: 22px;
  display: inline-grid;
  place-items: center;
  padding: 0 6px;
  border-radius: 999px;
  background: #ef4444;
  color: #fff;
  font-size: 12px;
  font-weight: 700;
  box-shadow: 0 6px 16px rgba(239,68,68,.35);
}
.badge.is-hidden {
  display: none;
}
```

JavaScript modular con export/import

Usa esta version si tu HTML carga `<script type="module" src="script.js"></script>`. La clase o funcion vive en funciones.js y la instancia se hace en script.js.

funciones.js + script.js

```
export class BadgeCounter {
  constructor(badgeId, initialValue = 0) {
    this.badge = document.getElementById(badgeId);
    this.value = initialValue;
    this.render();
  }

  increment(amount = 1) {
    this.value += amount;
    this.render();
  }

  decrement(amount = 1) {
    this.value = Math.max(0, this.value - amount);
    this.render();
  }

  reset() {
    this.value = 0;
    this.render();
  }

  render() {
    if (!this.badge) return;
    this.badge.textContent = this.value > 99 ? '99+' : this.value;
    this.badge.classList.toggle('is-hidden', this.value === 0);
  }
}

// script.js
import { BadgeCounter } from './funciones.js';
const badge = new BadgeCounter('notificationBadge', 3);
document.getElementById('addNotification').addEventListener('click', () => badge.increment());
document.getElementById('clearNotifications').addEventListener('click', () => badge.reset());
```

JavaScript normal sin modulos

Usa esta version si prefieres tener todo en un solo archivo como script-normal.js y cargarlo con `<script src="script-normal.js"></script>`.

script-normal.js

```
class BadgeCounter {
  constructor(badgeId, initialValue = 0) {
    this.badge = document.getElementById(badgeId);
    this.value = initialValue;
    this.render();
  }
  increment(amount = 1) { this.value += amount; this.render(); }
  decrement(amount = 1) { this.value = Math.max(0, this.value - amount); this.render(); }
  reset() { this.value = 0; this.render(); }
  render() {
    if (!this.badge) return;
    this.badge.textContent = this.value > 99 ? '99+' : this.value;
    this.badge.classList.toggle('is-hidden', this.value === 0);
  }
}

const badge = new BadgeCounter('notificationBadge', 3);
document.getElementById('addNotification').addEventListener('click', () => badge.increment());
document.getElementById('clearNotifications').addEventListener('click', () => badge.reset());
```

Variables, clases, ids y atributos que debes cambiar

Elemento	Para que sirve	Que debes cambiar
notificationBadge	Id del span que muestra el numero.	Cambia si tienes varios badges.
initialValue	Valor inicial.	Usa 0 si debe iniciar oculto.
99+	Limite visual.	Cambia a 9+ o 999+ segun tu interfaz.

Elemento	Para que sirve	Que debes cambiar
.badge.is-hidden	Clase que oculta el indicador.	Cambia display none por opacity si quieres animacion.

Implementacion paso a paso

- 1 Crea un span con clase badge dentro del boton o icono.
- 2 Copia los estilos del badge.
- 3 Inicializa BadgeCounter con el id correcto.
- 4 Llama increment, decrement o reset segun la accion del usuario.
- 5 Prueba que al llegar a 0 se oculte.

Errores comunes y como solucionarlos

- El badge aparece fuera de lugar: pon position: relative en el contenedor.
- No se actualiza: revisa que el id sea correcto.
- Muestra valores negativos: usa Math.max como en el ejemplo.

Checklist de prueba antes de reutilizar

- El componente existe en HTML y el id coincide con el usado en JavaScript.
- No hay ids duplicados en la pagina.
- La clase visual de estado se agrega y se quita correctamente.
- No aparecen errores en la consola del navegador.
- La funcionalidad sigue funcionando en pantalla pequena.
- El componente no rompe otros elementos del proyecto.

Mejoras opcionales

- Conectar el componente con ToastManager para mostrar mensajes de exito o error.
- Agregar estados de carga si depende de datos externos.
- Guardar preferencias en localStorage si el usuario puede cambiar configuraciones.
- Mejorar accesibilidad con aria-label, aria-hidden, focus-visible y navegacion por teclado cuando aplique.

31. Copy clipboard / copiar al portapapeles

Aspecto	Explicacion
Que hace	Permite copiar texto con un clic desde un bloque, input, codigo o cupon.
Donde se usa en industria	Cupones, enlaces de invitacion, codigos, tokens publicos, datos de contacto, comandos de instalacion.
Como se personaliza	Cambia el selector data-copy, el mensaje de confirmacion y el fallback.

Mapa rapido de reciclaje

- 1 Ubica el lugar exacto del HTML donde quieres insertar el componente.
- 2 Copia el HTML minimo y cambia textos visibles, ids y data-attributes.
- 3 Copia el CSS y ajusta colores, separaciones, radios, sombras y responsive si hace falta.
- 4 Escoge JS modular o JS normal, no ambos al mismo tiempo.
- 5 Inicializa el componente con el id correcto y prueba la interaccion principal.

HTML minimo que debes copiar

index.html

```
<p id="codigoCupon">SIERRA-2026</p>
<button class="btn" data-copy="#codigoCupon">Copiar cupon</button>

<input id="correoEmpresa" value="contacto@empresa.com" readonly>
<button class="btn" data-copy="#correoEmpresa">Copiar correo</button>
```

CSS del componente

styles.css

```
[data-copy] {
  cursor: pointer;
}
.copy-feedback {
  display: inline-block;
  margin-left: 8px;
  color: #16a34a;
  font-weight: 700;
}
```

JavaScript modular con export/import

Usa esta version si tu HTML carga `<script type="module" src="script.js"></script>`. La clase o funcion vive en funciones.js y la instancia se hace en script.js.

funciones.js + script.js

```

export class ClipboardManager {
  constructor({ onSuccess = null, onError = null } = {}) {
    this.onSuccess = onSuccess;
    this.onError = onError;
    this.init();
  }

  init() {
    document.querySelectorAll('[data-copy]').forEach((button) => {
      button.addEventListener('click', () => this.copyFromSelector(button.dataset.copy, button));
    });
  }

  async copyFromSelector(selector, trigger) {
    const target = document.querySelector(selector);
    if (!target) return;
    const text = target.value ?? target.textContent;

    try {
      await navigator.clipboard.writeText(text.trim());
      trigger.textContent = 'Copiado';
      setTimeout(() => trigger.textContent = 'Copiar', 1500);
      this.onSuccess?.(text);
    } catch (error) {
      this.fallbackCopy(text);
      this.onError?.(error);
    }
  }

  fallbackCopy(text) {
    const area = document.createElement('textarea');
    area.value = text;
    document.body.appendChild(area);
    area.select();
    document.execCommand('copy');
    area.remove();
  }
}

// script.js
import { ClipboardManager } from './funciones.js';
new ClipboardManager();

```

JavaScript normal sin modulos

Usa esta version si prefieres tener todo en un solo archivo como script-normal.js y cargarlo con `<script src="script-normal.js"></script>`.

script-normal.js

```

class ClipboardManager {
  constructor() { this.init(); }
  init() {
    document.querySelectorAll('[data-copy]').forEach((button) => {
      button.addEventListener('click', () => this.copyFromSelector(button.dataset.copy, button));
    });
  }
  async copyFromSelector(selector, trigger) {
    const target = document.querySelector(selector);
    if (!target) return;
    const text = target.value || target.textContent;
    try {
      await navigator.clipboard.writeText(text.trim());
      trigger.textContent = 'Copiado';
      setTimeout(() => trigger.textContent = 'Copiar', 1500);
    } catch (error) {
      const area = document.createElement('textarea');
      area.value = text;
      document.body.appendChild(area);
      area.select();
      document.execCommand('copy');
      area.remove();
    }
  }
}

new ClipboardManager();

```

Variables, clases, ids y atributos que debes cambiar

Elemento	Para que sirve	Que debes cambiar
data-copy	Selector del elemento que contiene el texto.	Debe apuntar a un id, clase o selector valido.
navigator.clipboard	API moderna de copia.	Funciona mejor en HTTPS o localhost.
trigger.textContent	Texto del boton despues de copiar.	Personaliza Copiado/Copiar segun tu UI.
fallbackCopy	Plan B para navegadores antiguos.	Mantenlo si quieres compatibilidad.

Implementacion paso a paso

- 1 Pon el texto a copiar en un elemento con id.
- 2 Crea un boton con data-copy apuntando a ese selector.
- 3 Copia la clase ClipboardManager y ejecútala.
- 4 Prueba en localhost o servidor seguro.
- 5 Conecta con ToastManager si quieres mostrar una notificacion bonita.

Errores comunes y como solucionarlos

- No copia: el selector data-copy no encuentra el elemento.
- Funciona local pero no en servidor: revisa permisos HTTPS.
- Cambia el texto del boton a Copiado pero no vuelve: revisa setTimeout.

Checklist de prueba antes de reutilizar

- El componente existe en HTML y el id coincide con el usado en JavaScript.
- No hay ids duplicados en la pagina.
- La clase visual de estado se agrega y se quita correctamente.
- No aparecen errores en la consola del navegador.
- La funcionalidad sigue funcionando en pantalla pequena.
- El componente no rompe otros elementos del proyecto.

Mejoras opcionales

- Conectar el componente con ToastManager para mostrar mensajes de exito o error.
- Agregar estados de carga si depende de datos externos.
- Guardar preferencias en localStorage si el usuario puede cambiar configuraciones.
- Mejorar accesibilidad con aria-label, aria-hidden, focus-visible y navegacion por teclado cuando aplique.

32. Cards dinamicas desde un array

Aspecto	Explicacion
Que hace	Renderiza tarjetas automaticamente a partir de datos JavaScript.
Donde se usa en industria	Catalogos, portafolios, productos, cursos, blogs, equipos, servicios y dashboards.
Como se personaliza	Edita el array de datos, el template HTML y el contenedor donde se pintan las cards.

Mapa rapido de reciclaje

- 1 Ubica el lugar exacto del HTML donde quieres insertar el componente.
- 2 Copia el HTML minimo y cambia textos visibles, ids y data-attributes.
- 3 Copia el CSS y ajusta colores, separaciones, radios, sombras y responsive si hace falta.
- 4 Escoge JS modular o JS normal, no ambos al mismo tiempo.
- 5 Inicializa el componente con el id correcto y prueba la interaccion principal.

HTML minimo que debes copiar

index.html

```
<section class="cards-grid" id="cardsGrid"></section>
```

CSS del componente

styles.css

```
.cards-grid {  
  display: grid;  
  grid-template-columns: repeat(auto-fit, minmax(240px, 1fr));  
  gap: 18px;  
}  
.card {  
  padding: 20px;  
  border-radius: 20px;  
  background: rgba(255,255,255,.12);  
  border: 1px solid rgba(255,255,255,.18);  
  box-shadow: 0 14px 35px rgba(0,0,0,.18);  
}  
.card__title { margin: 0 0 8px; font-size: 20px; }  
.card__meta { color: #64748b; font-size: 14px; }  
.card__price { font-weight: 800; color: #2563eb; }
```

JavaScript modular con export/import

Usa esta version si tu HTML carga `<script type="module" src="script.js"></script>`. La clase o funcion vive en funciones.js y la instancia se hace en script.js.

funciones.js + script.js

```

export class CatalogCards {
  constructor(containerId, items = []) {
    this.container = document.getElementById(containerId);
    this.items = items;
    this.render(this.items);
  }

  setItems(items) {
    this.items = items;
    this.render(this.items);
  }

  render(items) {
    if (!this.container) return;
    if (items.length === 0) {
      this.container.innerHTML = '<p>No hay resultados disponibles.</p>';
      return;
    }

    this.container.innerHTML = items.map((item) => `
      <article class="card" data-category="${item.category}">
        <h3 class="card__title">${item.name}</h3>
        <p>${item.description}</p>
        <p class="card__meta">Categoria: ${item.category}</p>
        <p class="card__price">${item.price}</p>
      </article>
    `).join('');
  }
}

// script.js
import { CatalogCards } from './funciones.js';
const products = [
  { name: 'Producto A', category: 'web', price: 12000, description: 'Descripcion corta.' },
  { name: 'Producto B', category: 'data', price: 18000, description: 'Otra descripcion.' }
];
const catalog = new CatalogCards('cardsGrid', products);

```

JavaScript normal sin modulos

Usa esta version si prefieres tener todo en un solo archivo como script-normal.js y cargarlo con `<script src="script-normal.js"></script>`.

script-normal.js

```

class CatalogCards {
  constructor(containerId, items = []) {
    this.container = document.getElementById(containerId);
    this.items = items;
    this.render(this.items);
  }
  setItems(items) {
    this.items = items;
    this.render(this.items);
  }
  render(items) {
    if (!this.container) return;
    if (items.length === 0) {
      this.container.innerHTML = '<p>No hay resultados disponibles.</p>';
      return;
    }
    this.container.innerHTML = items.map((item) => `
      <article class="card" data-category="${item.category}">
        <h3 class="card__title">${item.name}</h3>
        <p>${item.description}</p>
        <p class="card__meta">Categoria: ${item.category}</p>
        <p class="card__price">${item.price}</p>
      </article>
    `).join('');
  }
}
const products = [
  { name: 'Producto A', category: 'web', price: 12000, description: 'Descripcion corta.' },
  { name: 'Producto B', category: 'data', price: 18000, description: 'Otra descripcion.' }
];
const catalog = new CatalogCards('cardsGrid', products);

```

Variables, clases, ids y atributos que debes cambiar

Elemento	Para que sirve	Que debes cambiar
cardsGrid	Contenedor donde se pintan las cards.	Cambia el id si tu seccion tiene otro nombre.
products/items	Array de datos.	Agrega campos como image, rating, stock o url.
render()	Metodo que convierte datos en HTML.	Edita el template para cambiar el diseño.
data-category	Dato util para filtrar.	Debe coincidir con tus filtros chips.

Implementacion paso a paso

- 1 Crea un contenedor vacio para las cards.
- 2 Define un array con objetos consistentes.
- 3 Copia la clase CatalogCards.
- 4 Instancia la clase con el id y el array.
- 5 Edita el template dentro de render para adaptarlo a tu proyecto.

Errores comunes y como solucionarlos

- Se imprime undefined: falta una propiedad en algun objeto del array.
- No aparecen cards: revisa el id del contenedor.
- Hay riesgo de HTML no confiable si los datos vienen de usuarios. Sanitiza si viene de fuentes externas.

Checklist de prueba antes de reutilizar

- El componente existe en HTML y el id coincide con el usado en JavaScript.
- No hay ids duplicados en la pagina.
- La clase visual de estado se agrega y se quita correctamente.
- No aparecen errores en la consola del navegador.
- La funcionalidad sigue funcionando en pantalla pequena.
- El componente no rompe otros elementos del proyecto.

Mejoras opcionales

- Conectar el componente con ToastManager para mostrar mensajes de exito o error.
- Agregar estados de carga si depende de datos externos.
- Guardar preferencias en localStorage si el usuario puede cambiar configuraciones.
- Mejorar accesibilidad con aria-label, aria-hidden, focus-visible y navegacion por teclado cuando aplique.

33. Búsqueda con debounce

Aspecto	Explicacion
Que hace	Evita ejecutar una búsqueda en cada tecla inmediatamente. Espera unos milisegundos despues de que el usuario deja de escribir.
Donde se usa en industria	Filtros de catalogo, buscadores, tablas, llamadas a APIs, dashboards y autocompletados.
Como se personaliza	Cambia el delay, el input, la lista de datos y la funcion que filtra.

Mapa rapido de reciclaje

- 1 Ubica el lugar exacto del HTML donde quieres insertar el componente.
- 2 Copia el HTML minimo y cambia textos visibles, ids y data-attributes.
- 3 Copia el CSS y ajusta colores, separaciones, radios, sombras y responsive si hace falta.
- 4 Escoge JS modular o JS normal, no ambos al mismo tiempo.
- 5 Inicializa el componente con el id correcto y prueba la interaccion principal.

HTML minimo que debes copiar

index.html

```
<input type="search" id="searchInput" placeholder="Buscar productos...">
<section id="cardsGrid" class="cards-grid"></section>
```

CSS del componente

styles.css

```
input[type="search"] {
  width: 100%;
  padding: 12px 14px;
  border-radius: 14px;
  border: 1px solid #cbd5e1;
  margin-bottom: 18px;
}
```

JavaScript modular con export/import

Usa esta version si tu HTML carga `<script type="module" src="script.js"></script>`. La clase o funcion vive en `funciones.js` y la instancia se hace en `script.js`.

funciones.js + script.js


```

export function debounce(callback, delay = 300) {
  let timer;
  return (...args) => {
    clearTimeout(timer);
    timer = setTimeout(() => callback(...args), delay);
  };
}

export class SearchController {
  constructor(inputId, items, onResults, delay = 300) {
    this.input = document.getElementById(inputId);
    this.items = items;
    this.onResults = onResults;
    this.delay = delay;
    this.init();
  }

  init() {
    if (!this.input) return;
    this.input.addEventListener('input', debounce(() => this.search(), this.delay));
  }

  search() {
    const term = this.input.value.trim().toLowerCase();
    const results = this.items.filter((item) => {
      return item.name.toLowerCase().includes(term)
        || item.category.toLowerCase().includes(term)
        || item.description.toLowerCase().includes(term);
    });
    this.onResults(results);
  }
}

// script.js
import { SearchController, CatalogCards } from './funciones.js';
const catalog = new CatalogCards('cardsGrid', products);
new SearchController('searchInput', products, (results) => catalog.render(results), 350);

```

JavaScript normal sin modulos

Usa esta version si prefieres tener todo en un solo archivo como script-normal.js y cargarlo con `<script src="script-normal.js"></script>`.

script-normal.js

```

function debounce(callback, delay = 300) {
  let timer;
  return (...args) => {
    clearTimeout(timer);
    timer = setTimeout(() => callback(...args), delay);
  };
}

class SearchController {
  constructor(inputId, items, onResults, delay = 300) {
    this.input = document.getElementById(inputId);
    this.items = items;
    this.onResults = onResults;
    this.delay = delay;
    this.init();
  }

  init() {
    if (!this.input) return;
    this.input.addEventListener('input', debounce(() => this.search(), this.delay));
  }

  search() {
    const term = this.input.value.trim().toLowerCase();
    const results = this.items.filter((item) =>
      item.name.toLowerCase().includes(term) ||
      item.category.toLowerCase().includes(term) ||
      item.description.toLowerCase().includes(term)
    );
    this.onResults(results);
  }
}

new SearchController('searchInput', products, (results) => catalog.render(results), 350);

```

Variables, clases, ids y atributos que debes cambiar

Elemento	Para que sirve	Que debes cambiar
delay	Tiempo de espera para buscar.	300 a 500 ms suele sentirse bien.
items	Datos originales.	Debe ser el array completo, no el ya filtrado.
onResults	Funcion que recibe resultados.	Puedes renderizar cards, tabla o lista.
term	Texto escrito por el usuario.	Puedes normalizar acentos si lo necesitas.

Implementacion paso a paso

- 1 Crea un input tipo search.
- 2 Ten un array original sin modificar.
- 3 Copia debounce y SearchController.
- 4 Pasa una funcion callback que actualice la UI.
- 5 Prueba con mayusculas, minusculas y texto vacio.

Errores comunes y como solucionarlos

- Filtra sobre resultados ya filtrados y pierde datos: siempre filtra desde el array original.
- Busca muy lento: baja el delay.
- Busca demasiado: sube el delay si hace llamadas a una API.

Checklist de prueba antes de reutilizar

- El componente existe en HTML y el id coincide con el usado en JavaScript.
- No hay ids duplicados en la pagina.
- La clase visual de estado se agrega y se quita correctamente.
- No aparecen errores en la consola del navegador.
- La funcionalidad sigue funcionando en pantalla pequena.
- El componente no rompe otros elementos del proyecto.

Mejoras opcionales

- Conectar el componente con ToastManager para mostrar mensajes de exito o error.
- Agregar estados de carga si depende de datos externos.
- Guardar preferencias en localStorage si el usuario puede cambiar configuraciones.
- Mejorar accesibilidad con aria-label, aria-hidden, focus-visible y navegacion por teclado cuando aplique.

34. Filtros chips por categoria

Aspecto	Explicacion
Que hace	Permite filtrar datos con botones pequeños tipo etiqueta o categoria.
Donde se usa en industria	Catalogos, portafolios, blogs, dashboards, filtros de productos, categorias de servicios.
Como se personaliza	Cambia data-category y asegúrate de que coincida con la categoria del dato.

Mapa rapido de reciclaje

- 1 Ubica el lugar exacto del HTML donde quieres insertar el componente.
- 2 Copia el HTML minimo y cambia textos visibles, ids y data-attributes.
- 3 Copia el CSS y ajusta colores, separaciones, radios, sombras y responsive si hace falta.
- 4 Escoge JS modular o JS normal, no ambos al mismo tiempo.
- 5 Inicializa el componente con el id correcto y prueba la interaccion principal.

HTML minimo que debes copiar

index.html

```
<div class="chips" id="categoryChips">
  <button class="chip is-active" data-category="all">Todos</button>
  <button class="chip" data-category="web">Web</button>
  <button class="chip" data-category="data">Datos</button>
  <button class="chip" data-category="design">Diseno</button>
</div>

<section id="cardsGrid" class="cards-grid"></section>
```

CSS del componente

styles.css

```
.chips {
  display: flex;
  flex-wrap: wrap;
  gap: 10px;
  margin-bottom: 18px;
}
.chip {
  border: 1px solid #cbd5e1;
  border-radius: 999px;
  padding: 9px 14px;
  background: #fff;
  cursor: pointer;
  transition: background .2s ease, transform .2s ease;
}
.chip:hover { transform: translateY(-1px); }
.chip.is-active {
  background: linear-gradient(135deg, #2563eb, #7c3aed);
  color: #fff;
  border-color: transparent;
}
```

JavaScript modular con export/import

Usa esta version si tu HTML carga `<script type="module" src="script.js"></script>`. La clase o funcion vive en funciones.js y la instancia se hace en script.js.

funciones.js + script.js

```

export class ChipFilters {
  constructor(containerId, items, onResults) {
    this.container = document.getElementById(containerId);
    this.items = items;
    this.onResults = onResults;
    this.activeCategory = 'all';
    this.init();
  }

  init() {
    if (!this.container) return;
    this.container.addEventListener('click', (event) => {
      const chip = event.target.closest('[data-category]');
      if (!chip) return;
      this.activeCategory = chip.dataset.category;
      this.container.querySelectorAll('.chip').forEach((item) => item.classList.remove('is-
active'));
      chip.classList.add('is-active');
      this.apply();
    });
  }

  apply() {
    const results = this.activeCategory === 'all'
      ? this.items
      : this.items.filter((item) => item.category === this.activeCategory);
    this.onResults(results);
  }
}

// script.js
import { ChipFilters } from './funciones.js';
new ChipFilters('categoryChips', products, (results) => catalog.render(results));

```

JavaScript normal sin modulos

Usa esta version si prefieres tener todo en un solo archivo como script-normal.js y cargarlo con `<script src="script-normal.js"></script>`.

script-normal.js

```

class ChipFilters {
  constructor(containerId, items, onResults) {
    this.container = document.getElementById(containerId);
    this.items = items;
    this.onResults = onResults;
    this.activeCategory = 'all';
    this.init();
  }

  init() {
    if (!this.container) return;
    this.container.addEventListener('click', (event) => {
      const chip = event.target.closest('[data-category]');
      if (!chip) return;
      this.activeCategory = chip.dataset.category;
      this.container.querySelectorAll('.chip').forEach((item) => item.classList.remove('is-
active'));
      chip.classList.add('is-active');
      this.apply();
    });
  }

  apply() {
    const results = this.activeCategory === 'all' ? this.items : this.items.filter((item) => item.
category === this.activeCategory);
    this.onResults(results);
  }
}

new ChipFilters('categoryChips', products, (results) => catalog.render(results));

```

Variables, clases, ids y atributos que debes cambiar

Elemento	Para que sirve	Que debes cambiar
data-category	Categoría que representa cada chip.	Debe coincidir con item.category.
all	Categoría especial para mostrar todos.	Puedes cambiarla por todos, pero actualiza JS.

Elemento	Para que sirve	Que debes cambiar
is-active	Clase visual del filtro activo.	Cambia colores en CSS.
onResults	Funcion que actualiza la vista.	Renderiza cards, tabla o lista.

Implementacion paso a paso

- 1 Crea un contenedor de chips.
- 2 Agrega un boton por categoria.
- 3 Asegúrate de que las categorias existan en los datos.
- 4 Inicializa ChipFilters con el contenedor, datos y callback.
- 5 Prueba cada chip y revisa que el activo cambie visualmente.

Errores comunes y como solucionarlos

- Un chip no filtra: data-category no coincide exactamente con item.category.
- No se ve el activo: falta la clase CSS .chip.is-active.
- Al mezclar con busqueda, necesitas un estado global que combine filtros y texto.

Checklist de prueba antes de reutilizar

- El componente existe en HTML y el id coincide con el usado en JavaScript.
- No hay ids duplicados en la pagina.
- La clase visual de estado se agrega y se quita correctamente.
- No aparecen errores en la consola del navegador.
- La funcionalidad sigue funcionando en pantalla pequena.
- El componente no rompe otros elementos del proyecto.

Mejoras opcionales

- Conectar el componente con ToastManager para mostrar mensajes de exito o error.
- Agregar estados de carga si depende de datos externos.
- Guardar preferencias en localStorage si el usuario puede cambiar configuraciones.
- Mejorar accesibilidad con aria-label, aria-hidden, focus-visible y navegacion por teclado cuando aplique.

35. Ordenamiento de resultados

Aspecto	Explicacion
Que hace	Ordena listas por precio, nombre, calificacion, fecha u otro criterio.
Donde se usa en industria	E-commerce, catalogos, dashboards, tablas, listados de usuarios y reportes.
Como se personaliza	Cambia las opciones del select y la funcion de comparacion.

Mapa rapido de reciclaje

- 1 Ubica el lugar exacto del HTML donde quieres insertar el componente.
- 2 Copia el HTML minimo y cambia textos visibles, ids y data-attributes.
- 3 Copia el CSS y ajusta colores, separaciones, radios, sombras y responsive si hace falta.
- 4 Escoge JS modular o JS normal, no ambos al mismo tiempo.
- 5 Inicializa el componente con el id correcto y prueba la interaccion principal.

HTML minimo que debes copiar

index.html

```
<label for="sortSelect">Ordenar por:</label>
<select id="sortSelect">
  <option value="name-asc">Nombre A-Z</option>
  <option value="name-desc">Nombre Z-A</option>
  <option value="price-asc">Menor precio</option>
  <option value="price-desc">Mayor precio</option>
</select>
```

CSS del componente

styles.css

```
#sortSelect {
  width: 100%;
  max-width: 280px;
  padding: 10px 12px;
  border-radius: 12px;
  border: 1px solid #cbd5e1;
  background: #fff;
}
```

JavaScript modular con export/import

Usa esta version si tu HTML carga `<script type="module" src="script.js"></script>`. La clase o funcion vive en funciones.js y la instancia se hace en script.js.

funciones.js + script.js

```

export class SortController {
  constructor(selectId, items, onResults) {
    this.select = document.getElementById(selectId);
    this.items = items;
    this.onResults = onResults;
    this.init();
  }

  init() {
    if (!this.select) return;
    this.select.addEventListener('change', () => this.sort());
  }

  sort() {
    const value = this.select.value;
    const sorted = [...this.items];

    const strategies = {
      'name-asc': (a, b) => a.name.localeCompare(b.name),
      'name-desc': (a, b) => b.name.localeCompare(a.name),
      'price-asc': (a, b) => a.price - b.price,
      'price-desc': (a, b) => b.price - a.price,
    };

    sorted.sort(strategies[value] || strategies['name-asc']);
    this.onResults(sorted);
  }
}

// script.js
import { SortController } from './funciones.js';
new SortController('sortSelect', products, (results) => catalog.render(results));

```

JavaScript normal sin modulos

Usa esta version si prefieres tener todo en un solo archivo como script-normal.js y cargarlo con `<script src="script-normal.js"></script>`.

script-normal.js

```

class SortController {
  constructor(selectId, items, onResults) {
    this.select = document.getElementById(selectId);
    this.items = items;
    this.onResults = onResults;
    this.init();
  }

  init() {
    if (!this.select) return;
    this.select.addEventListener('change', () => this.sort());
  }

  sort() {
    const value = this.select.value;
    const sorted = [...this.items];
    const strategies = {
      'name-asc': (a, b) => a.name.localeCompare(b.name),
      'name-desc': (a, b) => b.name.localeCompare(a.name),
      'price-asc': (a, b) => a.price - b.price,
      'price-desc': (a, b) => b.price - a.price,
    };
    sorted.sort(strategies[value] || strategies['name-asc']);
    this.onResults(sorted);
  }
}

new SortController('sortSelect', products, (results) => catalog.render(results));

```

Variables, clases, ids y atributos que debes cambiar

Elemento	Para que sirve	Que debes cambiar
sortSelect	Id del select de ordenamiento.	Cambia si tienes otro id.
strategies	Objeto con funciones de orden.	Agrega mas criterios: rating, date, stock.
[...this.items]	Copia del array original.	Evita modificar directamente los datos base.
localeCompare	Orden alfabético.	Muy util para nombres y textos.

Implementacion paso a paso

- 1 Crea un select con opciones de orden.
- 2 Cada value debe existir como clave dentro de strategies.
- 3 Copia SortController.
- 4 Pasa el array y un callback de render.
- 5 Prueba orden ascendente y descendente.

Errores comunes y como solucionarlos

- El orden no cambia: el value del option no coincide con strategies.
- Se dañan los datos originales: no uses sort directamente sobre items si luego necesitas el orden original.
- Ordena numeros como texto: usa resta $a.price - b.price$ para valores numericos.

Checklist de prueba antes de reutilizar

- El componente existe en HTML y el id coincide con el usado en JavaScript.
- No hay ids duplicados en la pagina.
- La clase visual de estado se agrega y se quita correctamente.
- No aparecen errores en la consola del navegador.
- La funcionalidad sigue funcionando en pantalla pequena.
- El componente no rompe otros elementos del proyecto.

Mejoras opcionales

- Conectar el componente con ToastManager para mostrar mensajes de exito o error.
- Agregar estados de carga si depende de datos externos.
- Guardar preferencias en localStorage si el usuario puede cambiar configuraciones.
- Mejorar accesibilidad con aria-label, aria-hidden, focus-visible y navegacion por teclado cuando aplique.

36. Paginacion

Aspecto	Explicacion
Que hace	Divide muchos resultados en varias paginas para evitar interfaces pesadas.
Donde se usa en industria	Catalogos, tablas administrativas, blogs, busquedas, resultados de API y listados grandes.
Como se personaliza	Cambia perPage, contenedor de botones y render de resultados.

Mapa rapido de reciclaje

- 1 Ubica el lugar exacto del HTML donde quieres insertar el componente.
- 2 Copia el HTML minimo y cambia textos visibles, ids y data-attributes.
- 3 Copia el CSS y ajusta colores, separaciones, radios, sombras y responsive si hace falta.
- 4 Escoge JS modular o JS normal, no ambos al mismo tiempo.
- 5 Inicializa el componente con el id correcto y prueba la interaccion principal.

HTML minimo que debes copiar

index.html

```
<section id="cardsGrid" class="cards-grid"></section>
<nav class="pagination" id="pagination" aria-label="Paginacion"></nav>
```

CSS del componente

styles.css

```
.pagination {
  display: flex;
  justify-content: center;
  flex-wrap: wrap;
  gap: 8px;
  margin-top: 20px;
}
.pagination__button {
  border: 1px solid #cbd5e1;
  background: #fff;
  border-radius: 10px;
  padding: 8px 12px;
  cursor: pointer;
}
.pagination__button.is-active {
  color: #fff;
  background: #2563eb;
  border-color: #2563eb;
}
```

JavaScript modular con export/import

Usa esta version si tu HTML carga `<script type="module" src="script.js"></script>`. La clase o funcion vive en funciones.js y la instancia se hace en script.js.

funciones.js + script.js

```

export class Pagination {
  constructor(containerId, items, perPage, onPageChange) {
    this.container = document.getElementById(containerId);
    this.items = items;
    this.perPage = perPage;
    this.onPageChange = onPageChange;
    this.currentPage = 1;
    this.render();
    this.emit();
  }

  setItems(items) {
    this.items = items;
    this.currentPage = 1;
    this.render();
    this.emit();
  }

  get totalPages() {
    return Math.ceil(this.items.length / this.perPage) || 1;
  }

  getPageItems() {
    const start = (this.currentPage - 1) * this.perPage;
    return this.items.slice(start, start + this.perPage);
  }

  render() {
    if (!this.container) return;
    this.container.innerHTML = Array.from({ length: this.totalPages }, (_, i) => `
      <button class="pagination__button ${i + 1 === this.currentPage ? 'is-active' : ''}" data-
page="${i + 1}">
        ${i + 1}
      </button>
    `).join('');
    this.container.querySelectorAll('[data-page]').forEach((button) => {
      button.addEventListener('click', () => {
        this.currentPage = Number(button.dataset.page);
        this.render();
        this.emit();
      });
    });
  }

  emit() {
    this.onPageChange(this.getPageItems(), this.currentPage, this.totalPages);
  }
}

// script.js
import { Pagination } from './funciones.js';
const pager = new Pagination('pagination', products, 6, (pageItems) => catalog.render(pageItems));

```

JavaScript normal sin modulos

Usa esta version si prefieres tener todo en un solo archivo como script-normal.js y cargarlo con `<script src="script-normal.js"></script>`.

script-normal.js

```

class Pagination {
  constructor(containerId, items, perPage, onPageChange) {
    this.container = document.getElementById(containerId);
    this.items = items;
    this.perPage = perPage;
    this.onPageChange = onPageChange;
    this.currentPage = 1;
    this.render();
    this.emit();
  }
  setItems(items) { this.items = items; this.currentPage = 1; this.render(); this.emit(); }
  get totalPages() { return Math.ceil(this.items.length / this.perPage) || 1; }
  getPageItems() {
    const start = (this.currentPage - 1) * this.perPage;
    return this.items.slice(start, start + this.perPage);
  }
  render() {
    if (!this.container) return;
    this.container.innerHTML = Array.from({ length: this.totalPages }, (_, i) => `<button
class="pagination__button ${i + 1} === this.currentPage ? 'is-active' : ''" data-page="${i + 1}">${i +
1}</button>`).join('');
    this.container.querySelectorAll('[data-page]').forEach((button) => {
      button.addEventListener('click', () => {
        this.currentPage = Number(button.dataset.page);
        this.render();
        this.emit();
      });
    });
  }
  emit() { this.onPageChange(this.getPageItems(), this.currentPage, this.totalPages); }
}
const pager = new Pagination('pagination', products, 6, (pageItems) => catalog.render(pageItems));

```

Variables, clases, ids y atributos que debes cambiar

Elemento	Para que sirve	Que debes cambiar
pagination	Contenedor de botones de pagina.	Cambia id segun tu HTML.
perPage	Cantidad de elementos por pagina.	6, 9 o 12 son comunes para cards.
setItems()	Actualiza resultados paginados.	Usalo despues de buscar o filtrar.
slice	Extrae solo los elementos visibles.	No modifica el array original.

Implementacion paso a paso

- 1 Crea el contenedor de resultados y el nav de paginacion.
- 2 Copia la clase Pagination.
- 3 Pasa el array completo, cantidad por pagina y callback.
- 4 Cuando filtres o busques, llama pager.setItems(resultados).
- 5 Prueba ultima pagina y filtros con pocos resultados.

Errores comunes y como solucionarlos

- Se muestran resultados de otra pagina tras filtrar: debes reiniciar currentPage a 1.
- No aparecen botones: totalPages es 0 si el array no llega correctamente.
- Demasiados botones: para listas enormes conviene paginacion con puntos suspensivos.

Checklist de prueba antes de reutilizar

- El componente existe en HTML y el id coincide con el usado en JavaScript.
- No hay ids duplicados en la pagina.

- La clase visual de estado se agrega y se quita correctamente.
- No aparecen errores en la consola del navegador.
- La funcionalidad sigue funcionando en pantalla pequena.
- El componente no rompe otros elementos del proyecto.

Mejoras opcionales

- Conectar el componente con ToastManager para mostrar mensajes de exito o error.
- Agregar estados de carga si depende de datos externos.
- Guardar preferencias en localStorage si el usuario puede cambiar configuraciones.
- Mejorar accesibilidad con aria-label, aria-hidden, focus-visible y navegacion por teclado cuando aplique.

37. Tabla ordenable por columnas

Aspecto	Explicacion
Que hace	Permite ordenar filas al hacer clic en los encabezados de una tabla.
Donde se usa en industria	Paneles administrativos, reportes, inventarios, usuarios, facturas, indicadores y listados tecnicos.
Como se personaliza	Agrega data-sort-table a los th y define si la columna es texto o numero.

Mapa rapido de reciclaje

- 1 Ubica el lugar exacto del HTML donde quieres insertar el componente.
- 2 Copia el HTML minimo y cambia textos visibles, ids y data-attributes.
- 3 Copia el CSS y ajusta colores, separaciones, radios, sombras y responsive si hace falta.
- 4 Escoge JS modular o JS normal, no ambos al mismo tiempo.
- 5 Inicializa el componente con el id correcto y prueba la interaccion principal.

HTML minimo que debes copiar

index.html

```
<table class="data-table" id="ordersTable">
  <thead>
    <tr>
      <th data-sort-table="text">Cliente</th>
      <th data-sort-table="number">Total</th>
      <th data-sort-table="text">Estado</th>
    </tr>
  </thead>
  <tbody>
    <tr><td>Carlos</td><td>120000</td><td>Pagado</td></tr>
    <tr><td>Laura</td><td>80000</td><td>Pendiente</td></tr>
  </tbody>
</table>
```

CSS del componente

styles.css

```
.data-table {
  width: 100%;
  border-collapse: collapse;
  overflow: hidden;
  border-radius: 16px;
}
.data-table th,
.data-table td {
  padding: 12px;
  border-bottom: 1px solid #e2e8f0;
  text-align: left;
}
.data-table th {
  background: #0f172a;
  color: #fff;
  cursor: pointer;
  user-select: none;
}
.data-table tr:hover td { background: #f8fafc; }
```

JavaScript modular con export/import

Usa esta version si tu HTML carga `<script type="module" src="script.js"></script>`. La clase o funcion vive en `funciones.js` y la instancia se hace en `script.js`.

funciones.js + script.js

```
export class SortableTable {
  constructor(tableId) {
    this.table = document.getElementById(tableId);
    this.direction = 1;
    this.lastColumn = null;
    this.init();
  }

  init() {
    if (!this.table) return;
    this.table.querySelectorAll('[data-sort-table]').forEach((th, index) => {
      th.addEventListener('click', () => this.sortByColumn(index, th.dataset.sortTable));
    });
  }

  sortByColumn(index, type) {
    const tbody = this.table.querySelector('tbody');
    const rows = [...tbody.querySelectorAll('tr')];

    if (this.lastColumn === index) this.direction *= -1;
    else this.direction = 1;
    this.lastColumn = index;

    rows.sort((a, b) => {
      const valueA = a.children[index].textContent.trim();
      const valueB = b.children[index].textContent.trim();
      if (type === 'number') return (Number(valueA) - Number(valueB)) * this.direction;
      return valueA.localeCompare(valueB) * this.direction;
    });

    tbody.append(...rows);
  }
}

// script.js
import { SortableTable } from './funciones.js';
new SortableTable('ordersTable');
```

JavaScript normal sin modulos

Usa esta version si prefieres tener todo en un solo archivo como `script-normal.js` y cargarlo con `<script src="script-normal.js"></script>`.

script-normal.js

```
class SortableTable {
  constructor(tableId) {
    this.table = document.getElementById(tableId);
    this.direction = 1;
    this.lastColumn = null;
    this.init();
  }

  init() {
    if (!this.table) return;
    this.table.querySelectorAll('[data-sort-table]').forEach((th, index) => {
      th.addEventListener('click', () => this.sortByColumn(index, th.dataset.sortTable));
    });
  }

  sortByColumn(index, type) {
    const tbody = this.table.querySelector('tbody');
    const rows = [...tbody.querySelectorAll('tr')];
    if (this.lastColumn === index) this.direction *= -1;
    else this.direction = 1;
    this.lastColumn = index;
    rows.sort((a, b) => {
      const valueA = a.children[index].textContent.trim();
      const valueB = b.children[index].textContent.trim();
      if (type === 'number') return (Number(valueA) - Number(valueB)) * this.direction;
      return valueA.localeCompare(valueB) * this.direction;
    });
    tbody.append(...rows);
  }
}

new SortableTable('ordersTable');
```

Variables, clases, ids y atributos que debes cambiar

Elemento	Para que sirve	Que debes cambiar
ordersTable	Id de la tabla.	Cambia por el id real de tu tabla.
data-sort-table	Tipo de ordenamiento.	Usa text o number.
direction	Direccion asc/desc.	Se invierte al hacer clic dos veces en la misma columna.
children[index]	Celda correspondiente a la columna.	No debe haber celdas combinadas complicadas.

Implementacion paso a paso

- 1 Crea una tabla con thead y tbody.
- 2 Agrega data-sort-table a los th que quieras ordenar.
- 3 Copia la clase SortableTable.
- 4 Instancia con el id de la tabla.
- 5 Prueba orden en columnas de texto y numero.

Errores comunes y como solucionarlos

- Ordena mal numeros con simbolos: limpia valores antes de Number.
- No funciona con celdas colspan: evita estructuras complejas para tablas ordenables simples.
- No se nota que es clickeable: agrega cursor pointer y una flecha visual.

Checklist de prueba antes de reutilizar

- El componente existe en HTML y el id coincide con el usado en JavaScript.
- No hay ids duplicados en la pagina.
- La clase visual de estado se agrega y se quita correctamente.
- No aparecen errores en la consola del navegador.
- La funcionalidad sigue funcionando en pantalla pequena.
- El componente no rompe otros elementos del proyecto.

Mejoras opcionales

- Conectar el componente con ToastManager para mostrar mensajes de exito o error.
- Agregar estados de carga si depende de datos externos.
- Guardar preferencias en localStorage si el usuario puede cambiar configuraciones.
- Mejorar accesibilidad con aria-label, aria-hidden, focus-visible y navegacion por teclado cuando aplique.

38. Filtro de tabla por texto

Aspecto	Explicacion
Que hace	Filtra filas de una tabla segun lo que el usuario escribe.
Donde se usa en industria	Buscar usuarios, productos, facturas, pedidos, inventarios, registros o historiales.
Como se personaliza	Cambia el id del input, el id de la tabla y el criterio de busqueda.

Mapa rapido de reciclaje

- 1 Ubica el lugar exacto del HTML donde quieres insertar el componente.
- 2 Copia el HTML minimo y cambia textos visibles, ids y data-attributes.
- 3 Copia el CSS y ajusta colores, separaciones, radios, sombras y responsive si hace falta.
- 4 Escoge JS modular o JS normal, no ambos al mismo tiempo.
- 5 Inicializa el componente con el id correcto y prueba la interaccion principal.

HTML minimo que debes copiar

index.html

```
<input type="search" id="tableFilter" placeholder="Filtrar tabla...">

<table class="data-table" id="ordersTable">
  <tbody>
    <tr><td>Carlos</td><td>120000</td><td>Pagado</td></tr>
    <tr><td>Laura</td><td>80000</td><td>Pendiente</td></tr>
  </tbody>
</table>
```

CSS del componente

styles.css

```
#tableFilter {
  width: 100%;
  padding: 12px 14px;
  border: 1px solid #cbd5e1;
  border-radius: 12px;
  margin-bottom: 16px;
}
tr.is-hidden {
  display: none;
}
```

JavaScript modular con export/import

Usa esta version si tu HTML carga `<script type="module" src="script.js"></script>`. La clase o funcion vive en funciones.js y la instancia se hace en script.js.

funciones.js + script.js


```
export class TableFilter {
  constructor(inputId, tableId) {
    this.input = document.getElementById(inputId);
    this.table = document.getElementById(tableId);
    this.init();
  }

  init() {
    if (!this.input || !this.table) return;
    this.input.addEventListener('input', () => this.filter());
  }

  filter() {
    const term = this.input.value.trim().toLowerCase();
    const rows = this.table.querySelectorAll('tbody tr');

    rows.forEach((row) => {
      const text = row.textContent.toLowerCase();
      row.classList.toggle('is-hidden', !text.includes(term));
    });
  }
}

// script.js
import { TableFilter } from './funciones.js';
new TableFilter('tableFilter', 'ordersTable');
```

JavaScript normal sin modulos

Usa esta version si prefieres tener todo en un solo archivo como script-normal.js y cargarlo con `<script src="script-normal.js"></script>`.

script-normal.js

```
class TableFilter {
  constructor(inputId, tableId) {
    this.input = document.getElementById(inputId);
    this.table = document.getElementById(tableId);
    this.init();
  }

  init() {
    if (!this.input || !this.table) return;
    this.input.addEventListener('input', () => this.filter());
  }

  filter() {
    const term = this.input.value.trim().toLowerCase();
    const rows = this.table.querySelectorAll('tbody tr');
    rows.forEach((row) => {
      const text = row.textContent.toLowerCase();
      row.classList.toggle('is-hidden', !text.includes(term));
    });
  }
}

new TableFilter('tableFilter', 'ordersTable');
```

Variables, clases, ids y atributos que debes cambiar

Elemento	Para que sirve	Que debes cambiar
tableFilter	Id del input de busqueda.	Cambia segun tu HTML.
ordersTable	Id de la tabla.	Debe existir en el documento.
row.textContent	Texto completo de cada fila.	Puedes limitar a columnas especificas si quieres.
.is-hidden	Clase para ocultar filas.	Puedes usar visibility si quieres conservar espacio, pero no es comun.

Implementacion paso a paso

- 1 Crea un input search arriba de la tabla.
- 2 Asegura que la tabla tenga tbody.

- 3 Copia TableFilter.
- 4 Instancia con id del input y tabla.
- 5 Prueba con texto de distintas columnas.

Errores comunes y como solucionarlos

- No filtra: la tabla no tiene tbody o el id no coincide.
- Filtra muy lento en tablas enormes: combina con debounce.
- No encuentra acentos: normaliza texto con normalize si lo necesitas.

Checklist de prueba antes de reutilizar

- El componente existe en HTML y el id coincide con el usado en JavaScript.
- No hay ids duplicados en la pagina.
- La clase visual de estado se agrega y se quita correctamente.
- No aparecen errores en la consola del navegador.
- La funcionalidad sigue funcionando en pantalla pequena.
- El componente no rompe otros elementos del proyecto.

Mejoras opcionales

- Conectar el componente con ToastManager para mostrar mensajes de exito o error.
- Agregar estados de carga si depende de datos externos.
- Guardar preferencias en localStorage si el usuario puede cambiar configuraciones.
- Mejorar accesibilidad con aria-label, aria-hidden, focus-visible y navegacion por teclado cuando aplique.

39. Notas con localStorage

Aspecto	Explicacion
Que hace	Guarda notas simples en el navegador sin base de datos.
Donde se usa en industria	Borradores, notas rapidas, preferencias, tareas simples, formularios temporales y prototipos.
Como se personaliza	Cambia la clave localStorage, campos de nota y forma de renderizar.

Mapa rapido de reciclaje

- 1 Ubica el lugar exacto del HTML donde quieres insertar el componente.
- 2 Copia el HTML minimo y cambia textos visibles, ids y data-attributes.
- 3 Copia el CSS y ajusta colores, separaciones, radios, sombras y responsive si hace falta.
- 4 Escoge JS modular o JS normal, no ambos al mismo tiempo.
- 5 Inicializa el componente con el id correcto y prueba la interaccion principal.

HTML minimo que debes copiar

index.html

```
<form id="noteForm" class="form">
  <input id="noteTitle" placeholder="Titulo de la nota" required>
  <textarea id="noteText" placeholder="Escribe una nota" required></textarea>
  <button class="btn" type="submit">Guardar nota</button>
</form>

<section id="notesList" class="notes-list"></section>
```

CSS del componente

styles.css

```
.notes-list {
  display: grid;
  gap: 12px;
  margin-top: 18px;
}
.note {
  padding: 16px;
  border-radius: 16px;
  background: #fff;
  border: 1px solid #e2e8f0;
}
.note_actions {
  display: flex;
  justify-content: flex-end;
  gap: 8px;
}
```

JavaScript modular con export/import

Usa esta version si tu HTML carga `<script type="module" src="script.js"></script>`. La clase o funcion vive en funciones.js y la instancia se hace en script.js.

funciones.js + script.js

```

export function readJSON(key, fallback = []) {
  try {
    return JSON.parse(localStorage.getItem(key)) ?? fallback;
  } catch {
    return fallback;
  }
}

export function saveJSON(key, value) {
  localStorage.setItem(key, JSON.stringify(value));
}

export class NotesApp {
  constructor({ formId, titleId, textId, listId, storageKey = 'kit-notes' }) {
    this.form = document.getElementById(formId);
    this.title = document.getElementById(titleId);
    this.text = document.getElementById(textId);
    this.list = document.getElementById(listId);
    this.storageKey = storageKey;
    this.notes = readJSON(this.storageKey, []);
    this.init();
  }

  init() {
    this.render();
    this.form?.addEventListener('submit', (event) => {
      event.preventDefault();
      this.addNote();
    });
    this.list?.addEventListener('click', (event) => {
      const deleteButton = event.target.closest('[data-delete-note]');
      if (deleteButton) this.deleteNote(deleteButton.dataset.deleteNote);
    });
  }

  addNote() {
    const note = {
      id: crypto.randomUUID(),
      title: this.title.value.trim(),
      text: this.text.value.trim(),
      createdAt: new Date().toISOString()
    };
    this.notes.unshift(note);
    saveJSON(this.storageKey, this.notes);
    this.form.reset();
    this.render();
  }

  deleteNote(id) {
    this.notes = this.notes.filter((note) => note.id !== id);
    saveJSON(this.storageKey, this.notes);
    this.render();
  }

  render() {
    if (!this.list) return;
    this.list.innerHTML = this.notes.map((note) => `
      <article class="note">
        <h3>${note.title}</h3>
        <p>${note.text}</p>
        <div class="note__actions">
          <button type="button" data-delete-note="${note.id}">Eliminar</button>
        </div>
      </article>
    `).join('');
  }
}

// script.js
import { NotesApp } from './funciones.js';
new NotesApp({ formId: 'noteForm', titleId: 'noteTitle', textId: 'noteText', listId: 'notesList' });

```

JavaScript normal sin modulos

Usa esta version si prefieres tener todo en un solo archivo como script-normal.js y cargarlo con `<script src="script-normal.js"></script>`.

script-normal.js

```

function readJSON(key, fallback = []) {
  try { return JSON.parse(localStorage.getItem(key)) || fallback; }
  catch { return fallback; }
}
function saveJSON(key, value) {
  localStorage.setItem(key, JSON.stringify(value));
}
class NotesApp {
  constructor({ formId, titleId, textId, listId, storageKey = 'kit-notes' }) {
    this.form = document.getElementById(formId);
    this.title = document.getElementById(titleId);
    this.text = document.getElementById(textId);
    this.list = document.getElementById(listId);
    this.storageKey = storageKey;
    this.notes = readJSON(this.storageKey, []);
    this.init();
  }
  init() {
    this.render();
    this.form.addEventListener('submit', (event) => { event.preventDefault(); this.addNote(); });
    this.list.addEventListener('click', (event) => {
      const deleteButton = event.target.closest('[data-delete-note]');
      if (deleteButton) this.deleteNote(deleteButton.dataset.deleteNote);
    });
  }
  addNote() {
    const note = { id: Date.now().toString(), title: this.title.value.trim(), text: this.text.value.trim(), createdAt: new Date().toISOString() };
    this.notes.unshift(note);
    saveJSON(this.storageKey, this.notes);
    this.form.reset();
    this.render();
  }
  deleteNote(id) {
    this.notes = this.notes.filter((note) => note.id !== id);
    saveJSON(this.storageKey, this.notes);
    this.render();
  }
  render() {
    this.list.innerHTML = this.notes.map((note) => `<article class="note"><h3>${note.title}</h3><p>${note.text}</p><button data-delete-note="${note.id}">Eliminar</button></article>`).join('');
  }
}
new NotesApp({ formId: 'noteForm', titleId: 'noteTitle', textId: 'noteText', listId: 'notesList' });

```

Variables, clases, ids y atributos que debes cambiar

Elemento	Para que sirve	Que debes cambiar
kit-notes	Clave de localStorage.	Cambiala para no mezclar proyectos.
noteTitle/noteText	Campos del formulario.	Puedes agregar categoria, prioridad o fecha.
crypto.randomUUID()	Genera id unico.	Usa Date.now si quieres compatibilidad simple.
readJSON/saveJSON	Helpers reutilizables.	Usalos para cualquier dato guardado.

Implementacion paso a paso

- 1 Crea formulario y contenedor de notas.
- 2 Copia helpers readJSON y saveJSON.
- 3 Copia NotesApp.
- 4 Inicializa con los ids correctos.
- 5 Crea, recarga la pagina y confirma que la nota sigue guardada.

Errores comunes y como solucionarlos

- No guarda: revisa que localStorage no este bloqueado.
- Se duplican notas: no inicialices NotesApp dos veces.

- Se borra todo al cambiar de proyecto: usa storageKey diferente por app.

Checklist de prueba antes de reutilizar

- El componente existe en HTML y el id coincide con el usado en JavaScript.
- No hay ids duplicados en la pagina.
- La clase visual de estado se agrega y se quita correctamente.
- No aparecen errores en la consola del navegador.
- La funcionalidad sigue funcionando en pantalla pequena.
- El componente no rompe otros elementos del proyecto.

Mejoras opcionales

- Conectar el componente con ToastManager para mostrar mensajes de exito o error.
- Agregar estados de carga si depende de datos externos.
- Guardar preferencias en localStorage si el usuario puede cambiar configuraciones.
- Mejorar accesibilidad con aria-label, aria-hidden, focus-visible y navegacion por teclado cuando aplique.

40. Exportar JSON

Aspecto	Explicacion
Que hace	Descarga datos del navegador o de la app como archivo .json.
Donde se usa en industria	Exportar reportes, backups, notas, configuraciones, carritos, filtros o datos de prototipo.
Como se personaliza	Cambia el origen de datos, nombre del archivo y estructura del JSON.

Mapa rapido de reciclaje

- 1 Ubica el lugar exacto del HTML donde quieres insertar el componente.
- 2 Copia el HTML minimo y cambia textos visibles, ids y data-attributes.
- 3 Copia el CSS y ajusta colores, separaciones, radios, sombras y responsive si hace falta.
- 4 Escoge JS modular o JS normal, no ambos al mismo tiempo.
- 5 Inicializa el componente con el id correcto y prueba la interaccion principal.

HTML minimo que debes copiar

index.html

```
<button class="btn" id="exportNotes">Exportar notas JSON</button>
```

CSS del componente

styles.css

```
.btn--export {  
  background: linear-gradient(135deg, #0f766e, #2563eb);  
  color: #fff;  
}
```

JavaScript modular con export/import

Usa esta version si tu HTML carga `<script type="module" src="script.js"></script>`. La clase o funcion vive en funciones.js y la instancia se hace en script.js.

funciones.js + script.js

```
export class JSONExporter {
  static download(data, filename = 'datos.json') {
    const json = JSON.stringify(data, null, 2);
    const blob = new Blob([json], { type: 'application/json' });
    const url = URL.createObjectURL(blob);

    const link = document.createElement('a');
    link.href = url;
    link.download = filename;
    document.body.appendChild(link);
    link.click();
    link.remove();

    URL.revokeObjectURL(url);
  }
}

// script.js
import { JSONExporter, readJSON } from './funciones.js';
document.getElementById('exportNotes').addEventListener('click', () => {
  const notes = readJSON('kit-notes', []);
  JSONExporter.download(notes, 'mis-notas.json');
});
```

JavaScript normal sin modulos

Usa esta version si prefieres tener todo en un solo archivo como script-normal.js y cargarlo con `<script src="script-normal.js"></script>`.

script-normal.js

```
class JSONExporter {
  static download(data, filename = 'datos.json') {
    const json = JSON.stringify(data, null, 2);
    const blob = new Blob([json], { type: 'application/json' });
    const url = URL.createObjectURL(blob);
    const link = document.createElement('a');
    link.href = url;
    link.download = filename;
    document.body.appendChild(link);
    link.click();
    link.remove();
    URL.revokeObjectURL(url);
  }
}

document.getElementById('exportNotes').addEventListener('click', () => {
  const notes = JSON.parse(localStorage.getItem('kit-notes')) || [];
  JSONExporter.download(notes, 'mis-notas.json');
});
```

Variables, clases, ids y atributos que debes cambiar

Elemento	Para que sirve	Que debes cambiar
data	Informacion que quieres exportar.	Puede venir de localStorage, un array o una respuesta de API.
filename	Nombre del archivo descargado.	Usa extension .json.
Blob	Objeto que representa archivo en memoria.	Cambia type si exportas otro formato.
URL.revokeObjectURL	Libera memoria despues de descargar.	Dejalo para buenas practicas.

Implementacion paso a paso

- 1 Crea un boton de exportacion.
- 2 Define de donde saldrán los datos: array, localStorage o API.
- 3 Copia JSONExporter.

- 4 Llama `JSONExporter.download(datos, nombreArchivo)`.
- 5 Abre el archivo descargado y revisa que tenga estructura correcta.

Errores comunes y como solucionarlos

- El archivo sale vacio: revisa que los datos existan antes de exportar.
- El JSON sale en una sola linea: usa `JSON.stringify(data, null, 2)`.
- El navegador bloquea la descarga: asegúrate de llamar download desde un clic del usuario.

Checklist de prueba antes de reutilizar

- El componente existe en HTML y el id coincide con el usado en JavaScript.
- No hay ids duplicados en la pagina.
- La clase visual de estado se agrega y se quita correctamente.
- No aparecen errores en la consola del navegador.
- La funcionalidad sigue funcionando en pantalla pequena.
- El componente no rompe otros elementos del proyecto.

Mejoras opcionales

- Conectar el componente con `ToastManager` para mostrar mensajes de exito o error.
- Agregar estados de carga si depende de datos externos.
- Guardar preferencias en `localStorage` si el usuario puede cambiar configuraciones.
- Mejorar accesibilidad con `aria-label`, `aria-hidden`, `focus-visible` y navegacion por teclado cuando aplique.

Cierre: como combinar las funcionalidades 26 a 40

Estas funcionalidades se pueden usar de manera individual, pero en proyectos reales normalmente se combinan. Por ejemplo, un catalogo profesional puede usar Cards dinamicas para mostrar productos, Busqueda debounce para filtrar por texto, Filtros chips para categorias, Ordenamiento para precio o nombre, Paginacion para dividir resultados, Modal para ver detalle del producto, Toast para confirmar acciones y Copy clipboard para copiar un codigo de descuento.

Patron recomendado para combinar busqueda + filtros + orden + paginacion

Ejemplo de estado global

```
const state = {
  originalItems: products,
  searchTerm: '',
  category: 'all',
  sort: 'name-asc',
  page: 1,
  perPage: 6,
};

function applyState() {
  let results = [...state.originalItems];

  if (state.searchTerm !== '') {
    results = results.filter((item) => item.name.toLowerCase().includes(state.searchTerm));
  }

  if (state.category !== 'all') {
    results = results.filter((item) => item.category === state.category);
  }

  results.sort((a, b) => a.name.localeCompare(b.name));

  const start = (state.page - 1) * state.perPage;
  const pageItems = results.slice(start, start + state.perPage);

  catalog.render(pageItems);
}
```

Checklist final general

- Si usas modulos, todas las clases reutilizables deben exportarse desde funciones.js.
- Si usas version normal, no uses import/export.
- Mantén nombres claros para ids y data-attributes.
- Usa localStorage solo para prototipos, preferencias o datos no sensibles.
- No guardes contraseñas, tokens privados o informacion delicada en localStorage.
- Documenta que variables cambiaste para poder reutilizar el componente despues.

Guia de reciclaje de componentes web

Funcionalidades 41 a 50 - cierre del recurso unificado

HTML + CSS + JavaScript modular y version normal

Este documento continua el mismo formato de las guias anteriores. Aqui se cubren las funcionalidades restantes del recurso unificado, desde Todo list hasta Responsive design. La meta es que puedas reciclar cada funcionalidad sin depender del proyecto original completo.

El orden sigue la tabla de contenido del recurso unificado: 41 Todo list, 42 Drag and drop, 43 Conversor C/F/K, 44 Cambio de fondo, 45 Imprimir pagina, 46 Reset visual, 47 Helpers DOM/storage, 48 Version modular, 49 Version normal y 50 Responsive design.

Rango	Funcionalidades cubiertas	Objetivo de esta guia
41 a 50	Todo list, Drag and drop, Conversor C/F/K, Cambio de fondo, Imprimir pagina, Reset visual, Helpers DOM/storage, Version modular, Version normal y Responsive design.	Cerrar el banco de componentes con funcionalidades de productividad, estado visual, organizacion de JavaScript y adaptabilidad responsive.

Tabla de contenido

- 41. Todo list reutilizable
- 42. Drag and drop para reordenar tareas
- 43. Conversor C/F/K con botones de unidad
- 44. Cambio de fondo segun boton seleccionado
- 45. Imprimir pagina o seccion
- 46. Reset visual de estados y clases
- 47. Helpers DOM y storage
- 48. Version modular con export/import
- 49. Version normal sin modulos
- 50. Responsive design reutilizable

Como usar esta guia

- 1 Lee primero la descripcion de la funcionalidad y confirma que es lo que necesitas.
- 2 Copia el HTML minimo y cambia textos, ids y data-attributes.
- 3 Copia el CSS y adapta colores, espaciados, responsive y estados visuales.
- 4 Escoge una sola version de JavaScript: modular o normal.
- 5 Cambia los valores indicados en la tabla de variables de cada funcionalidad.
- 6 Prueba el checklist antes de reutilizar el componente en otro proyecto.

Regla importante: estas funcionalidades se pueden combinar. Por ejemplo, Todo list + Drag and drop + localStorage te permite crear una lista reordenable persistente. Conversor C/F/K + Cambio de fondo + Reset visual crea una mini app interactiva con estados visuales.

Convenciones que se repiten en las funcionalidades 41 a 50

Convencion	Para que sirve	Como adaptarla
storageKey	Clave unica para guardar datos en localStorage.	Usa nombres especificos como project-todos o sierra-notes para evitar mezclar datos.
data-id / data-unit / data-bg-class	Atributos HTML que comunican datos a JavaScript.	Cambia los valores, pero manten el mismo nombre si el JS los lee.
is-active / is-done / is-dragging	Clases de estado que activan estilos.	No las uses como clases base; JavaScript debe agregarlas o quitarlas.
callback	Funcion que se ejecuta cuando ocurre algo.	Usala para guardar cambios, mostrar toasts o conectar componentes.
@media	Reglas CSS para adaptar la vista a pantallas pequenas.	Ajusta breakpoints segun el diseno real.

Consejo de reutilizacion: cuando un componente usa ids, no repitas el mismo id mas de una vez. Si quieres dos TodoList en la misma pagina, cada una debe tener formId, inputId, listId y storageKey diferentes.

Mapa de combinaciones recomendadas

Combinacion	Para que sirve	Componentes involucrados
Todo list + Drag and drop	Lista de tareas reordenable con prioridades.	41 + 42 + 47
Conversor + Fondo dinamico + Reset	Calculadora interactiva con estados visuales y boton de reinicio.	43 + 44 + 46
Imprimir + Responsive	Reporte que se ve bien en pantalla y tambien en PDF/impresion.	45 + 50
Helpers + Version modular	Codigo mas limpio y reutilizable en proyectos grandes.	47 + 48
Version normal + Responsive	Proyecto simple sin modulos que aun se adapta a celular.	49 + 50

41. Todo list reutilizable

Aspecto	Explicacion
Que hace	Lista de tareas que permite crear, marcar como hecha y eliminar elementos. Es una base muy usada para paneles, checklists, tareas administrativas y prototipos CRUD.
Donde se usa en industria	Tableros de tareas, dashboards internos, seguimiento de pendientes, listas de compras, listas de validacion, checklists de onboarding y mini sistemas CRUD.
Como se personaliza	Cambia el id del formulario, input, lista, clave de localStorage, estructura de cada tarea y textos visibles. Puedes agregar prioridad, categoria, fecha limite o responsable.

Mapa rapido de reciclaje

- 1 Ubica el lugar exacto del HTML donde quieres insertar el componente.
- 2 Copia el HTML minimo y cambia textos visibles, ids y data-attributes.
- 3 Copia el CSS y ajusta colores, separaciones, radios, sombras y responsive si hace falta.
- 4 Escoge JS modular o JS normal, no ambos al mismo tiempo.
- 5 Inicializa el componente con el id correcto y prueba la interaccion principal.

HTML minimo que debes copiar

index.html

```
<section class="panel todo-widget" id="todoWidget">
  <h2>Lista de tareas</h2>

  <form class="todo-form" id="todoForm">
    <input id="todoInput" type="text" placeholder="Escribe una tarea" required>
    <button class="btn" type="submit">Agregar</button>
  </form>

  <ul class="todo-list" id="todoList"></ul>
</section>
```

CSS del componente

styles.css

```

.todo-widget {
  display: grid;
  gap: 16px;
}
.todo-form {
  display: grid;
  grid-template-columns: 1fr auto;
  gap: 10px;
}
.todo-form input {
  min-height: 44px;
  padding: 10px 12px;
  border-radius: 14px;
  border: 1px solid rgba(148, 163, 184, .5);
}
.todo-list {
  display: grid;
  gap: 10px;
  margin: 0;
  padding: 0;
  list-style: none;
}
.todo-item {
  display: grid;
  grid-template-columns: auto 1fr auto;
  align-items: center;
  gap: 10px;
  padding: 12px;
  border-radius: 16px;
  background: rgba(255,255,255,.12);
  border: 1px solid rgba(255,255,255,.18);
}
.todo-item.is-done .todo-text {
  text-decoration: line-through;
  opacity: .55;
}
.todo-delete {
  border: 0;
  border-radius: 999px;
  padding: 8px 10px;
  cursor: pointer;
}

```

JavaScript modular con export/import

Usa esta version si tu HTML carga `<script type="module" src="script.js"></script>`. La clase o funcion vive en funciones.js y la instancia se hace en script.js.

funciones.js + script.js


```

export class TodoList {
  constructor({ formId, inputId, listId, storageKey = 'kit-todos' }) {
    this.form = document.getElementById(formId);
    this.input = document.getElementById(inputId);
    this.list = document.getElementById(listId);
    this.storageKey = storageKey;
    this.todos = JSON.parse(localStorage.getItem(this.storageKey)) || [];
    this.init();
  }

  init() {
    this.form.addEventListener('submit', (event) => {
      event.preventDefault();
      this.add(this.input.value);
      this.input.value = '';
    });
    this.render();
  }

  save() {
    localStorage.setItem(this.storageKey, JSON.stringify(this.todos));
  }

  add(text) {
    const cleanText = text.trim();
    if (!cleanText) return;
    this.todos.push({ id: crypto.randomUUID(), text: cleanText, done: false });
    this.save();
    this.render();
  }

  toggle(id) {
    this.todos = this.todos.map((todo) => todo.id === id ? { ...todo, done: !todo.done } : todo);
    this.save();
    this.render();
  }

  remove(id) {
    this.todos = this.todos.filter((todo) => todo.id !== id);
    this.save();
    this.render();
  }

  render() {
    this.list.innerHTML = this.todos.map((todo) => `
      <li class="todo-item ${todo.done ? 'is-done' : ''}">
        <input type="checkbox" ${todo.done ? 'checked' : ''} data-toggle-todo="${todo.id}">
        <span class="todo-text">${todo.text}</span>
        <button class="todo-delete" type="button" data-delete-todo="${todo.id}">Eliminar</button>
      </li>
    `).join('');

    this.list.querySelectorAll('[data-toggle-todo]').forEach((checkbox) => {
      checkbox.addEventListener('change', () => this.toggle(checkbox.dataset.toggleTodo));
    });
    this.list.querySelectorAll('[data-delete-todo]').forEach((button) => {
      button.addEventListener('click', () => this.remove(button.dataset.deleteTodo));
    });
  }
}

// script.js
import { TodoList } from './funciones.js';
new TodoList({ formId: 'todoForm', inputId: 'todoInput', listId: 'todoList', storageKey: 'mis-tareas' });

```

JavaScript normal sin modulos

Usa esta version si prefieres tener todo en un solo archivo como script-normal.js y cargarlo con `<script src="script-normal.js"></script>`.

script-normal.js

```

class TodoList {
  constructor(config) {
    this.form = document.getElementById(config.formId);
    this.input = document.getElementById(config.inputId);
    this.list = document.getElementById(config.listId);
    this.storageKey = config.storageKey || 'kit-todos';
    this.todos = JSON.parse(localStorage.getItem(this.storageKey)) || [];
    this.init();
  }
  init() {
    this.form.addEventListener('submit', (event) => {
      event.preventDefault();
      this.add(this.input.value);
      this.input.value = '';
    });
    this.render();
  }
  save() { localStorage.setItem(this.storageKey, JSON.stringify(this.todos)); }
  add(text) {
    const cleanText = text.trim();
    if (!cleanText) return;
    this.todos.push({ id: Date.now().toString(), text: cleanText, done: false });
    this.save();
    this.render();
  }
  toggle(id) {
    this.todos = this.todos.map((todo) => todo.id === id ? { ...todo, done: !todo.done } : todo);
    this.save();
    this.render();
  }
  remove(id) {
    this.todos = this.todos.filter((todo) => todo.id !== id);
    this.save();
    this.render();
  }
  render() {
    this.list.innerHTML = this.todos.map((todo) => `
      <li class="todo-item ${todo.done ? 'is-done' : ''}>
        <input type="checkbox" ${todo.done ? 'checked' : ''} data-toggle-todo="${todo.id}">
        <span class="todo-text">${todo.text}</span>
        <button class="todo-delete" type="button" data-delete-todo="${todo.id}">Eliminar</button>
      </li>
    `).join('');
    this.list.querySelectorAll('[data-toggle-todo]').forEach((el) => el.addEventListener('change', (
    ) => this.toggle(el.dataset.toggleTodo)));
    this.list.querySelectorAll('[data-delete-todo]').forEach((el) => el.addEventListener('click', (
    ) => this.remove(el.dataset.deleteTodo)));
  }
}
new TodoList({ formId: 'todoForm', inputId: 'todoInput', listId: 'todoList', storageKey: 'mis-tareas' });

```

Variables, clases, ids y atributos que debes cambiar

Elemento	Para que sirve	Que debes cambiar
todoForm	Formulario que captura la nueva tarea.	Cambia el id si tienes varios formularios.
todoInput	Input donde el usuario escribe.	Debe coincidir con el input real.
todoList	Contenedor donde se pintan las tareas.	Puede ser ul, div o section.
storageKey	Clave usada en localStorage.	Usa una clave unica por proyecto.
todo-item / is-done	Clases CSS de tarjeta y estado completado.	Cambia nombres si tu CSS usa otra convencion.

Implementacion paso a paso

- 1 Copia el HTML del widget donde quieras mostrar la lista.
- 2 Copia el CSS de la lista y ajusta colores o bordes segun tu diseno.
- 3 Copia la clase TodoList en funciones.js o en script-normal.js.
- 4 Inicializa la clase pasando los ids correctos.
- 5 Prueba agregar, marcar y eliminar una tarea.

6 Recarga la pagina para verificar que localStorage conserva los datos.

Errores comunes y como solucionarlos

- No se agrega la tarea: revisa que el formId y inputId sean correctos.
- Se duplican eventos: no inicialices TodoList dos veces con el mismo contenedor.
- Las tareas se pierden al recargar: revisa storageKey y permisos del navegador.
- El texto queda inseguro si aceptas HTML del usuario: en produccion conviene escapar el texto antes de pintar.

Checklist de prueba antes de reutilizar

- El componente existe en HTML y el id coincide con el usado en JavaScript.
- No hay ids duplicados en la pagina.
- La clase visual de estado se agrega y se quita correctamente.
- No aparecen errores en la consola del navegador.
- La funcionalidad sigue funcionando en pantalla pequena.
- El componente no rompe otros elementos del proyecto.

Mejoras opcionales

- Conectar el componente con un Toast para avisar acciones exitosas o errores.
- Guardar preferencias o datos en localStorage cuando tenga sentido.
- Agregar validaciones antes de ejecutar acciones importantes.
- Mejorar accesibilidad con aria-label, aria-live, focus-visible y navegacion por teclado cuando aplique.

42. Drag and drop para reordenar tareas

Aspecto	Explicacion
Que hace	Permite arrastrar elementos para cambiar su orden. Es muy util cuando el usuario necesita priorizar, ordenar columnas, mover cards o reorganizar listas.
Donde se usa en industria	Kanban, listas de tareas, gestores de prioridades, administradores de menus, orden de productos, dashboards y constructores visuales.
Como se personaliza	Cambia el selector de items, el contenedor, la clase visual al arrastrar y la funcion que guarda el nuevo orden. Si ya tienes TodoList, este componente se conecta al array de tareas.

Mapa rapido de reciclaje

- 1 Ubica el lugar exacto del HTML donde quieres insertar el componente.
- 2 Copia el HTML minimo y cambia textos visibles, ids y data-attributes.
- 3 Copia el CSS y ajusta colores, separaciones, radios, sombras y responsive si hace falta.
- 4 Escoge JS modular o JS normal, no ambos al mismo tiempo.
- 5 Inicializa el componente con el id correcto y prueba la interaccion principal.

HTML minimo que debes copiar

index.html

```
<ul class="sortable-list" id="sortableList">
  <li class="sortable-item" draggable="true" data-id="1">Diseñar navbar</li>
  <li class="sortable-item" draggable="true" data-id="2">Crear sidebar</li>
  <li class="sortable-item" draggable="true" data-id="3">Validar formulario</li>
</ul>
```

CSS del componente

styles.css

```
.sortable-list {
  display: grid;
  gap: 10px;
  margin: 0;
  padding: 0;
  list-style: none;
}
.sortable-item {
  padding: 14px 16px;
  border-radius: 16px;
  background: rgba(255,255,255,.12);
  border: 1px solid rgba(255,255,255,.18);
  cursor: grab;
  user-select: none;
  transition: transform .2s ease, opacity .2s ease, border-color .2s ease;
}
.sortable-item:active {
  cursor: grabbing;
}
.sortable-item.is-dragging {
  opacity: .45;
  transform: scale(.98);
  border-color: #60a5fa;
}
.sortable-item.drag-over {
  outline: 2px dashed #60a5fa;
  outline-offset: 4px;
}
```

JavaScript modular con export/import

Usa esta version si tu HTML carga `<script type="module" src="script.js"></script>`. La clase o funcion vive en funciones.js y la instancia se hace en script.js.

funciones.js + script.js

```
export class DragSorter {
  constructor(listId, onChange = null) {
    this.list = document.getElementById(listId);
    this.onChange = onChange;
    this.dragging = null;
    this.init();
  }

  init() {
    this.list.addEventListener('dragstart', (event) => {
      const item = event.target.closest('[draggable="true"]');
      if (!item) return;
      this.dragging = item;
      item.classList.add('is-dragging');
    });

    this.list.addEventListener('dragend', () => {
      if (this.dragging) this.dragging.classList.remove('is-dragging');
      this.dragging = null;
      this.emitOrder();
    });

    this.list.addEventListener('dragover', (event) => {
      event.preventDefault();
      const afterElement = this.getAfterElement(event.clientY);
      if (!this.dragging) return;
      if (afterElement == null) this.list.appendChild(this.dragging);
      else this.list.insertBefore(this.dragging, afterElement);
    });
  }

  getAfterElement(y) {
    const items = [...this.list.querySelectorAll('[draggable="true"]:not(.is-dragging)')];
    return items.reduce((closest, child) => {
      const box = child.getBoundingClientRect();
      const offset = y - box.top - box.height / 2;
      if (offset < 0 && offset > closest.offset) return { offset, element: child };
      return closest;
    }, { offset: Number.NEGATIVE_INFINITY }).element;
  }

  emitOrder() {
    const order = [...this.list.children].map((item) => item.dataset.id);
    if (this.onChange) this.onChange(order);
  }
}

// script.js
import { DragSorter } from './funciones.js';
new DragSorter('sortableList', (order) => console.log('Nuevo orden:', order));
```

JavaScript normal sin modulos

Usa esta version si prefieres tener todo en un solo archivo como script-normal.js y cargarlo con `<script src="script-normal.js"></script>`.

script-normal.js

```

class DragSorter {
  constructor(listId, onChange) {
    this.list = document.getElementById(listId);
    this.onChange = onChange;
    this.dragging = null;
    this.init();
  }
  init() {
    this.list.addEventListener('dragstart', (event) => {
      const item = event.target.closest('[draggable="true"]');
      if (!item) return;
      this.dragging = item;
      item.classList.add('is-dragging');
    });
    this.list.addEventListener('dragend', () => {
      if (this.dragging) this.dragging.classList.remove('is-dragging');
      this.dragging = null;
      this.emitOrder();
    });
    this.list.addEventListener('dragover', (event) => {
      event.preventDefault();
      const afterElement = this.getAfterElement(event.clientY);
      if (!this.dragging) return;
      if (afterElement == null) this.list.appendChild(this.dragging);
      else this.list.insertBefore(this.dragging, afterElement);
    });
  }
  getAfterElement(y) {
    const items = [...this.list.querySelectorAll('[draggable="true"]:not(.is-dragging)')];
    return items.reduce((closest, child) => {
      const box = child.getBoundingClientRect();
      const offset = y - box.top - box.height / 2;
      return offset < 0 && offset > closest.offset ? { offset, element: child } : closest;
    }, { offset: Number.NEGATIVE_INFINITY }).element;
  }
  emitOrder() {
    const order = [...this.list.children].map((item) => item.dataset.id);
    if (this.onChange) this.onChange(order);
  }
}
new DragSorter('sortableList', (order) => console.log('Nuevo orden:', order));

```

Variables, clases, ids y atributos que debes cambiar

Elemento	Para que sirve	Que debes cambiar
sortableList	Id del contenedor que contiene los elementos arrastrables.	Debe coincidir con el ul o div real.
draggable="true"	Activa la capacidad de arrastrar un elemento.	Debe ir en cada item.
data-id	Identificador real de cada item.	Usa ids de base de datos o ids internos.
is-dragging	Clase visual aplicada durante el arrastre.	Puedes cambiar opacidad, borde o escala.
onChange(order)	Callback que recibe el nuevo orden.	Usalo para guardar en localStorage o enviar al backend.

Implementacion paso a paso

1. Agrega draggable="true" a cada item que quieras mover.
2. Agrega data-id para reconocer cada elemento despues de moverlo.
3. Copia el CSS del estado normal y del estado is-dragging.
4. Copia DragSorter y pasale el id del contenedor.
5. En el callback onChange guarda el nuevo orden si lo necesitas.
6. Prueba mover elementos hacia arriba y hacia abajo.

Errores comunes y como solucionarlos

- No se mueve nada: falta `event.preventDefault()` en `dragover`.
- Se pierde el orden al recargar: debes guardar el array `order` en `localStorage` o backend.
- En movil no funciona igual: `drag and drop` nativo en `touch` puede requerir libreria o eventos `pointer`.
- Todos los items tienen el mismo `data-id`: usa identificadores unicos.

Checklist de prueba antes de reutilizar

- El componente existe en HTML y el id coincide con el usado en JavaScript.
- No hay ids duplicados en la pagina.
- La clase visual de estado se agrega y se quita correctamente.
- No aparecen errores en la consola del navegador.
- La funcionalidad sigue funcionando en pantalla pequena.
- El componente no rompe otros elementos del proyecto.

Mejoras opcionales

- Conectar el componente con un `Toast` para avisar acciones exitosas o errores.
- Guardar preferencias o datos en `localStorage` cuando tenga sentido.
- Agregar validaciones antes de ejecutar acciones importantes.
- Mejorar accesibilidad con `aria-label`, `aria-live`, `focus-visible` y navegacion por teclado cuando aplique.

43. Conversor C/F/K con botones de unidad

Aspecto	Explicacion
Que hace	Herramienta donde el usuario elige la unidad con botones C, F o K y luego calcula las conversiones restantes. Sirve como patron para apps donde un boton controla la logica.
Donde se usa en industria	Calculadoras, conversores, filtros rapidos, configuradores, formularios con opciones de modo, dashboards con cambio de metrica y herramientas educativas.
Como se personaliza	Cambia los botones data-unit, formulas, textos del resultado, ids del input y contenedor. Puedes reemplazar temperatura por moneda, medidas, porcentajes o cualquier calculo.

Mapa rapido de reciclaje

- 1 Ubica el lugar exacto del HTML donde quieres insertar el componente.
- 2 Copia el HTML minimo y cambia textos visibles, ids y data-attributes.
- 3 Copia el CSS y ajusta colores, separaciones, radios, sombras y responsive si hace falta.
- 4 Escoge JS modular o JS normal, no ambos al mismo tiempo.
- 5 Inicializa el componente con el id correcto y prueba la interaccion principal.

HTML minimo que debes copiar

index.html

```
<section class="panel temperature-tool" id="temperatureTool">
  <h2>Conversor de temperatura</h2>

  <div class="unit-buttons" role="group" aria-label="Unidad inicial">
    <button class="unit-button is-active" type="button" data-unit="celsius">C</button>
    <button class="unit-button" type="button" data-unit="fahrenheit">F</button>
    <button class="unit-button" type="button" data-unit="kelvin">K</button>
  </div>

  <input id="temperatureInput" type="number" placeholder="Ingrese temperatura">
  <button class="btn" id="temperatureCalculate" type="button">Calcular</button>
  <p class="message" id="temperatureMessage"></p>
  <div class="result-box" id="temperatureResult"></div>
</section>
```

CSS del componente

styles.css


```

.temperature-tool {
  display: grid;
  gap: 16px;
}
.unit-buttons {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 10px;
}
.unit-button {
  min-height: 46px;
  border: 1px solid rgba(255,255,255,.25);
  border-radius: 16px;
  color: #fff;
  background: rgba(255,255,255,.12);
  cursor: pointer;
  font-weight: 800;
  transition: transform .2s ease, background .2s ease;
}
.unit-button:hover,
.unit-button.is-active {
  transform: translateY(-2px);
  background: linear-gradient(135deg, #2563eb, #7c3aed);
}
.result-box {
  padding: 16px;
  border-radius: 18px;
  background: rgba(255,255,255,.12);
  border: 1px solid rgba(255,255,255,.18);
  line-height: 1.7;
}

```

JavaScript modular con export/import

Usa esta version si tu HTML carga `<script type="module" src="script.js"></script>`. La clase o funcion vive en `funciones.js` y la instancia se hace en `script.js`.

funciones.js + script.js

```

export class TemperatureTool {
  constructor(config) {
    this.buttons = document.querySelectorAll(config.buttonSelector);
    this.input = document.getElementById(config.inputId);
    this.calculateButton = document.getElementById(config.calculateId);
    this.message = document.getElementById(config.messageId);
    this.result = document.getElementById(config.resultId);
    this.unit = 'celsius';
    this.init();
  }

  init() {
    this.buttons.forEach((button) => {
      button.addEventListener('click', () => this.selectUnit(button));
    });
    this.calculateButton.addEventListener('click', () => this.calculate());
  }

  selectUnit(button) {
    this.unit = button.dataset.unit;
    this.buttons.forEach((item) => item.classList.remove('is-active'));
    button.classList.add('is-active');
  }

  calculate() {
    const value = Number(this.input.value.trim());
    if (!Number.isFinite(value)) {
      this.message.textContent = 'Ingresa un numero valido.';
      this.result.innerHTML = '';
      return;
    }
    this.message.textContent = 'Calculo realizado.';

    if (this.unit === 'celsius') {
      this.result.innerHTML = `Fahrenheit: ${((value * 9 / 5) + 32).toFixed(2)} F<br>Kelvin: ${((
value + 273.15).toFixed(2))} K`;
    }
    if (this.unit === 'fahrenheit') {
      const c = (value - 32) * 5 / 9;
      this.result.innerHTML = `Celsius: ${c.toFixed(2)} C<br>Kelvin: ${((c + 273.15).toFixed(2))} K`;
    }
    if (this.unit === 'kelvin') {
      const c = value - 273.15;
      this.result.innerHTML = `Celsius: ${c.toFixed(2)} C<br>Fahrenheit: ${(((c * 9 / 5) + 32).
toFixed(2))} F`;
    }
  }
}

// script.js
import { TemperatureTool } from './funciones.js';
new TemperatureTool({
  buttonSelector: '.unit-button', inputId: 'temperatureInput', calculateId: 'temperatureCalculate',
  messageId: 'temperatureMessage', resultId: 'temperatureResult'
});

```

JavaScript normal sin modulos

Usa esta version si prefieres tener todo en un solo archivo como script-normal.js y cargarlo con `<script src="script-normal.js"></script>`.

script-normal.js

```

class TemperatureTool {
  constructor(config) {
    this.buttons = document.querySelectorAll(config.buttonSelector);
    this.input = document.getElementById(config.inputId);
    this.calculateButton = document.getElementById(config.calculateId);
    this.message = document.getElementById(config.messageId);
    this.result = document.getElementById(config.resultId);
    this.unit = 'celsius';
    this.init();
  }
  init() {
    this.buttons.forEach((button) => button.addEventListener('click', () => this.selectUnit(
button)));
    this.calculateButton.addEventListener('click', () => this.calculate());
  }
  selectUnit(button) {
    this.unit = button.dataset.unit;
    this.buttons.forEach((item) => item.classList.remove('is-active'));
    button.classList.add('is-active');
  }
  calculate() {
    const value = Number(this.input.value.trim());
    if (!Number.isFinite(value)) {
      this.message.textContent = 'Ingresa un numero valido.';
      this.result.innerHTML = '';
      return;
    }
    this.message.textContent = 'Calculo realizado.';
    if (this.unit === 'celsius') this.result.innerHTML = `Fahrenheit: ${((value * 9 / 5) + 32).
toFixed(2)} F<br>Kelvin: ${((value + 273.15).toFixed(2))} K`;
    if (this.unit === 'fahrenheit') {
      const c = (value - 32) * 5 / 9;
      this.result.innerHTML = `Celsius: ${c.toFixed(2)} C<br>Kelvin: ${((c + 273.15).toFixed(2))} K`;
    }
    if (this.unit === 'kelvin') {
      const c = value - 273.15;
      this.result.innerHTML = `Celsius: ${c.toFixed(2)} C<br>Fahrenheit: ${(((c * 9 / 5) + 32).
toFixed(2))} F`;
    }
  }
}
new TemperatureTool({ buttonSelector: '.unit-button', inputId: 'temperatureInput', calculateId:
'temperatureCalculate', messageId: 'temperatureMessage', resultId: 'temperatureResult' });

```

Variables, clases, ids y atributos que debes cambiar

Elemento	Para que sirve	Que debes cambiar
data-unit	Define la unidad seleccionada por cada boton.	Valores esperados: celsius, fahrenheit, kelvin.
unit-button	Clase de los botones de unidad.	Usala para estilos y seleccion con querySelectorAll.
temperatureInput	Campo numerico principal.	Cambia si tu calculadora usa otro input.
temperatureResult	Caja donde se imprime el resultado.	Puedes usar innerHTML para multiples lineas.
this.unit	Estado actual de la unidad seleccionada.	Puedes iniciar en fahrenheit o kelvin si quieres.

Implementacion paso a paso

- 1 Copia el HTML de los tres botones y el input.
- 2 Asegurate de que cada boton tenga data-unit diferente.
- 3 Copia el CSS de unit-buttons, unit-button y result-box.
- 4 Copia la clase TemperatureTool y cambiale los ids en la instancia.
- 5 Prueba cada unidad con un valor conocido.
- 6 Si lo reciclas para otro calculo, reemplaza las formulas dentro de calculate().

Errores comunes y como solucionarlos

- Siempre calcula Celsius: revisa que data-unit este bien escrito.
- El boton no se activa visualmente: revisa la clase is-active.
- El resultado dice NaN: valida el input antes de calcular.
- innerHTML no aparece: revisa que resultId coincida con el HTML.

Checklist de prueba antes de reutilizar

- El componente existe en HTML y el id coincide con el usado en JavaScript.
- No hay ids duplicados en la pagina.
- La clase visual de estado se agrega y se quita correctamente.
- No aparecen errores en la consola del navegador.
- La funcionalidad sigue funcionando en pantalla pequena.
- El componente no rompe otros elementos del proyecto.

Mejoras opcionales

- Conectar el componente con un Toast para avisar acciones exitosas o errores.
- Guardar preferencias o datos en localStorage cuando tenga sentido.
- Agregar validaciones antes de ejecutar acciones importantes.
- Mejorar accesibilidad con aria-label, aria-live, focus-visible y navegacion por teclado cuando aplique.

44. Cambio de fondo segun boton seleccionado

Aspecto	Explicacion
Que hace	Cambia el estado visual global de la pagina agregando clases al body. En el ejemplo, C coloca fondo rojo, F azul y K verde.
Donde se usa en industria	Temas por categoria, pasos de un wizard, modo de visualizacion, cambio de marca, estados de calculadora, filtros destacados y pantallas interactivas.
Como se personaliza	Cambia las clases body.unit-celsius, body.unit-fahrenheit y body.unit-kelvin. Puedes usar cualquier nombre: mode-admin, theme-gold, state-success, etc.

Mapa rapido de reciclaje

- 1 Ubica el lugar exacto del HTML donde quieres insertar el componente.
- 2 Copia el HTML minimo y cambia textos visibles, ids y data-attributes.
- 3 Copia el CSS y ajusta colores, separaciones, radios, sombras y responsive si hace falta.
- 4 Escoge JS modular o JS normal, no ambos al mismo tiempo.
- 5 Inicializa el componente con el id correcto y prueba la interaccion principal.

HTML minimo que debes copiar

index.html

```
<div class="unit-buttons" role="group" aria-label="Cambiar fondo">
  <button class="unit-button" type="button" data-bg-class="unit-celsius">Fondo C</button>
  <button class="unit-button" type="button" data-bg-class="unit-fahrenheit">Fondo F</button>
  <button class="unit-button" type="button" data-bg-class="unit-kelvin">Fondo K</button>
</div>
```

CSS del componente

styles.css

```
body.unit-celsius {
  background: radial-gradient(circle at top left, #ef4444, transparent 35%),
    linear-gradient(135deg, #450a0a, #111827);
}
body.unit-fahrenheit {
  background: radial-gradient(circle at top left, #3b82f6, transparent 35%),
    linear-gradient(135deg, #082f49, #111827);
}
body.unit-kelvin {
  background: radial-gradient(circle at top left, #22c55e, transparent 35%),
    linear-gradient(135deg, #052e16, #111827);
}
body.unit-celsius .unit-button[data-bg-class="unit-celsius"],
body.unit-fahrenheit .unit-button[data-bg-class="unit-fahrenheit"],
body.unit-kelvin .unit-button[data-bg-class="unit-kelvin"] {
  background: rgba(255,255,255,.28);
  border-color: rgba(255,255,255,.75);
}
```

JavaScript modular con export/import

Usa esta version si tu HTML carga `<script type="module" src="script.js"></script>`. La clase o funcion vive en funciones.js y la instancia se hace en script.js.

funciones.js + script.js

```
export class BodyClassSwitcher {
  constructor(buttonSelector, allowedClasses) {
    this.buttons = document.querySelectorAll(buttonSelector);
    this.allowedClasses = allowedClasses;
    this.init();
  }

  init() {
    this.buttons.forEach((button) => {
      button.addEventListener('click', () => {
        this.setClass(button.dataset.bgClass);
      });
    });
  }

  setClass(className) {
    document.body.classList.remove(...this.allowedClasses);
    if (this.allowedClasses.includes(className)) {
      document.body.classList.add(className);
    }
  }
}

// script.js
import { BodyClassSwitcher } from './funciones.js';
new BodyClassSwitcher('[data-bg-class]', ['unit-celsius', 'unit-fahrenheit', 'unit-kelvin']);
```

JavaScript normal sin modulos

Usa esta version si prefieres tener todo en un solo archivo como script-normal.js y cargarlo con `<script src="script-normal.js"></script>`.

script-normal.js

```
class BodyClassSwitcher {
  constructor(buttonSelector, allowedClasses) {
    this.buttons = document.querySelectorAll(buttonSelector);
    this.allowedClasses = allowedClasses;
    this.init();
  }

  init() {
    this.buttons.forEach((button) => {
      button.addEventListener('click', () => this.setClass(button.dataset.bgClass));
    });
  }

  setClass(className) {
    document.body.classList.remove(...this.allowedClasses);
    if (this.allowedClasses.includes(className)) document.body.classList.add(className);
  }
}

new BodyClassSwitcher('[data-bg-class]', ['unit-celsius', 'unit-fahrenheit', 'unit-kelvin']);
```

Variables, clases, ids y atributos que debes cambiar

Elemento	Para que sirve	Que debes cambiar
data-bg-class	Clase que el boton quiere poner en body.	Debe existir en el CSS.
allowedClasses	Lista de clases permitidas.	Agrega todas las clases que se deben limpiar antes de cambiar.
document.body.classList	Controla clases globales del body.	No lo cambies salvo que quieras afectar otro contenedor.
body.unit-*	Reglas CSS que cambian fondo.	Cambia degradados, colores e imagenes.

Implementacion paso a paso

- 1 Define que clases visuales quieres usar en body.
- 2 Crea un boton por cada clase usando data-bg-class.

- 3 Copia las reglas CSS `body.nombreClase`.
- 4 Inicializa `BodyClassSwitcher` con el selector y array de clases permitidas.
- 5 Haz clic en cada boton y revisa que el body solo tenga una clase visual activa.

Errores comunes y como solucionarlos

- Se mezclan fondos: olvidaste quitar clases anteriores con `classList.remove(...allowedClasses)`.
- No cambia el fondo: el valor `data-bg-class` no coincide con la clase CSS.
- El cambio afecta toda la pagina: si quieres solo una seccion, aplica la clase a un contenedor en vez de `body`.
- El texto pierde contraste: ajusta colores de texto para cada fondo.

Checklist de prueba antes de reutilizar

- El componente existe en HTML y el id coincide con el usado en JavaScript.
- No hay ids duplicados en la pagina.
- La clase visual de estado se agrega y se quita correctamente.
- No aparecen errores en la consola del navegador.
- La funcionalidad sigue funcionando en pantalla pequena.
- El componente no rompe otros elementos del proyecto.

Mejoras opcionales

- Conectar el componente con un Toast para avisar acciones exitosas o errores.
- Guardar preferencias o datos en `localStorage` cuando tenga sentido.
- Agregar validaciones antes de ejecutar acciones importantes.
- Mejorar accesibilidad con `aria-label`, `aria-live`, `focus-visible` y navegacion por teclado cuando aplique.

45. Imprimir pagina o seccion

Aspecto	Explicacion
Que hace	Abre el dialogo de impresion del navegador. Es util para facturas, reportes, comprobantes, ordenes, hojas de vida, certificados o resúmenes.
Donde se usa en industria	Paneles administrativos, recibos, reportes internos, tickets, comprobantes de pedido, documentos de soporte y paginas de resumen.
Como se personaliza	Puedes imprimir toda la pagina con <code>window.print()</code> o preparar CSS @media print para ocultar botones, navbar y elementos que no deben aparecer.

Mapa rapido de reciclaje

- 1 Ubica el lugar exacto del HTML donde quieres insertar el componente.
- 2 Copia el HTML minimo y cambia textos visibles, ids y data-attributes.
- 3 Copia el CSS y ajusta colores, separaciones, radios, sombras y responsive si hace falta.
- 4 Escoge JS modular o JS normal, no ambos al mismo tiempo.
- 5 Inicializa el componente con el id correcto y prueba la interaccion principal.

HTML minimo que debes copiar

index.html

```
<section class="print-area" id="invoiceArea">
  <h2>Resumen imprimible</h2>
  <p>Cliente: Carlos Madero</p>
  <p>Total: $120.000</p>
</section>

<button class="btn no-print" id="printPage" type="button">Imprimir</button>
```

CSS del componente

styles.css

```
.print-area {
  padding: 24px;
  border-radius: 18px;
  background: rgba(255,255,255,.12);
  border: 1px solid rgba(255,255,255,.18);
}
@media print {
  body {
    background: #ffffff !important;
    color: #111827 !important;
  }
  .no-print,
  .navbar,
  .sidebar,
  .toast-container,
  .modal {
    display: none !important;
  }
  .print-area {
    background: #ffffff !important;
    color: #111827 !important;
    border: 1px solid #d1d5db !important;
    box-shadow: none !important;
  }
}
```

JavaScript modular con export/import

Usa esta version si tu HTML carga `<script type="module" src="script.js"></script>`. La clase o funcion vive en funciones.js y la instancia se hace en script.js.

funciones.js + script.js

```
export class PrintButton {
  constructor(buttonId, beforePrint = null) {
    this.button = document.getElementById(buttonId);
    this.beforePrint = beforePrint;
    this.init();
  }

  init() {
    this.button.addEventListener('click', () => {
      if (this.beforePrint) this.beforePrint();
      window.print();
    });
  }
}

// script.js
import { PrintButton } from './funciones.js';
new PrintButton('printPage', () => console.log('Preparando impresion...'));
```

JavaScript normal sin modulos

Usa esta version si prefieres tener todo en un solo archivo como script-normal.js y cargarlo con `<script src="script-normal.js"></script>`.

script-normal.js

```
class PrintButton {
  constructor(buttonId, beforePrint) {
    this.button = document.getElementById(buttonId);
    this.beforePrint = beforePrint;
    this.init();
  }

  init() {
    this.button.addEventListener('click', () => {
      if (this.beforePrint) this.beforePrint();
      window.print();
    });
  }
}

new PrintButton('printPage', () => console.log('Preparando impresion...'));
```

Variables, clases, ids y atributos que debes cambiar

Elemento	Para que sirve	Que debes cambiar
printPage	Boton que dispara la impresion.	Cambia el id si tienes varios botones.
no-print	Clase para ocultar elementos al imprimir.	Agregala a botones, menus o elementos decorativos.
@media print	CSS especial para impresion.	Ajusta colores, margenes y elementos visibles.
beforePrint	Callback antes de imprimir.	Usalo para calcular totales o cerrar modales.

Implementacion paso a paso

- 1 Agrega el boton de imprimir con id unico.
- 2 Agrega clase no-print a todo lo que no debe salir en papel.
- 3 Copia @media print en tu CSS.
- 4 Inicializa PrintButton en tu JS.
- 5 Prueba vista previa de impresion y ajusta margenes o colores.

Errores comunes y como solucionarlos

- Se imprime el menu: agrega no-print o regla display none en @media print.
- Los colores salen oscuros: redefine body y print-area en @media print.
- El boton no responde: revisa que el id printPage coincida.
- Quieres imprimir solo una seccion: debes usar CSS print para ocultar todo excepto esa seccion.

Checklist de prueba antes de reutilizar

- El componente existe en HTML y el id coincide con el usado en JavaScript.
- No hay ids duplicados en la pagina.
- La clase visual de estado se agrega y se quita correctamente.
- No aparecen errores en la consola del navegador.
- La funcionalidad sigue funcionando en pantalla pequena.
- El componente no rompe otros elementos del proyecto.

Mejoras opcionales

- Conectar el componente con un Toast para avisar acciones exitosas o errores.
- Guardar preferencias o datos en localStorage cuando tenga sentido.
- Agregar validaciones antes de ejecutar acciones importantes.
- Mejorar accesibilidad con aria-label, aria-live, focus-visible y navegacion por teclado cuando aplique.

46. Reset visual de estados y clases

Aspecto	Explicacion
Que hace	Limpia clases temporales de la interfaz para regresar a un estado base. Es util cuando muchos botones cambian fondos, temas, filtros o estados visuales.
Donde se usa en industria	Dashboards con filtros, calculadoras, formularios multi-paso, juegos sencillos, prototipos de UI, reset de demos y limpieza de estados visuales.
Como se personaliza	Cambia el array de clases a limpiar, el boton de reset y si tambien quieres limpiar inputs, resultados o localStorage.

Mapa rapido de reciclaje

- 1 Ubica el lugar exacto del HTML donde quieres insertar el componente.
- 2 Copia el HTML minimo y cambia textos visibles, ids y data-attributes.
- 3 Copia el CSS y ajusta colores, separaciones, radios, sombras y responsive si hace falta.
- 4 Escoge JS modular o JS normal, no ambos al mismo tiempo.
- 5 Inicializa el componente con el id correcto y prueba la interaccion principal.

HTML minimo que debes copiar

index.html

```
<button class="btn" id="resetVisual" type="button">Reset visual</button>  
<p id="resetMessage" class="message"></p>
```

CSS del componente

styles.css

```
.message {  
  min-height: 20px;  
  color: rgba(255,255,255,.8);  
}  
body.unit-celsius,  
body.unit-fahrenheit,  
body.unit-kelvin {  
  transition: background .35s ease, color .35s ease;  
}
```

JavaScript modular con export/import

Usa esta version si tu HTML carga `<script type="module" src="script.js"></script>`. La clase o funcion vive en funciones.js y la instancia se hace en script.js.

funciones.js + script.js

```
export class VisualReset {
  constructor(config) {
    this.button = document.getElementById(config.buttonId);
    this.message = document.getElementById(config.messageId);
    this.target = config.target || document.body;
    this.classesToRemove = config.classesToRemove;
    this.extraReset = config.extraReset || null;
    this.init();
  }

  init() {
    this.button.addEventListener('click', () => this.reset());
  }

  reset() {
    this.target.classList.remove(...this.classesToRemove);
    if (this.extraReset) this.extraReset();
    if (this.message) this.message.textContent = 'Vista reiniciada.';
  }
}

// script.js
import { VisualReset } from './funciones.js';
new VisualReset({
  buttonId: 'resetVisual',
  messageId: 'resetMessage',
  classesToRemove: ['unit-celsius', 'unit-fahrenheit', 'unit-kelvin']
});
```

JavaScript normal sin modulos

Usa esta version si prefieres tener todo en un solo archivo como script-normal.js y cargarlo con `<script src="script-normal.js"></script>`.

script-normal.js

```
class VisualReset {
  constructor(config) {
    this.button = document.getElementById(config.buttonId);
    this.message = document.getElementById(config.messageId);
    this.target = config.target || document.body;
    this.classesToRemove = config.classesToRemove;
    this.extraReset = config.extraReset || null;
    this.init();
  }
  init() { this.button.addEventListener('click', () => this.reset()); }
  reset() {
    this.target.classList.remove(...this.classesToRemove);
    if (this.extraReset) this.extraReset();
    if (this.message) this.message.textContent = 'Vista reiniciada.';
  }
}
new VisualReset({ buttonId: 'resetVisual', messageId: 'resetMessage', classesToRemove: ['unit-celsius', 'unit-fahrenheit', 'unit-kelvin'] });
```

Variables, clases, ids y atributos que debes cambiar

Elemento	Para que sirve	Que debes cambiar
buttonId	Boton que ejecuta el reset.	Debe coincidir con el id HTML.
classesToRemove	Array con clases que se limpian.	Agrega aqui todos los estados visuales temporales.
target	Elemento al que se le quitan clases.	Por defecto es document.body.
extraReset	Funcion opcional para limpiar inputs o resultados.	Usala si quieres resetear mas cosas.

Implementacion paso a paso

- 1 Identifica las clases visuales temporales del proyecto.
- 2 Crea el boton de reset.

- 3 Copia la clase VisualReset.
- 4 Pasa el array de clases en classesToRemove.
- 5 Si necesitas limpiar campos, agrega extraReset.
- 6 Prueba activar varios estados y luego reiniciarlos.

Errores comunes y como solucionarlos

- No limpia nada: la clase en classesToRemove no coincide con la clase real.
- Limpia demasiado: no metas clases base del layout en el array.
- El mensaje no aparece: messageId no coincide o el elemento no existe.
- El reset visual no borra datos: solo borra clases; usa extraReset para datos.

Checklist de prueba antes de reutilizar

- El componente existe en HTML y el id coincide con el usado en JavaScript.
- No hay ids duplicados en la pagina.
- La clase visual de estado se agrega y se quita correctamente.
- No aparecen errores en la consola del navegador.
- La funcionalidad sigue funcionando en pantalla pequena.
- El componente no rompe otros elementos del proyecto.

Mejoras opcionales

- Conectar el componente con un Toast para avisar acciones exitosas o errores.
- Guardar preferencias o datos en localStorage cuando tenga sentido.
- Agregar validaciones antes de ejecutar acciones importantes.
- Mejorar accesibilidad con aria-label, aria-live, focus-visible y navegacion por teclado cuando aplique.

47. Helpers DOM y storage

Aspecto	Explicacion
Que hace	Pequeñas funciones reutilizables para escribir menos código repetido al seleccionar elementos, guardar JSON, leer JSON, mostrar mensajes o formatear datos.
Donde se usa en industria	Cualquier proyecto frontend: landing pages, formularios, dashboards, e-commerce, proyectos académicos, prototipos y aplicaciones SPA simples.
Como se personaliza	Puedes agregar helpers nuevos según tu estilo: validarEmail, formatCurrency, createElement, showError, clearHTML, debounce, throttle, etc.

Mapa rapido de reciclaje

- 1 Ubica el lugar exacto del HTML donde quieres insertar el componente.
- 2 Copia el HTML mínimo y cambia textos visibles, ids y data-attributes.
- 3 Copia el CSS y ajusta colores, separaciones, radios, sombras y responsive si hace falta.
- 4 Escoge JS modular o JS normal, no ambos al mismo tiempo.
- 5 Inicializa el componente con el id correcto y prueba la interacción principal.

HTML mínimo que debes copiar

index.html

```
<p id="helperMessage"></p>
<button class="btn" id="helperDemo" type="button">Probar helper</button>
```

CSS del componente

styles.css

```
.helper-ok {
  color: #22c55e;
  font-weight: 700;
}
.helper-error {
  color: #ef4444;
  font-weight: 700;
}
```

JavaScript modular con export/import

Usa esta versión si tu HTML carga `<script type="module" src="script.js"></script>`. La clase o función vive en `funciones.js` y la instancia se hace en `script.js`.

funciones.js + script.js

```

export const $ = (selector, parent = document) => parent.querySelector(selector);
export const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

export function saveJSON(key, value) {
  localStorage.setItem(key, JSON.stringify(value));
}

export function readJSON(key, fallback = null) {
  try {
    const value = localStorage.getItem(key);
    return value ? JSON.parse(value) : fallback;
  } catch (error) {
    console.error('Error leyendo JSON:', error);
    return fallback;
  }
}

export function showMessage(element, text, type = 'ok') {
  element.textContent = text;
  element.classList.remove('helper-ok', 'helper-error');
  element.classList.add(type === 'ok' ? 'helper-ok' : 'helper-error');
}

export function formatMoney(value, locale = 'es-CO', currency = 'COP') {
  return new Intl.NumberFormat(locale, { style: 'currency', currency }).format(value);
}

// script.js
import { $, saveJSON, readJSON, showMessage, formatMoney } from './funciones.js';
$('#helperDemo').addEventListener('click', () => {
  saveJSON('demo-price', { price: 12000 });
  const data = readJSON('demo-price', { price: 0 });
  showMessage($('#helperMessage'), `Precio: ${formatMoney(data.price)}`, 'ok');
});

```

JavaScript normal sin modulos

Usa esta version si prefieres tener todo en un solo archivo como script-normal.js y cargarlo con `<script src="script-normal.js"></script>`.

script-normal.js

```

const $ = (selector, parent = document) => parent.querySelector(selector);
const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];
function saveJSON(key, value) {
  localStorage.setItem(key, JSON.stringify(value));
}
function readJSON(key, fallback = null) {
  try {
    const value = localStorage.getItem(key);
    return value ? JSON.parse(value) : fallback;
  } catch (error) {
    console.error('Error leyendo JSON:', error);
    return fallback;
  }
}
function showMessage(element, text, type = 'ok') {
  element.textContent = text;
  element.classList.remove('helper-ok', 'helper-error');
  element.classList.add(type === 'ok' ? 'helper-ok' : 'helper-error');
}
function formatMoney(value, locale = 'es-CO', currency = 'COP') {
  return new Intl.NumberFormat(locale, { style: 'currency', currency }).format(value);
}
$('#helperDemo').addEventListener('click', () => {
  saveJSON('demo-price', { price: 12000 });
  const data = readJSON('demo-price', { price: 0 });
  showMessage($('#helperMessage'), `Precio: ${formatMoney(data.price)}`, 'ok');
});

```

Variables, clases, ids y atributos que debes cambiar

Elemento	Para que sirve	Que debes cambiar
\$	Selecciona el primer elemento que coincide.	Usalo como atajo de querySelector.

Elemento	Para que sirve	Que debes cambiar
\$\$	Selecciona varios elementos y los devuelve array.	Util para forEach, map, filter.
saveJSON / readJSON	Guarda y lee objetos en localStorage.	Cambia la key segun el proyecto.
showMessage	Muestra texto y aplica clase de estado.	Cambia clases helper-ok/helper-error.
formatMoney	Formatea dinero segun pais y moneda.	Cambia locale y currency.

Implementacion paso a paso

- 1 Copia los helpers al inicio de funciones.js o script-normal.js.
- 2 Exportalos si trabajas modular.
- 3 Usalos en tus componentes para evitar repetir codigo.
- 4 Agrega nuevos helpers cuando una accion se repita varias veces.
- 5 No crees helpers demasiado especificos si solo los usaras una vez.

Errores comunes y como solucionarlos

- \$ no existe: en modular debes importarlo antes de usarlo.
- readJSON falla: el contenido guardado no es JSON valido.
- localStorage no guarda objetos directamente: usa JSON.stringify como en saveJSON.
- formatMoney muestra otra moneda: cambia locale y currency.

Checklist de prueba antes de reutilizar

- El componente existe en HTML y el id coincide con el usado en JavaScript.
- No hay ids duplicados en la pagina.
- La clase visual de estado se agrega y se quita correctamente.
- No aparecen errores en la consola del navegador.
- La funcionalidad sigue funcionando en pantalla pequena.
- El componente no rompe otros elementos del proyecto.

Mejoras opcionales

- Conectar el componente con un Toast para avisar acciones exitosas o errores.
- Guardar preferencias o datos en localStorage cuando tenga sentido.
- Agregar validaciones antes de ejecutar acciones importantes.
- Mejorar accesibilidad con aria-label, aria-live, focus-visible y navegacion por teclado cuando aplique.

48. Version modular con export/import

Aspecto	Explicacion
Que hace	Organiza el codigo en archivos separados. funciones.js contiene clases o utilidades exportadas y script.js importa lo necesario para iniciar la pagina.
Donde se usa en industria	Frontend moderno, proyectos escalables, equipos de desarrollo, componentes reutilizables, separacion de responsabilidades y mantenimiento limpio.
Como se personaliza	Cambia los nombres de clases, rutas de importacion y funciones exportadas. Recuerda cargar el script principal con type="module".

Mapa rapido de reciclaje

- 1 Ubica el lugar exacto del HTML donde quieres insertar el componente.
- 2 Copia el HTML minimo y cambia textos visibles, ids y data-attributes.
- 3 Copia el CSS y ajusta colores, separaciones, radios, sombras y responsive si hace falta.
- 4 Escoge JS modular o JS normal, no ambos al mismo tiempo.
- 5 Inicializa el componente con el id correcto y prueba la interaccion principal.

HTML minimo que debes copiar

index.html

```
<!-- index.html -->
<script type="module" src="script.js"></script>
```

CSS del componente

styles.css

```
/* CSS no cambia por usar modulos.
   La modularidad es una forma de organizar JavaScript. */
.app {
  min-height: 100vh;
}
```

JavaScript modular con export/import

Usa esta version si tu HTML carga <script type="module" src="script.js"></script>. La clase o funcion vive en funciones.js y la instancia se hace en script.js.

funciones.js + script.js

```
// funciones.js
export class SaludoApp {
  constructor(messageId) {
    this.message = document.getElementById(messageId);
  }

  show(name) {
    this.message.textContent = `Hola, ${name}`;
  }
}

export function cleanText(value) {
  return value.trim();
}

// script.js
import { SaludoApp, cleanText } from './funciones.js';

const app = new SaludoApp('mensaje');
const nombre = cleanText(' Carlos ');
app.show(nombre);
```

JavaScript normal sin modulos

Usa esta version si prefieres tener todo en un solo archivo como script-normal.js y cargarlo con `<script src="script-normal.js"></script>`.

script-normal.js

```
// En version normal no usas export ni import.
// Todo queda disponible en el mismo archivo script-normal.js.
class SaludoApp {
  constructor(messageId) {
    this.message = document.getElementById(messageId);
  }
  show(name) {
    this.message.textContent = `Hola, ${name}`;
  }
}
function cleanText(value) {
  return value.trim();
}
const app = new SaludoApp('mensaje');
const nombre = cleanText(' Carlos ');
app.show(nombre);
```

Variables, clases, ids y atributos que debes cambiar

Elemento	Para que sirve	Que debes cambiar
type="module"	Activa soporte de import/export en el navegador.	Debe ir en el script principal.
export	Permite sacar una clase o funcion de un archivo.	Ponlo delante de class, function o const.
import	Trae codigo desde otro archivo.	La ruta debe empezar con ./ si es archivo local.
./funciones.js	Ruta del archivo que exporta.	Cambia si tu archivo se llama distinto.
script.js	Archivo que arranca la aplicacion.	Debe importar y crear instancias.

Implementacion paso a paso

- 1 Crea funciones.js para clases y funciones reutilizables.
- 2 Exporta cada clase o funcion que necesites usar afuera.
- 3 Crea script.js como archivo de arranque.
- 4 Importa lo necesario desde funciones.js.

- 5 En HTML carga script.js con type="module".
- 6 Ejecuta con Live Server; no abras el HTML como archivo local si hay problemas de modulo.

Errores comunes y como solucionarlos

- Cannot use import statement outside a module: falta type="module".
- Failed to resolve module specifier: la ruta del import esta mal.
- No se encuentra una clase: falta exportarla o importarla con el mismo nombre.
- Se ejecuta antes del HTML: coloca el script al final del body o usa DOMContentLoaded.

Checklist de prueba antes de reutilizar

- El componente existe en HTML y el id coincide con el usado en JavaScript.
- No hay ids duplicados en la pagina.
- La clase visual de estado se agrega y se quita correctamente.
- No aparecen errores en la consola del navegador.
- La funcionalidad sigue funcionando en pantalla pequena.
- El componente no rompe otros elementos del proyecto.

Mejoras opcionales

- Conectar el componente con un Toast para avisar acciones exitosas o errores.
- Guardar preferencias o datos en localStorage cuando tenga sentido.
- Agregar validaciones antes de ejecutar acciones importantes.
- Mejorar accesibilidad con aria-label, aria-live, focus-visible y navegacion por teclado cuando aplique.

49. Version normal sin modulos

Aspecto	Explicacion
Que hace	Permite usar JavaScript en un solo archivo sin export/import. Es util para ejercicios simples, entregas basicas o proyectos donde no quieres configurar modulos.
Donde se usa en industria	Landing pages pequenas, ejercicios academicos, scripts de una sola pagina, prototipos rapidos, integraciones muy simples y demos.
Como se personaliza	Cambia el nombre del archivo script-normal.js y el orden de carga. Si separas varios scripts normales, el orden importa mucho.

Mapa rapido de reciclaje

- 1 Ubica el lugar exacto del HTML donde quieres insertar el componente.
- 2 Copia el HTML minimo y cambia textos visibles, ids y data-attributes.
- 3 Copia el CSS y ajusta colores, separaciones, radios, sombras y responsive si hace falta.
- 4 Escoge JS modular o JS normal, no ambos al mismo tiempo.
- 5 Inicializa el componente con el id correcto y prueba la interaccion principal.

HTML minimo que debes copiar

index.html

```
<!-- index.html -->
<script src="script-normal.js"></script>
```

CSS del componente

styles.css

```
/* CSS tampoco cambia por usar version normal. */
.page-ready {
  opacity: 1;
}
```

JavaScript modular con export/import

Usa esta version si tu HTML carga `<script type="module" src="script.js"></script>`. La clase o funcion vive en funciones.js y la instancia se hace en script.js.

funciones.js + script.js

```
// Equivalencia conceptual en modular:
// funciones.js
export class AppNormalEquivalente {
  start() {
    console.log('App iniciada en modo modular');
  }
}

// script.js
import { AppNormalEquivalente } from './funciones.js';
new AppNormalEquivalente().start();
```

JavaScript normal sin modulos

Usa esta version si prefieres tener todo en un solo archivo como script-normal.js y cargarlo con <script src="script-normal.js"></script>.

script-normal.js

```
// script-normal.js
class AppNormal {
  start() {
    console.log('App iniciada sin modulos');
  }
}

function initApp() {
  const app = new AppNormal();
  app.start();
}

// Opcion 1: si el script esta al final del body
initApp();

// Opcion 2: si el script esta en el head
// document.addEventListener('DOMContentLoaded', initApp);
```

Variables, clases, ids y atributos que debes cambiar

Elemento	Para que sirve	Que debes cambiar
script-normal.js	Archivo unico donde vive todo el codigo.	Puedes cambiar el nombre, pero actualiza el src.
DOMContentLoaded	Evento que espera a que cargue el HTML.	Usalo si el script esta en head.
orden de scripts	En JS normal, lo de arriba debe existir antes de usarse.	Carga helpers antes que componentes que los usan.
variables globales	Todo puede quedar expuesto en window si usas var.	Prefiere const, let y clases para evitar choques.

Implementacion paso a paso

- 1 Crea un archivo script-normal.js.
- 2 Pega helpers, clases y luego el codigo de inicializacion.
- 3 Carga el script al final del body.
- 4 No uses import ni export en este archivo.
- 5 Si tienes varios archivos normales, carga primero helpers, despues clases, despues script principal.

Errores comunes y como solucionarlos

- Unexpected token export: estas usando export en un script normal.
- Clase no definida: cargaste los scripts en mal orden.
- document.getElementById retorna null: el script se ejecuto antes de que exista el HTML.
- Choques de nombres: evita variables globales genericas como data, item, app si hay muchos scripts.

Checklist de prueba antes de reutilizar

- El componente existe en HTML y el id coincide con el usado en JavaScript.
- No hay ids duplicados en la pagina.
- La clase visual de estado se agrega y se quita correctamente.
- No aparecen errores en la consola del navegador.

- La funcionalidad sigue funcionando en pantalla pequena.
- El componente no rompe otros elementos del proyecto.

Mejoras opcionales

- Conectar el componente con un Toast para avisar acciones exitosas o errores.
- Guardar preferencias o datos en localStorage cuando tenga sentido.
- Agregar validaciones antes de ejecutar acciones importantes.
- Mejorar accesibilidad con aria-label, aria-live, focus-visible y navegacion por teclado cuando aplique.

50. Responsive design reutilizable

Aspecto	Explicacion
Que hace	Hace que la pagina se adapte a celular, tablet y escritorio. Es una de las partes mas importantes para que una web se vea profesional.
Donde se usa en industria	Todas las paginas web modernas: landings, portafolios, ecommerce, dashboards, formularios, blogs, sistemas internos y aplicaciones web.
Como se personaliza	Cambia breakpoints, grid, tamanos de fuente, padding, columnas y comportamiento de navbar/sidebar. Prueba siempre en menos de 520px.

Mapa rapido de reciclaje

- 1 Ubica el lugar exacto del HTML donde quieres insertar el componente.
- 2 Copia el HTML minimo y cambia textos visibles, ids y data-attributes.
- 3 Copia el CSS y ajusta colores, separaciones, radios, sombras y responsive si hace falta.
- 4 Escoge JS modular o JS normal, no ambos al mismo tiempo.
- 5 Inicializa el componente con el id correcto y prueba la interaccion principal.

HTML minimo que debes copiar

index.html

```
<main class="responsive-layout">
  <section class="hero-card">Contenido principal</section>
  <aside class="side-card">Contenido secundario</aside>
</main>
```

CSS del componente

styles.css

```

:root {
  --container: 1120px;
  --space: 24px;
  --radius: 22px;
}
.responsive-layout {
  width: min(var(--container), calc(100% - 32px));
  margin: 0 auto;
  display: grid;
  grid-template-columns: minmax(0, 2fr) minmax(280px, 1fr);
  gap: var(--space);
}
.hero-card,
.side-card {
  padding: 24px;
  border-radius: var(--radius);
  background: rgba(255,255,255,.12);
  border: 1px solid rgba(255,255,255,.18);
}
@media (max-width: 900px) {
  .responsive-layout {
    grid-template-columns: 1fr;
  }
}
@media (max-width: 520px) {
  :root {
    --space: 14px;
  }
  .responsive-layout {
    width: min(100% - 20px, var(--container));
  }
  .hero-card,
  .side-card {
    padding: 18px;
    border-radius: 16px;
  }
  h1 {
    font-size: clamp(1.7rem, 9vw, 2.4rem);
  }
}

```

JavaScript modular con export/import

Usa esta version si tu HTML carga `<script type="module" src="script.js"></script>`. La clase o funcion vive en funciones.js y la instancia se hace en script.js.

funciones.js + script.js

```

export class ResponsiveHelper {
  constructor(outputId) {
    this.output = document.getElementById(outputId);
    this.init();
  }

  init() {
    this.update();
    window.addEventListener('resize', () => this.update());
  }

  update() {
    const width = window.innerWidth;
    let label = 'desktop';
    if (width <= 520) label = 'mobile';
    else if (width <= 900) label = 'tablet';
    if (this.output) this.output.textContent = `Vista actual: ${label} (${width}px)`;
    document.body.dataset.viewport = label;
  }
}

// script.js
import { ResponsiveHelper } from './funciones.js';
new ResponsiveHelper('viewportLabel');

```

JavaScript normal sin modulos

Usa esta version si prefieres tener todo en un solo archivo como script-normal.js y cargarlo con `<script src="script-normal.js"></script>`.

script-normal.js


```

class ResponsiveHelper {
  constructor(outputId) {
    this.output = document.getElementById(outputId);
    this.init();
  }
  init() {
    this.update();
    window.addEventListener('resize', () => this.update());
  }
  update() {
    const width = window.innerWidth;
    let label = 'desktop';
    if (width <= 520) label = 'mobile';
    else if (width <= 900) label = 'tablet';
    if (this.output) this.output.textContent = `Vista actual: ${label} (${width}px)`;
    document.body.dataset.viewport = label;
  }
}
new ResponsiveHelper('viewportLabel');

```

Variables, clases, ids y atributos que debes cambiar

Elemento	Para que sirve	Que debes cambiar
--container	Ancho maximo del contenido.	Cambia segun diseno: 960px, 1120px, 1280px.
breakpoints	Puntos donde cambia el layout.	Ajusta 900px y 520px segun tu pagina.
grid-template-columns	Define columnas en desktop.	Cambia 2fr/1fr por 1fr/1fr o repeat.
clamp()	Controla tamanos fluidos.	Util para titulos responsive.
data-viewport	Etiqueta el estado actual desde JS.	Puedes usarla para depurar o cambiar estilos.

Implementacion paso a paso

- 1 Empieza disenando mobile o desktop, pero prueba ambos.
- 2 Usa contenedores con width: min(...) para evitar desbordes.
- 3 Convierte grids de varias columnas a una columna en tablet/mobile.
- 4 Reduce padding y radios en pantallas pequenas.
- 5 Prueba navbar, sidebar, formularios, modales y tablas en menos de 520px.
- 6 Usa DevTools para revisar distintos anchos.

Errores comunes y como solucionarlos

- Aparece scroll horizontal: revisa widths fijos, imagenes grandes o grid sin minmax(0, 1fr).
- El texto queda gigante en movil: usa clamp o media queries.
- Las tablas se rompen: agrega overflow-x:auto al contenedor.
- El sidebar tapa todo: define comportamiento mobile para menus fijos.

Checklist de prueba antes de reutilizar

- El componente existe en HTML y el id coincide con el usado en JavaScript.
- No hay ids duplicados en la pagina.
- La clase visual de estado se agrega y se quita correctamente.
- No aparecen errores en la consola del navegador.
- La funcionalidad sigue funcionando en pantalla pequena.

- El componente no rompe otros elementos del proyecto.

Mejoras opcionales

- Conectar el componente con un Toast para avisar acciones exitosas o errores.
- Guardar preferencias o datos en localStorage cuando tenga sentido.
- Agregar validaciones antes de ejecutar acciones importantes.
- Mejorar accesibilidad con aria-label, aria-live, focus-visible y navegacion por teclado cuando aplique.

Cierre: como usar las funcionalidades 41 a 50 en proyectos reales

Estas ultimas funcionalidades completan el banco porque cubren tareas muy comunes: guardar tareas, reordenar elementos, calcular valores, cambiar estados visuales, imprimir, reiniciar interfaces, usar helpers, decidir entre JS modular o normal y hacer responsive el proyecto. Si entiendes estas piezas, puedes armar muchas paginas profesionales sin empezar desde cero.

Patron recomendado para un proyecto reutilizable

Ejemplo de integracion modular

```
// 1. HTML define estructura e ids.
// 2. CSS define apariencia y estados visuales.
// 3. funciones.js guarda clases reutilizables.
// 4. script.js importa e inicializa.
// 5. localStorage guarda estado si el usuario debe conservar datos.
// 6. @media adapta el diseno a celular.

import { TodoList, DragSorter, TemperatureTool, BodyClassSwitcher, VisualReset } from './funciones.js';

new TodoList({ formId: 'todoForm', inputId: 'todoInput', listId: 'todoList', storageKey: 'project-todos'
});
new DragSorter('todoList', (order) => console.log(order));
new TemperatureTool({
  buttonSelector: '.unit-button',
  inputId: 'temperatureInput',
  calculateId: 'temperatureCalculate',
  messageId: 'temperatureMessage',
  resultId: 'temperatureResult'
});
new BodyClassSwitcher('[data-bg-class]', ['unit-celsius', 'unit-fahrenheit', 'unit-kelvin']);
new VisualReset({ buttonId: 'resetVisual', classesToRemove: ['unit-celsius', 'unit-fahrenheit', 'unit-kelvin'] });
```

Checklist final de integracion

- Cada id del HTML existe una sola vez.
- Cada clase usada por JavaScript existe en CSS si necesita estilo visual.
- Cada data-attribute tiene el mismo nombre que lee JavaScript.
- Las claves de localStorage son unicas por proyecto.
- No mezclas import/export con script normal en el mismo flujo.
- La pagina se prueba en desktop, tablet y celular.
- La consola del navegador no muestra errores.

Actualizacion funcional del bloque 41 a 57

Este anexo actualiza los codigos de copiar y pegar para que queden alineados con la pagina actual y funcionen como referencia implementable.

- Usa estos bloques como fuente de consulta y copiar/pegar para HTML, CSS y JavaScript.
- El contenido anterior del PDF base se conserva, pero este anexo manda para implementacion.

51. Confirmación visual

¿Que hace?

Agrega retroalimentación inmediata después de una acción: guardar, eliminar, copiar, iniciar sesión o terminar una validación. En este proyecto normalmente se resuelve con toasts, badges de estado o mensajes cortos cerca del componente.

¿Cuándo usarlo?

Usalo cuando quieras que el usuario sepa de inmediato si una acción salió bien, si faltó un dato o si algo cambió en la interfaz.

Paso a paso para reciclarlo en otro proyecto

1. Define que acciones deben mostrar confirmación: guardar, borrar, copiar o actualizar.
2. Elige el canal visual: toast flotante, badge temporal, texto de estado o combinación de ellos.
3. Mantén un mensaje corto y claro, por ejemplo: "Nota guardada." o "Código copiado."
4. Dispara la confirmación justo después de que la acción termine correctamente.
5. Si también hay error, usa un mensaje alternativo como "No se pudo guardar" o "Revisa los campos".
6. Prueba tanto el caso exitoso como el caso fallido para que la retroalimentación no engañe al usuario.

Checklist recomendado

- El mensaje aparece en el momento correcto.
- El usuario entiende que pasó sin releer la pantalla completa.
- El color y el contraste siguen siendo legibles en tema oscuro y claro.
- La confirmación desaparece sola o puede cerrarse fácilmente.

Con que combinarlo

Esta funcionalidad encaja muy bien con Toast, Login demo, Notas, Todo list, Copy clipboard y Reset demo.

Paso a paso detallado para construirlo desde cero

1. Identifica en tu proyecto el lugar exacto donde necesitas insertar el componente. Analiza si va en el header, en el main, en un sidebar o en un modal.
2. Crea el HTML mínimo con los ids y clases correctas. Si copias de este banco, asegúrate de que no haya ids duplicados en tu página.
3. Copia el CSS que define la apariencia del componente. Presta atención a las variables CSS que controlan colores, espacios, radios y sombras.
4. Si el componente usa animaciones o transiciones, verifica que las propiedades `transition` y `animation` estén definidas en CSS.
5. Copia la clase JavaScript modular o la función normal. Si usas módulos, asegúrate de que `script.js` tenga `type="module"` y que las rutas de import sean correctas.
6. Inicializa el componente pasando los ids correctos. Si el constructor espera un id

- y tu HTML tiene otro, cambia el argumento al crear la instancia.
7. Prueba el componente en el navegador. Abre la consola (F12) y verifica que no haya errores rojos.
 8. Si hay errores, revisa que los elementos existan en el DOM antes de que JavaScript intente usarlos. Coloca el script al final del body o usa DOMContentLoaded.
 9. Prueba el componente en diferentes resoluciones: escritorio, tablet y movil. Usa DevTools para simular distintos anchos de pantalla.
 10. Si todo funciona correctamente, guarda una copia de seguridad y documenta cualquier personalizacion que hayas hecho.

Errores comunes y como solucionarlos

El componente no aparece: revisa que el HTML tenga el id correcto y que el script se cargue despues de que el HTML exista. Si el script esta en el head, el elemento no existe todavia cuando JavaScript intenta seleccionarlo.

El estilo no se aplica: revisa que las clases CSS del componente esten en styles.css y que no haya errores de sintaxis que rompan el resto del archivo.

La interaccion no funciona: revisa que los event listeners se conecten a elementos que realmente existen. Si usas querySelector, confirma que el selector coincida con el HTML.

El componente se ve mal en movil: revisa las reglas @media. Asegurate de que los anchos fijos, imagenes grandes o grids sin minmax no causen scroll horizontal.

Conflictos con otros componentes: si tienes varios componentes con ids o clases similares, puede haber choques. Usa nombres descriptivos y unicos para cada componente.

Checklist de prueba antes de reutilizar

El componente existe en HTML y el id coincide con el usado en JavaScript.

No hay ids duplicados en la pagina.

La clase visual de estado se agrega y se quita correctamente.

No aparecen errores en la consola del navegador.

La funcionalidad sigue funcionando en pantalla pequena.

El componente no rompe otros elementos del proyecto.

El CSS no genera scroll horizontal en ninguna resolucion.

Los eventos responden al clic, al teclado o al scroll segun corresponda.

Que se puede reutilizar directamente

El patron de estructura HTML + CSS + JS de esta funcionalidad se puede adaptar para cualquier componente similar. La logica de seleccion de elementos, manejo de eventos y actualizacion del DOM es un patron universal que se repite en la mayoria de los componentes web.

La forma de organizar el codigo en clases con constructor, init y metodos especificos se puede copiar para crear nuevos componentes. Solo cambias los selectores, los eventos y la logica interna.

Las variables CSS personalizadas se pueden reutilizar en otros componentes para mantener consistencia de colores, espacios y radios.

Variables, clases, ids y atributos que debes cambiar

- ****Elemento**** (Para que sirve): Que debes cambiar
- ****id principal**** (identifica el contenedor del componente): Cambia si ya existe ese id en tu pagina
- ****clases CSS**** (definen la apariencia visual): Cambia si colisionan con otras clases
- ****data-***** (atributos personalizados para JS): Cambia los valores segun tus necesidades
- ****textos visibles**** (etiquetas, titulos, botones): Cambia a los textos de tu proyecto
- ****colores**** (variables CSS o valores hardcodeados): Cambia para que coincidan con tu marca

52. Debounce

¿Que hace?

Retrasa la ejecución de una función hasta que el usuario deja de escribir o de disparar eventos repetidos durante unos milisegundos. Sirve para búsquedas, filtros, autoguardado, validaciones en tiempo real y renderizados costosos.

¿Cuándo usarlo?

Usalo cuando un input dispara demasiadas actualizaciones seguidas y quieres evitar renders o cálculos innecesarios.

Implementación conceptual

```
function debounce(callback, delay = 300) {  
  let timeoutId;  
  return (...args) => {  
    clearTimeout(timeoutId);  
    timeoutId = setTimeout(() => callback(...args), delay);  
  };  
}
```

Paso a paso para reciclarlo en otro proyecto

1. Crea la función debounce en tu archivo de utilidades.
2. Envuelve con debounce el callback que hoy se ejecuta demasiado seguido.
3. Ajusta el delay según la experiencia que quieras: 200 a 350 ms suele funcionar bien para búsquedas.
4. Mantén la lógica principal dentro del callback original y deja a debounce solo el control del tiempo.
5. Prueba escribiendo rápido y confirma que la interfaz responde sin saltos.

Donde se usa mejor

- Buscadores de catálogo
- Filtros de tablas
- Autoguardado de formularios
- Validación mientras se escribe

Errores comunes

- Aplicarlo a acciones que deben ser instantáneas, como un botón principal.
- Usar un delay demasiado alto y hacer que la interfaz se sienta lenta.
- Mezclar debounce y lógica de render sin reiniciar correctamente el timeout.

Paso a paso detallado para construirlo desde cero

1. Identifica en tu proyecto el lugar exacto donde necesitas insertar el componente.
Analiza si va en el header, en el main, en un sidebar o en un modal.

2. Crea el HTML mínimo con los ids y clases correctas. Si copias de este banco, asegurate de que no haya ids duplicados en tu página.
3. Copia el CSS que define la apariencia del componente. Presta atención a las variables CSS que controlan colores, espacios, radios y sombras.
4. Si el componente usa animaciones o transiciones, verifica que las propiedades `transition` y `animation` estén definidas en CSS.
5. Copia la clase JavaScript modular o la función normal. Si usas módulos, asegurate de que `script.js` tenga `type="module"` y que las rutas de `import` sean correctas.
6. Inicializa el componente pasando los ids correctos. Si el constructor espera un id y tu HTML tiene otro, cambia el argumento al crear la instancia.
7. Prueba el componente en el navegador. Abre la consola (F12) y verifica que no haya errores rojos.
8. Si hay errores, revisa que los elementos existan en el DOM antes de que JavaScript intente usarlos. Coloca el script al final del body o usa `DOMContentLoaded`.
9. Prueba el componente en diferentes resoluciones: escritorio, tablet y móvil. Usa DevTools para simular distintos anchos de pantalla.
10. Si todo funciona correctamente, guarda una copia de seguridad y documenta cualquier personalización que hayas hecho.

Errores comunes y como solucionarlos

El componente no aparece: revisa que el HTML tenga el id correcto y que el script se cargue después de que el HTML exista. Si el script está en el head, el elemento no existe todavía cuando JavaScript intenta seleccionarlo.

El estilo no se aplica: revisa que las clases CSS del componente estén en `styles.css` y que no haya errores de sintaxis que rompan el resto del archivo.

La interacción no funciona: revisa que los event listeners se conecten a elementos que realmente existen. Si usas `querySelector`, confirma que el selector coincida con el HTML.

El componente se ve mal en móvil: revisa las reglas `@media`. Asegurate de que los anchos fijos, imágenes grandes o grids sin `minmax` no causen scroll horizontal.

Conflictos con otros componentes: si tienes varios componentes con ids o clases similares, puede haber choques. Usa nombres descriptivos y únicos para cada componente.

Checklist de prueba antes de reutilizar

El componente existe en HTML y el id coincide con el usado en JavaScript.

No hay ids duplicados en la página.

La clase visual de estado se agrega y se quita correctamente.

No aparecen errores en la consola del navegador.

La funcionalidad sigue funcionando en pantalla pequeña.

El componente no rompe otros elementos del proyecto.

El CSS no genera scroll horizontal en ninguna resolución.

Los eventos responden al clic, al teclado o al scroll según corresponda.

Que se puede reutilizar directamente

El patron de estructura HTML + CSS + JS de esta funcionalidad se puede adaptar para cualquier componente similar. La logica de seleccion de elementos, manejo de eventos y actualizacion del DOM es un patron universal que se repite en la mayoría de los componentes web.

La forma de organizar el codigo en clases con constructor, init y metodos especificos se puede copiar para crear nuevos componentes. Solo cambias los selectores, los eventos y la logica interna.

Las variables CSS personalizadas se pueden reutilizar en otros componentes para mantener consistencia de colores, espacios y radios.

Variables, clases, ids y atributos que debes cambiar

- ****Elemento**** (Para que sirve): Que debes cambiar
- ****id principal**** (identifica el contenedor del componente): Cambia si ya existe ese id en tu pagina
- ****clases CSS**** (definen la apariencia visual): Cambia si colisionan con otras clases
- ****data-***** (atributos personalizados para JS): Cambia los valores segun tus necesidades
- ****textos visibles**** (etiquetas, titulos, botones): Cambia a los textos de tu proyecto
- ****colores**** (variables CSS o valores hardcodedos): Cambia para que coincidan con tu marca

53. Tarjeta de presentación en Bootstrap

Que hace?

Es una actividad guiada para construir una tarjeta de presentación personal usando la lógica de componentes de Bootstrap como referencia, pero adaptada al estilo visual de este proyecto. La card muestra información personal en una cara frontal (nombre, rol, descripción, enlaces) y una cara trasera con tecnologías o habilidades. El cambio de cara se realiza con un efecto flip 3D al hacer clic en los botones. Todo sin recargar la página.

Objetivo de la actividad

Aprender a estructurar una card con jerarquía visual, aplicar conceptos de Bootstrap (container, row, col, card, badge, btn, shadow) en un contexto personalizado, crear un efecto flip 3D con CSS puro y controlar la interacción con JavaScript. En esta página se resolvió con una interfaz que simula el laboratorio Bootstrap pero con estilos propios.

Estructura general del ejercicio

La interfaz se divide en dos caras dentro de un contenedor 3D. La cara frontal muestra un avatar circular, nombre completo, rol profesional, badges de tecnologías, descripción breve, enlaces de contacto y un botón para ver tecnologías. La cara trasera muestra una lista de tecnologías dominadas, descripción del perfil técnico y un botón para volver al perfil. El contenedor padre aplica perspective y transform-style para lograr el efecto de giro.

HTML mínimo que debes copiar

El contenedor principal usa la clase profile-lab para agrupar toda la actividad. El id profileLab es necesario para que JavaScript pueda seleccionar el contenedor si necesita manipularlo.

El contenedor del flip usa la clase profile-lab__flip y el id profileLabFlip. Este es el elemento que rota. Debe tener position: relative para que las dos caras se posicionen absolutamente dentro de él.

La cara frontal usa profile-lab__face y profile-lab__face--front. Dentro tiene: avatar (profile-lab__avatar con iniciales), nombre (h4), rol (profile-lab__role), badges (profile-lab__badges con span), descripción (p.muted), enlaces (profile-lab__links con a) y botón (showProfileTech con id).

La cara trasera usa profile-lab__face y profile-lab__face--back. Dentro tiene: título (h4), grid de tecnologías (profile-lab__tech-grid con span), descripción (p.muted) y botón (hideProfileTech con id).

Los ids showProfileTech y hideProfileTech son fundamentales porque JavaScript los busca para agregar y quitar la clase is-flipped del contenedor profileLabFlip.

index.html

```
<div class="profile-lab" id="profileLab">
  <div class="profile-lab__flip" id="profileLabFlip">
    <section class="profile-lab__face profile-lab__face--front">
      <div class="profile-lab__avatar">CM</div>
      <h4>Carlos Madero</h4>
```

```

    <p class="profile-lab__role">Full Stack Java Developer</p>
    <div class="profile-lab__badges">
      <span>Bootstrap</span>
      <span>JavaScript</span>
      <span>Data</span>
    </div>
    <p class="muted">Card de presentacion personal con una estructura parecida a
un ejercicio Bootstrap, pero adaptada al look de esta guia.</p>
    <div class="profile-lab__links">
      <a href="mailto:cmadero08x@gmail.com">Email</a>
      <a href="https://github.com/raydan08x" target="_blank"
rel="nofollow">GitHub</a>
      <a href="https://www.linkedin.com" target="_blank"
rel="nofollow">LinkedIn</a>
    </div>
    <button type="button" class="primary-button" id="showProfileTech">Ver
tecnologias</button>
  </section>
  <section class="profile-lab__face profile-lab__face--back">
    <h4>Tecnologias</h4>
    <div class="profile-lab__tech-grid">
      <span>Bootstrap grid</span>
      <span>Card</span>
      <span>Badge</span>
      <span>Buttons</span>
      <span>Shadow</span>
      <span>Responsive</span>
    </div>
    <p class="muted">La actividad te pide comprender que pone Bootstrap y que debe
completar tu CSS personalizado.</p>
    <button type="button" class="secondary-button" id="hideProfileTech">Volver al
perfil</button>
  </section>
</div>
</div>

```

CSS del componente

Las reglas de profile-lab usan perspective: 1000px para crear profundidad 3D. Sin esta propiedad, el giro se ve plano y sin efecto de acercamiento. El width y max-width limitan el ancho de la card para que no se desborde en pantallas grandes.

La clase profile-lab__flip usa position: relative, width: 100%, height: 100%, transform-style: preserve-3d y transition: transform 0.6s ease. La propiedad transform-style: preserve-3d es obligatoria para que las dos caras se mantengan en el espacio 3D mientras giran. Sin ella, el navegador aplanar el giro y se ve mal. El transition controla la duracion del giro.

La clase is-flipped aplica transform: rotateY(180deg). Cuando JavaScript agrega esta clase al contenedor, la card gira 180 grados en el eje Y, mostrando la cara trasera.

La clase profile-lab__face usa position: absolute, inset: 0, backface-visibility: hidden y border-radius. La propiedad backface-visibility: hidden es critica porque evita que se vea la cara trasera al reves cuando la frontal esta visible. Sin ella, el texto de ambas caras se superpone y se lee mal.

La cara trasera `profile-lab__face--back` usa `transform: rotateY(180deg)` para que ya este orientada al revés desde el inicio. Cuando el contenedor gira, esta cara queda frontal.

El avatar `profile-lab__avatar` usa `width`, `height`, `border-radius: 50%`, `display: grid`, `place-items: center` y un gradiente de fondo. Esto crea un círculo perfecto con iniciales centradas.

Los badges `profile-lab__badges span` usan `padding`, `border-radius`, `border` y `background` para parecerse a los badges de Bootstrap. El `display: flex` con `gap` alinea los badges horizontalmente.

Los enlaces `profile-lab__links a` usan `padding`, `border-radius`, `border` y `text-decoration: none` para parecer botones secundarios. El `hover` cambia el fondo y el borde para dar retroalimentación.

La grid de tecnologías `profile-lab__tech-grid` usa `display: grid` con `grid-template-columns: repeat(3, 1fr)` y `gap`. Cada `span` tiene `padding`, `border-radius`, `background` y `border` para parecerse a ítems de conocimiento.

styles.css

```
.profile-lab {
  perspective: 1000px;
  width: 100%;
  max-width: 400px;
  margin: 0 auto;
}
.profile-lab__flip {
  position: relative;
  width: 100%;
  height: 100%;
  transform-style: preserve-3d;
  transition: transform 0.6s ease;
}
.profile-lab__flip.is-flipped {
  transform: rotateY(180deg);
}
.profile-lab__face {
  position: absolute;
  inset: 0;
  backface-visibility: hidden;
  border-radius: 20px;
  padding: 24px;
  background: rgba(255, 255, 255, 0.08);
  border: 1px solid rgba(255, 255, 255, 0.22);
  display: grid;
  gap: 16px;
  align-content: start;
}
.profile-lab__face--back {
  transform: rotateY(180deg);
}
.profile-lab__avatar {
  width: 80px;
  height: 80px;
```

```
border-radius: 50%;
background: linear-gradient(135deg, #2563eb, #7c3aed);
color: #ffffff;
display: grid;
place-items: center;
font-size: 28px;
font-weight: 800;
margin: 0 auto;
}
.profile-lab__role {
  font-size: 14px;
  color: var(--muted);
  margin: 0;
}
.profile-lab__badges {
  display: flex;
  justify-content: center;
  gap: 8px;
  flex-wrap: wrap;
}
.profile-lab__badges span {
  padding: 4px 10px;
  border-radius: 999px;
  background: rgba(255, 255, 255, 0.1);
  border: 1px solid rgba(255, 255, 255, 0.22);
  font-size: 12px;
  font-weight: 700;
}
.profile-lab__links {
  display: flex;
  justify-content: center;
  gap: 10px;
  flex-wrap: wrap;
}
.profile-lab__links a {
  padding: 8px 14px;
  border-radius: 10px;
  border: 1px solid rgba(255, 255, 255, 0.22);
  background: rgba(255, 255, 255, 0.08);
  color: inherit;
  text-decoration: none;
  font-size: 13px;
  font-weight: 700;
  transition: background var(--transition), border-color var(--transition);
}
.profile-lab__links a:hover {
  background: rgba(255, 255, 255, 0.14);
  border-color: var(--primary);
}
.profile-lab__tech-grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 10px;
}
.profile-lab__tech-grid span {
```

```
padding: 8px;
border-radius: 10px;
background: rgba(255, 255, 255, 0.08);
border: 1px solid rgba(255, 255, 255, 0.22);
font-size: 12px;
font-weight: 700;
text-align: center;
}
```

JavaScript modular con export/import

Usa esta version si tu HTML carga el script con type="module".

La clase ProfilePresentationActivity busca el contenedor del flip con getElementById('profileLabFlip'). Tambien busca los dos botones: showProfileTech y hideProfileTech. Si alguno de estos elementos no existe, la clase no se inicializa.

En el constructor se guardan las referencias en this. El metodo init() conecta los eventos. Cuando se hace clic en showProfileTech, se agrega la clase is-flipped al contenedor. Cuando se hace clic en hideProfileTech, se quita la clase is-flipped.

La logica es intencionalmente simple porque la mayor parte del trabajo visual la hace CSS. JavaScript solo actua como interruptor: agrega o quita una clase. Este patron es muy eficiente porque delega la animacion al navegador a traves de CSS, en lugar de calcular frames con JavaScript.

La propiedad classList.toggle podria usarse, pero en este caso se usa classList.add y classList.remove para ser explicito. Si usaras toggle en ambos botones, podrias terminar con estados inesperados si el usuario hace clic dos veces en el mismo boton.

funciones.js + script.js

```
export class ProfilePresentationActivity {
  constructor() {
    this.flip = document.getElementById('profileLabFlip');
    this.showButton = document.getElementById('showProfileTech');
    this.hideButton = document.getElementById('hideProfileTech');
    if (this.flip && this.showButton && this.hideButton) {
      this.init();
    }
  }

  init() {
    this.showButton.addEventListener('click', () => {
      this.flip.classList.add('is-flipped');
    });
    this.hideButton.addEventListener('click', () => {
      this.flip.classList.remove('is-flipped');
    });
  }
}

// script.js
import { ProfilePresentationActivity } from './funciones.js';
new ProfilePresentationActivity();
```

JavaScript normal sin modulos

Usa esta version si tu HTML carga el script sin type="module".

La logica es exactamente la misma que la version modular. La diferencia es que no hay export/import. Todo vive en el mismo archivo. La clase ProfilePresentationActivity no lleva export.

Esta version es util para ejercicios academicos, paginas simples o cuando no quieres configurar modulos. Solo tienes que asegurarte de que este script se cargue despues de que el HTML exista, idealmente al final del body o con DOMContentLoaded.

script.js

```
class ProfilePresentationActivity {
  constructor() {
    this.flip = document.getElementById('profileLabFlip');
    this.showButton = document.getElementById('showProfileTech');
    this.hideButton = document.getElementById('hideProfileTech');
    if (this.flip && this.showButton && this.hideButton) {
      this.init();
    }
  }

  init() {
    this.showButton.addEventListener('click', () => {
      this.flip.classList.add('is-flipped');
    });
    this.hideButton.addEventListener('click', () => {
      this.flip.classList.remove('is-flipped');
    });
  }
}

new ProfilePresentationActivity();
```

Errores comunes y como solucionarlos

El giro se ve plano o sin profundidad: si falta perspective en el contenedor padre, el giro parece una simple transparencia en lugar de una rotacion 3D. Asegurate de que profile-lab tenga perspective: 1000px o similar. Valores entre 800px y 1200px suelen funcionar bien.

El texto se superpone al girar: si falta backface-visibility: hidden en las caras, ambas caras son visibles al mismo tiempo y el texto se lee invertido o superpuesto. Asegurate de que profile-lab__face tenga backface-visibility: hidden.

La cara trasera se ve al reves: si olvidas transform: rotateY(180deg) en profile-lab__face--back, la cara trasera aparece invertida cuando el contenedor gira. Esta regla es obligatoria porque la cara trasera debe estar orientada al reves desde el inicio para que al girar el contenedor quede frontal.

El giro no funciona: si los ids showProfileTech o hideProfileTech no coinciden con los del HTML, los botones no encuentran sus listeners. Revisa que los ids en el HTML y en JavaScript sean exactamente iguales. Tambien verifica que el script se cargue despues del HTML.

La card se corta en movil: si el contenido de la cara trasera es mas alto que el de la frontal, el contenedor puede cortarse al girar. Asegurate de que ambas caras tengan contenido similar en altura, o usa min-height en el contenedor para reservar espacio suficiente.

Los badges no se alinean: si usas display: flex sin justify-content: center o flex-wrap: wrap, los badges pueden quedar desalineados o desbordarse. Revisa que profile-lab__badges tenga ambas propiedades.

Checklist de prueba antes de reutilizar

El componente existe en HTML y los ids coinciden exactamente con los usados en JavaScript.

No hay ids duplicados en la pagina porque getElementById solo encuentra el primero.

El contenedor padre tiene perspective para el efecto 3D.

Las caras tienen backface-visibility: hidden.

La cara trasera tiene transform: rotateY(180deg).

El giro funciona al hacer clic en ambos botones.

El contenido se lee bien en tema oscuro y claro.

Los badges y enlaces se alinean correctamente.

La card no se corta en pantallas pequenas.

No hay errores en la consola del navegador.

Mejoras opcionales

Conectar el componente con un Toast para avisar cuando se cambia de cara.

Agregar una tercera cara con proyectos o portafolio.

Guardar el estado actual (cara frontal o trasera) en localStorage para recordarlo al recargar.

Permitir girar la card al hacer clic en cualquier parte de ella, no solo en los botones.

Agregar animaciones suaves al pasar el mouse sobre los badges o enlaces.

Mejorar accesibilidad con aria-label en los botones y aria-live para anunciar el cambio de cara a lectores de pantalla.

Reemplazar el avatar de iniciales por una imagen real usando img con border-radius: 50%.

Que se puede reutilizar directamente

El componente profile-lab se puede llevar a cualquier proyecto que necesite una tarjeta de presentacion con dos caras. El patron de flip con CSS 3D es facil de adaptar para tarjetas de producto, tarjetas de servicio, tarjetas de equipo o tarjetas de certificacion.

La estructura de avatar circular con iniciales se puede reemplazar por una imagen real con img y border-radius: 50%. El gradiente de fondo del avatar se puede cambiar por cualquier color o imagen.

El sistema de badges con display: flex, justify-content: center y flex-wrap: wrap se puede reutilizar para cualquier lista de etiquetas: tecnologias, habilidades, categorias, roles,

estados o tags.

Los enlaces estilizados como botones secundarios con hover se pueden reutilizar para cualquier grupo de acciones: contacto, redes sociales, descargas, compartir o navegacion interna.

La grid de tecnologías con grid-template-columns: repeat(3, 1fr) se puede adaptar para cualquier lista de items: herramientas, skills, proyectos, certificaciones o estadísticas.

El efecto flip con perspective, transform-style: preserve-3d y backface-visibility: hidden es un patron CSS puro que funciona en cualquier navegador moderno sin necesidad de librerías.

Variables, clases, ids y atributos que debes cambiar

- ****Elemento**** (Para que sirve): Que debes cambiar
- ****profileLab**** (id del contenedor principal): Cambia si ya tienes otro id igual
- ****profileLabFlip**** (id del contenedor que rota): Cambia si ya existe ese id en la pagina
- ****showProfileTech**** (id del boton para mostrar tecnologías): Cambia si ya existe ese id
- ****hideProfileTech**** (id del boton para volver al perfil): Cambia si ya existe ese id
- ****profile-lab**** (clase del contenedor principal): Cambia si colisiona con otra clase
- ****profile-lab__flip**** (clase del contenedor 3D): Cambia si colisiona con otra clase
- ****profile-lab__face**** (clase base de las caras): No cambies el nombre porque ya existe en el proyecto
- ****profile-lab__face--front**** (clase de la cara frontal): Cambia si quieres otro nombre
- ****profile-lab__face--back**** (clase de la cara trasera): Cambia si quieres otro nombre
- ****profile-lab__avatar**** (clase del avatar circular): Cambia si colisiona con otra clase
- ****profile-lab__role**** (clase del rol profesional): Cambia si colisiona con otra clase
- ****profile-lab__badges**** (clase de los badges): Cambia si colisiona con otra clase
- ****profile-lab__links**** (clase de los enlaces): Cambia si colisiona con otra clase
- ****profile-lab__tech-grid**** (clase de la grid de tecnologías): Cambia si colisiona con otra clase
- ****is-flipped**** (clase que activa el giro): No cambies el nombre porque ya existe en el proyecto
- ****CM**** (iniciales del avatar): Cambia por las iniciales de la persona
- ****Carlos Madero**** (nombre completo): Cambia por el nombre real
- ****Full Stack Java Developer**** (rol profesional): Cambia por el rol real
- ****Bootstrap**** (texto del badge): Cambia por la tecnología real
- ****JavaScript**** (texto del badge): Cambia por la tecnología real
- ****Data**** (texto del badge): Cambia por la tecnología real
- ****Email / GitHub / LinkedIn**** (textos de los enlaces): Cambia por las redes sociales reales
- ****Ver tecnologías**** (texto del boton frontal): Cambia por el texto que necesites

- ****Volver al perfil**** (texto del boton trasero): Cambia por el texto que necesites

54. Carrito de compras DOM

Que hace?

Es una actividad guiada para practicar manipulacion del DOM con JavaScript puro. Simula una mini tienda donde puedes agregar productos, actualizar un contador visual (badge), calcular el precio total acumulado, eliminar items individuales y vaciar el carrito completo. Todo sin recargar la pagina.

Objetivo de la actividad

Aprender a conectar botones con data-attributes, crear listas dinamicas en el DOM y mantener calculos en tiempo real. En esta pagina se resolvio con una interfaz compacta que sigue el estilo del proyecto.

Estructura general del ejercicio

La interfaz se divide en tres zonas claras. Una barra superior muestra el nombre de la tienda y el badge del carrito. El grid de productos contiene cards con nombre, precio y boton Agregar. La columna del carrito muestra la lista de items agregados, el total acumulado y un boton para vaciar todo.

HTML minimo que debes copiar

El contenedor principal usa la clase shop-lab para agrupar todo el componente.

La barra superior shop-lab__topbar muestra el titulo Mini tienda y el badge shop-lab__badge con id shopBadge. Este badge es el numero circular que cambia cada vez que agregas un producto. El id es necesario para que JavaScript pueda seleccionarlo y actualizar su texto.

El grid shop-lab__grid usa dos columnas para mostrar los productos. Cada article con clase shop-product representa un producto. Dentro hay un boton con clase shop-add-button y dos atributos data-name y data-price. Esos atributos son fundamentales porque JavaScript los lee para saber que nombre y precio tiene cada producto sin tener que buscar texto dentro del HTML.

La seccion del carrito usa shop-cart. Tiene un encabezado shop-cart__head con el titulo Carrito y el boton Vaciar con id clearShopCart. El mensaje vacio usa shop-cart__empty con id shopEmptyMessage. Este parrafo solo se muestra cuando no hay items. La lista shop-cart__list con id shopCartList es el ul donde JavaScript inserta cada item nuevo. El total usa shop-cart__total con shopCartTotal para actualizar el precio acumulado.

index.html

```
<div class="shop-lab">
  <div class="shop-lab__topbar">
    <strong>Mini tienda</strong>
    <span class="shop-lab__badge" id="shopBadge">0</span>
  </div>

  <div class="shop-lab__grid">
    <article class="shop-product">
      <h4>Pack IPA</h4>
      <p class="muted">$18,000</p>
```

```

        <button type="button" class="primary-button shop-add-button"
            data-name="Pack IPA" data-price="18000">Agregar</button>
    </article>

    <article class="shop-product">
        <h4>Stout Reserva</h4>
        <p class="muted">$16,500</p>
        <button type="button" class="primary-button shop-add-button"
            data-name="Stout Reserva" data-price="16500">Agregar</button>
    </article>
</div>

<div class="shop-cart">
    <div class="shop-cart__head">
        <h4>Carrito</h4>
        <button type="button" class="danger-button" id="clearShopCart">Vaciar</button>
    </div>
    <p class="muted shop-cart__empty" id="shopEmptyMessage">Tu carrito esta
vacio.</p>
    <ul class="shop-cart__list" id="shopCartList"></ul>
    <div class="shop-cart__total">
        <span>Total</span>
        <strong id="shopCartTotal">$0</strong>
    </div>
</div>
</div>

```

CSS del componente

Las reglas de shop-lab usan display: grid para separar las zonas con gap. La barra superior usa display: flex con justify-content: space-between para que el titulo quede a la izquierda y el badge a la derecha. El fondo usa rgba(255,255,255,0.08) para mantener el estilo glass del proyecto sin tapar lo de atras.

El badge usa border-radius: 999px para hacerlo completamente circular. El fondo es un gradiente de azul a morado para que resalte visualmente como un contador activo.

El grid de productos usa grid-template-columns: repeat(2, minmax(0, 1fr)) para mostrar dos columnas. Cada producto tiene padding y bordes redondeados para mantener la coherencia visual.

Los items del carrito usan shop-cart__item con display: flex y justify-content: space-between para que el nombre y precio queden a la izquierda y el boton Eliminar a la derecha. Este layout es clave para que la lista se vea ordenada sin necesidad de tablas.

styles.css

```

.shop-lab {
    display: grid;
    gap: 18px;
}
.shop-lab__topbar {
    display: flex;
    align-items: center;
    justify-content: space-between;
    gap: 12px;
}

```

```
padding: 14px 16px;
border-radius: 16px;
background: rgba(255, 255, 255, 0.08);
border: 1px solid rgba(255, 255, 255, 0.22);
}
.shop-lab__badge {
  min-width: 34px;
  height: 34px;
  display: inline-grid;
  place-items: center;
  padding: 0 10px;
  border-radius: 999px;
  color: #ffffff;
  font-weight: 800;
  background: linear-gradient(135deg, #2563eb, #7c3aed);
}
.shop-lab__grid {
  display: grid;
  grid-template-columns: repeat(2, minmax(0, 1fr));
  gap: 12px;
}
.shop-product {
  padding: 16px;
  border-radius: 16px;
  background: rgba(255, 255, 255, 0.08);
  border: 1px solid rgba(255, 255, 255, 0.22);
}
.shop-cart {
  padding: 16px;
  border-radius: 16px;
  background: rgba(255, 255, 255, 0.08);
  border: 1px solid rgba(255, 255, 255, 0.22);
  display: grid;
  gap: 12px;
}
.shop-cart__head {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 12px;
}
.shop-cart__list {
  list-style: none;
  padding: 0;
  margin: 0;
  display: grid;
  gap: 10px;
}
.shop-cart__item {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 12px;
  padding: 12px;
  border-radius: 10px;
```

```
background: rgba(255, 255, 255, 0.08);
border: 1px solid rgba(255, 255, 255, 0.22);
}
.shop-cart__total {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 12px;
}
```

JavaScript modular con export/import

Usa esta version si tu HTML carga el script con type="module".

Los helpers \$ y \$\$ abrevian querySelector y querySelectorAll. Se usan para no repetir document.querySelector cada vez que necesitas un elemento. El helper \$ busca el primer match. El helper \$\$ convierte los resultados en un array con [...] para poder usar metodos como forEach directamente.

La clase ShoppingCartActivity recibe toast como parametro opcional. El toast se usa para mostrar mensajes tipo Producto agregado al carrito. Si no lo necesitas, puedes omitirlo.

En el constructor se guardan las referencias a los elementos del DOM. Se usa querySelectorAll para obtener todos los botones .shop-add-button. Se usa getElementById para la lista, el badge, el total, el mensaje vacio y el boton de vaciar. Estas referencias se guardan en this para que todos los metodos de la clase puedan usarlas sin volver a buscarlas.

Las variables itemCount y totalAmount empiezan en 0. itemCount cuenta cuantos productos hay en el carrito. totalAmount suma los precios de todos los productos agregados.

El metodo init conecta los eventos. Recorre cada boton con forEach y agrega un listener de click. Cuando haces click, lee button.dataset.name y button.dataset.price. Esos valores vienen directamente de los atributos data-* del HTML. El precio se convierte a numero con Number() para poder sumarlo despues.

El boton de vaciar tambien tiene un listener que llama a this.clear().

El metodo renderSummary() se ejecuta al final del init para que el badge muestre 0, el total muestre \$0 y el mensaje vacio sea visible desde el inicio.

El metodo addItem recibe name y price. Crea un li con createElement. Usa innerHTML para insertar la estructura del item: un div con el nombre y precio, y un boton Eliminar. El boton usa clase danger-button para el color rojo.

El listener del boton Eliminar se crea dentro de addItem. Busca el boton dentro del item con \$('button', item). El signo de interrogacion ? es optional chaining: si no encuentra el boton, no falla. El listener llama a removeItem pasando el item completo y el precio.

Despues de agregar el item al DOM con appendChild, se suma 1 al contador y el precio al total. Luego se llama renderSummary() para actualizar la UI. Si existe el toast, muestra un mensaje de confirmacion.

El metodo removeItem usa item.remove() para eliminar el elemento del DOM. Es una de las

formas mas directas de quitar un nodo. Resta 1 al contador y el precio al total. `Math.max(0, ...)` evita que los numeros queden negativos si algo sale mal. Luego `renderSummary()` actualiza todo.

El metodo `clear()` selecciona todos los `li` de la lista con `$$('li', this.list)`. `forEach` los elimina uno por uno. Resetea los contadores a 0 y actualiza la UI. Si hay toast, muestra Carrito vaciado.

El metodo `renderSummary()` actualiza tres cosas: el badge con la cantidad, el total con el precio formateado, y la visibilidad del mensaje vacio. Si hay items, oculta el mensaje. Si no hay, lo muestra. La propiedad `style.display` controla esto directamente.

El metodo `formatPrice` usa `toLocaleString('es-CO')` para mostrar los precios con puntos de miles. El valor se convierte en algo como \$18.000 en lugar de \$18000.

funciones.js + script.js

```
export const $ = (selector, parent = document) => parent.querySelector(selector);
export const $$ = (selector, parent = document) =>
[...parent.querySelectorAll(selector)];
```

```
export class ShoppingCartActivity {
  constructor(toast) {
    this.toast = toast;
    this.buttons = $$('.shop-add-button');
    this.list = $('#shopCartList');
    this.badge = $('#shopBadge');
    this.total = $('#shopCartTotal');
    this.emptyMessage = $('#shopEmptyMessage');
    this.clearButton = $('#clearShopCart');
    this.itemsCount = 0;
    this.totalAmount = 0;
    if (this.buttons.length && this.list && this.badge && this.total &&
this.emptyMessage && this.clearButton) {
      this.init();
    }
  }
}
```

```
init() {
  this.buttons.forEach((button) => {
    button.addEventListener('click', () => {
      this.addItem(button.dataset.name, Number(button.dataset.price));
    });
  });
  this.clearButton.addEventListener('click', () => this.clear());
  this.renderSummary();
}
```

```
addItem(name, price) {
  const item = document.createElement('li');
  item.className = 'shop-cart__item';
  item.innerHTML = '
    <div>
      <strong>${name}</strong>
      <p class="muted">${this.formatPrice(price)}</p>
```



```

        </div>
        <button type="button" class="danger-button">X</button>
    ';
    $('button', item)?.addEventListener('click', () => this.removeItem(item,
price));
    this.list.appendChild(item);
    this.itemsCount += 1;
    this.totalAmount += price;
    this.renderSummary();
    this.toast?.show(`${name} agregado al carrito.`);
}

removeItem(item, price) {
    item.remove();
    this.itemsCount = Math.max(0, this.itemsCount - 1);
    this.totalAmount = Math.max(0, this.totalAmount - price);
    this.renderSummary();
}

clear() {
    $('li', this.list).forEach((item) => item.remove());
    this.itemsCount = 0;
    this.totalAmount = 0;
    this.renderSummary();
    this.toast?.show('Carrito vaciado.`);
}

renderSummary() {
    this.badge.textContent = String(this.itemsCount);
    this.total.textContent = this.formatPrice(this.totalAmount);
    this.emptyMessage.style.display = this.itemsCount ? 'none' : 'block';
}

formatPrice(value) {
    return `$$${value.toLocaleString('es-CO')}`;
}
}

// script.js
import { ShoppingCartActivity } from './funciones.js';
const cart = new ShoppingCartActivity();

```

JavaScript normal sin modulos

Usa esta version si tu HTML carga el script sin type="module".

La logica es exactamente la misma que la version modular. La diferencia es que no hay export/import. Todo vive en el mismo archivo. Los helpers \$ y \$\$ se definen como constantes normales. La clase ShoppingCartActivity no lleva export.

Esta version es util para ejercicios academicos, paginas simples o cuando no quieres configurar modulos. Solo tienes que asegurarte de que este script se cargue despues de que el HTML exista, idealmente al final del body o con DOMContentLoaded.

script.js

```
const $ = (selector, parent = document) => parent.querySelector(selector);
const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];
```

```
class ShoppingCartActivity {
  constructor(toast) {
    this.toast = toast;
    this.buttons = $$('.shop-add-button');
    this.list = $('#shopCartList');
    this.badge = $('#shopBadge');
    this.total = $('#shopCartTotal');
    this.emptyMessage = $('#shopEmptyMessage');
    this.clearButton = $('#clearShopCart');
    this.itemsCount = 0;
    this.totalAmount = 0;
    if (this.buttons.length && this.list && this.badge && this.total &&
this.emptyMessage && this.clearButton) {
      this.init();
    }
  }

  init() {
    this.buttons.forEach((button) => {
      button.addEventListener('click', () => {
        this.addItem(button.dataset.name, Number(button.dataset.price));
      });
    });
    this.clearButton.addEventListener('click', () => this.clear());
    this.renderSummary();
  }

  addItem(name, price) {
    const item = document.createElement('li');
    item.className = 'shop-cart__item';
    item.innerHTML = `
      <div>
        <strong>${name}</strong>
        <p class="muted">${this.formatPrice(price)}</p>
      </div>
      <button type="button" class="danger-button">X</button>
    `;
    $('button', item)?.addEventListener('click', () => this.removeItem(item,
price));
    this.list.appendChild(item);
    this.itemsCount += 1;
    this.totalAmount += price;
    this.renderSummary();
    this.toast?.show(`${name} agregado al carrito.`);
  }

  removeItem(item, price) {
    item.remove();
    this.itemsCount = Math.max(0, this.itemsCount - 1);
    this.totalAmount = Math.max(0, this.totalAmount - price);
    this.renderSummary();
  }
}
```

```

clear() {
  $$('li', this.list).forEach((item) => item.remove());
  this.itemsCount = 0;
  this.totalAmount = 0;
  this.renderSummary();
  this.toast?.show('Carrito vaciado.');
```

```

}

renderSummary() {
  this.badge.textContent = String(this.itemsCount);
  this.total.textContent = this.formatPrice(this.totalAmount);
  this.emptyMessage.style.display = this.itemsCount ? 'none' : 'block';
}

formatPrice(value) {
  return '$${value.toLocaleString('es-CO')}}';
}
}

const cart = new ShoppingCartActivity();
```

Errores comunes y como solucionarlos

No cambia el total: el precio se lee como texto en lugar de numero. Si olvidas `Number()` al leer `dataset.price`, el signo + se convierte en concatenacion de strings y terminas con \$018000 en lugar de \$18.000. Siempre usa `Number()` para convertir el precio a valor numerico antes de sumar.

El boton Eliminar no funciona: el listener debe crearse dentro del metodo `addItem`, justo despues de crear el item con `createElement`. Si lo creas fuera o en el constructor, no existe el boton todavia. La forma correcta es buscar el boton dentro del item nuevo con `$( 'button', item)` y conectarle el listener en ese momento.

El badge queda descuadrado: si solo actualizas el badge al agregar pero no al eliminar o vaciar, el numero queda desactualizado. Asegurate de llamar `renderSummary()` al final de `addItem`, `removeItem` y `clear()` para que el badge siempre refleje la cantidad real.

El mensaje vacio no desaparece: si olvidas manejar la visibilidad del mensaje vacio en `renderSummary()`, el parrafo Tu carrito esta vacio sigue visible aunque haya items. La propiedad `style.display` con valor `'none'` o `'block'` controla esto. Revisa que `emptyMessage` exista y que `renderSummary()` se ejecute despues de cada cambio.

Checklist de prueba antes de reutilizar

El componente existe en HTML y los ids coinciden exactamente con los usados en JavaScript.

No hay ids duplicados en la pagina porque `getElementById` solo encuentra el primero.

Los botones Agregar tienen `data-name` y `data-price` con valores correctos.

El mensaje vacio aparece y desaparece segun haya o no items.

No hay errores en la consola del navegador.

La funcionalidad sigue funcionando en pantalla pequena.

El componente no rompe otros elementos del proyecto.

Mejoras opcionales

Conectar el componente con un Toast para avisar cada accion: agregar, eliminar o vaciar.

Guardar el carrito en localStorage para que no se pierda al recargar la pagina.

Agregar validaciones para evitar agregar el mismo producto dos veces o permitir cantidades.

Mejorar accesibilidad con aria-label en los botones y aria-live en la lista para lectores de pantalla.

Agregar animaciones suaves al agregar o eliminar items para que la UI se sienta mas viva.

Que se puede reutilizar directamente

El componente shop-lab se puede llevar a cualquier proyecto que necesite una mini tienda, carrito de compras, lista de servicios o cotizador. El patron de botones con data-name y data-price es facil de adaptar para cualquier catalogo: productos, servicios, reservas, paquetes o suscripciones.

El sistema de badge con contador visual se puede reutilizar para cualquier contador: notificaciones, mensajes, tareas, items seleccionados, pasos completados o alertas. El gradiente de azul a morado se puede cambiar por cualquier otro color.

El grid de productos con grid-template-columns: repeat(2, minmax(0, 1fr)) se puede adaptar para cualquier lista de items: cards de producto, cards de servicio, cards de equipo o cards de proyecto. Cambia el numero de columnas segun el ancho disponible.

El layout del carrito con flex y justify-content: space-between se puede reutilizar para cualquier lista de items con acciones: lista de tareas, lista de compras, lista de favoritos, lista de pendientes o lista de contactos.

El metodo de formateo de precios con toLocaleString('es-CO') se puede adaptar para cualquier moneda: cambia 'es-CO' por 'en-US', 'es-ES', 'pt-BR' o cualquier otro locale. Cambia el simbolo \$ por el de la moneda que necesites.

El sistema de vaciar carrito con forEach y remove() es un patron reutilizable para cualquier accion de limpieza: vaciar lista, borrar seleccion, limpiar filtros o resetear formulario.

Variables, clases, ids y atributos que debes cambiar

- ****Elemento**** (Para que sirve): Que debes cambiar
- ****shopLab**** (id del contenedor principal): Cambia si ya tienes otro id igual
- ****shopBadge**** (id del badge del carrito): Cambia si ya existe ese id
- ****shopCartList**** (id de la lista de items): Cambia si ya existe ese id
- ****shopCartTotal**** (id del total acumulado): Cambia si ya existe ese id
- ****shopEmptyMessage**** (id del mensaje de carrito vacio): Cambia si ya existe ese id
- ****clearShopCart**** (id del boton vaciar): Cambia si ya existe ese id
- ****shop-lab**** (clase del contenedor principal): Cambia si colisiona con otra clase
- ****shop-lab__topbar**** (clase de la barra superior): Cambia si colisiona con otra clase
- ****shop-lab__badge**** (clase del badge circular): Cambia si colisiona con otra clase

- **shop-lab__grid** (clase del grid de productos): Cambia si colisiona con otra clase
- **shop-product** (clase de cada producto): Cambia si colisiona con otra clase
- **shop-add-button** (clase de los botones agregar): Cambia si colisiona con otra clase
- **shop-cart** (clase del contenedor del carrito): Cambia si colisiona con otra clase
- **shop-cart__head** (clase del encabezado del carrito): Cambia si colisiona con otra clase
- **shop-cart__list** (clase de la lista de items): Cambia si colisiona con otra clase
- **shop-cart__item** (clase de cada item en la lista): Cambia si colisiona con otra clase
- **shop-cart__total** (clase del total): Cambia si colisiona con otra clase
- **shop-cart__empty** (clase del mensaje vacio): Cambia si colisiona con otra clase
- **itemsCount** (variable interna de cantidad): Cambia el nombre si prefieres otro
- **totalAmount** (variable interna de precio total): Cambia el nombre si prefieres otro
- **data-name** (atributo del boton con nombre del producto): Cambia los valores por los productos reales
- **data-price** (atributo del boton con precio del producto): Cambia los valores por los precios reales
- **Pack IPA** (nombre del producto ejemplo): Cambia por los productos reales
- **\$18,000** (precio del producto ejemplo): Cambia por los precios reales
- **Mini tienda** (titulo de la tienda): Cambia por el nombre de tu tienda
- **Carrito** (titulo del carrito): Cambia por el nombre de tu lista
- **Vaciar** (texto del boton vaciar): Cambia por el texto que necesites
- **X** (texto del boton eliminar): Cambia por icono o texto que prefieras
- **Total** (texto del total): Cambia por el texto que necesites

55. Footer reutilizable

Que hace?

Es un componente estructural que se coloca al final de toda pagina web. Proporciona informacion de marca, enlaces de navegacion rapidos, datos de contacto, redes sociales y una barra de copyright. Es fundamental para cualquier sitio profesional porque cierra la jerarquia visual, mejora la navegacion y da credibilidad al proyecto.

Objetivo del componente

Crear un pie de pagina que funcione como cierre coherente de la interfaz, que sea totalmente responsive y que se pueda adaptar a cualquier proyecto simplemente cambiando textos, enlaces y colores. En esta pagina se resolvió con un estilo glass sutil que combina con el tema del proyecto.

Estructura general del componente

La interfaz se divide en tres zonas principales. La zona superior usa una grid de tres columnas: marca con descripcion, enlaces de navegacion y datos de contacto. La zona inferior es una barra separada por borde que muestra copyright y una etiqueta informativa. Todo esta contenido dentro de un footer con fondo semitransparente y borde superior.

HTML minimo que debes copiar

El contenedor principal usa la etiqueta semantica footer con clase site-footer y id siteFooter. El id es opcional si no necesitas manipularlo con JavaScript, pero es util para aplicar estilos especificos o animaciones de entrada.

La grid superior usa site-footer__grid con display: grid. En escritorio usa tres columnas: 2fr para la marca y 1fr para cada una de las otras dos columnas. Esto da mas espacio a la descripcion y distribuye equitativamente los enlaces.

La columna de marca site-footer__brand contiene el mismo logo o marca que la navbar. El span brand__mark usa el mismo estilo circular que el header para mantener coherencia visual. El parrafo muted usa la clase existente del proyecto para descripciones secundarias.

La columna de enlaces site-footer__links usa un elemento nav semantico con aria-label para accesibilidad. Dentro hay enlaces a que apuntan a secciones internas con hash o rutas externas. Los enlaces internos se conectan automaticamente con el router de secciones si tu proyecto usa el patron data-section.

La columna de contacto site-footer__contact usa enlaces a para mailto, GitHub y LinkedIn. El atributo target=_blank y rel=noreferrer son obligatorios para enlaces externos.

La barra inferior site-footer__bottom usa dos span: uno para copyright con el simbolo y ano, y otro para una etiqueta informativa como el numero de funcionalidades o version del proyecto.

index.html

```
<footer class="site-footer" id="siteFooter">
  <div class="site-footer__grid">
    <div class="site-footer__brand">
```

```

        <span class="brand__mark">CW</span>
        <strong>Componentes Web</strong>
        <p class="muted">Banco de componentes reutilizables HTML + CSS + JS para
proyectos profesionales.</p>
    </div>
    <div class="site-footer__links">
        <strong>Secciones</strong>
        <nav aria-label="Footer navegacion">
            <a href="#inicio" data-section="inicio">Inicio</a>
            <a href="#componentes" data-section="componentes">Componentes</a>
            <a href="#formularios" data-section="formularios">Formularios</a>
            <a href="#datos" data-section="datos">Datos</a>
            <a href="#utilidades" data-section="utilidades">Utilidades</a>
            <a href="#guia" data-section="guia">Guia</a>
        </nav>
    </div>
    <div class="site-footer__contact">
        <strong>Contacto</strong>
        <a href="mailto:cmadero08x@gmail.com">cmadero08x@gmail.com</a>
        <a href="https://github.com/raydan08x" target="_blank"
rel="noreferrer">GitHub</a>
        <a href="https://www.linkedin.com" target="_blank"
rel="noreferrer">LinkedIn</a>
    </div>
</div>
<div class="site-footer__bottom">
    <span>2026 Carlos Madero. Todos los derechos reservados.</span>
    <span>Kit Web Reutilizable 55 funcionalidades documentadas</span>
</div>
</footer>

```

CSS del componente

Las reglas de site-footer usan margin-top: 40px para separar el footer del contenido principal. El padding superior es mayor que el inferior para dar espacio visual. El background usa rgba(255, 255, 255, 0.04) que es un gris muy sutil que contrasta con el fondo de la pagina sin tapar el efecto glass. El border-top usa var(--border) para mantener coherencia con el resto del proyecto.

La clase site-footer__grid usa display: grid con max-width: 1120px y margin: 0 auto para centrar el contenido. El gap: 40px separa las columnas sin necesidad de padding interno. La propiedad grid-template-columns: 2fr 1fr 1fr distribuye el espacio: la marca ocupa el doble de ancho que cada columna de enlaces.

La marca site-footer__brand reutiliza la clase brand__mark del header para el circulo del logo. El strong usa font-size: 18px para que el nombre del proyecto sea visible pero no compita con el contenido principal.

Los titulos de columna site-footer__links strong y site-footer__contact strong usan text-transform: uppercase, letter-spacing: 0.04em y font-size: 14px para parecerse a subtítulos de footer profesionales. El color usa var(--text) para mantener legibilidad.

La navegacion de enlaces site-footer__links nav usa display: grid con gap: 8px para apilar los enlaces verticalmente. Cada enlace usa display: block para ocupar toda la anchura de la

columna, facilitando el clic en dispositivos táctiles.

Los enlaces a comparten las mismas reglas entre links y contact. Usan color: `var(--muted)` para no competir con el contenido principal. El hover cambia a `var(--primary)` para dar retroalimentación visual. La propiedad `transition: color var(--transition)` hace el cambio suave.

La barra inferior `site-footer__bottom` usa `display: flex` con `justify-content: space-between` para alinear copyright a la izquierda y etiqueta a la derecha. El `border-top` y `padding-top` crean la separación visual. El `font-size: 13px` y color: `var(--muted)` mantienen la jerarquía: esta información es secundaria.

En el tema claro, los colores se adaptan automáticamente porque todas las reglas usan variables CSS: `var(--border)`, `var(--text)`, `var(--muted)`. No se necesita CSS adicional para el tema claro.

styles.css

```
.site-footer {
  margin-top: 40px;
  padding: 48px 32px 24px;
  background: rgba(255, 255, 255, 0.04);
  border-top: 1px solid var(--border);
}
.site-footer__grid {
  display: grid;
  grid-template-columns: 2fr 1fr 1fr;
  gap: 40px;
  max-width: 1120px;
  margin: 0 auto 32px;
}
.site-footer__brand .brand__mark {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  width: 36px;
  height: 36px;
  border-radius: 50%;
  background: linear-gradient(135deg, var(--primary), var(--secondary));
  color: #ffffff;
  font-weight: 800;
  font-size: 14px;
  margin-right: 10px;
}
.site-footer__brand strong {
  font-size: 18px;
  font-weight: 700;
}
.site-footer__brand p {
  margin-top: 10px;
  font-size: 14px;
  max-width: 320px;
}
.site-footer__links strong,
.site-footer__contact strong {
```



```

    display: block;
    font-size: 14px;
    font-weight: 700;
    text-transform: uppercase;
    letter-spacing: 0.04em;
    margin-bottom: 14px;
    color: var(--text);
  }
  .site-footer__links nav {
    display: grid;
    gap: 8px;
  }
  .site-footer__links a,
  .site-footer__contact a {
    display: block;
    font-size: 14px;
    color: var(--muted);
    text-decoration: none;
    transition: color var(--transition);
  }
  .site-footer__links a:hover,
  .site-footer__contact a:hover {
    color: var(--primary);
  }
  .site-footer__bottom {
    display: flex;
    justify-content: space-between;
    align-items: center;
    gap: 16px;
    max-width: 1120px;
    margin: 0 auto;
    padding-top: 24px;
    border-top: 1px solid var(--border);
    font-size: 13px;
    color: var(--muted);
    flex-wrap: wrap;
  }

```

JavaScript

Este componente no requiere JavaScript propio. Es completamente estatico: HTML y CSS son suficientes para todo su funcionamiento. Los enlaces internos usan hash (#inicio, #componentes) que el navegador maneja automaticamente. Si tu proyecto usa el Router de secciones, los enlaces con data-section se conectan automaticamente a la clase RouterSections.

Si quieres agregar comportamiento dinamico, puedes hacerlo opcionalmente:

Actualizar el año automaticamente: reemplaza 2026 con `new Date().getFullYear()`.

Abrir enlaces externos en nueva pestana: ya esta resuelto con `target=_blank`.

Scroll suave al hacer clic en enlaces internos: agrega `scroll-behavior: smooth` al html.

Errores comunes y como solucionarlos

El footer tapa contenido en movil: si el margin-top del footer es muy pequeno o si el contenido anterior tiene margin-bottom negativo, el footer puede quedar pegado. Asegurate de que el elemento anterior tenga suficiente margin-bottom o que el footer tenga margin-top adecuado.

Los enlaces no funcionan: si usas rutas con hash (#inicio) pero tu pagina usa un router SPA que intercepta hashchange, el navegador puede no hacer scroll. Conecta los enlaces con data-section al router o usa scrollToView manualmente.

El footer no llega al final: si el body es mas corto que la ventana, el footer queda en medio de la pantalla. Solucion: usa min-height: 100vh en el contenedor principal o usa flexbox en el body para empujar el footer al final.

Texto del copyright desborda: si el texto del copyright es muy largo y la pantalla es estrecha, los dos span pueden quedar en la misma linea y desbordarse. La propiedad flex-wrap: wrap en site-footer__bottom lo soluciona: si no caben, se apilan.

Checklist de prueba antes de reutilizar

El footer se ve al final de la pagina sin tapar contenido.

Los enlaces internos llevan a las secciones correctas.

Los enlaces externos abren en nueva pestana y tienen rel=noreferrer.

El color y el contraste siguen siendo legibles en tema oscuro y claro.

El footer se adapta a tres columnas en escritorio, dos en tablet y una en movil.

No hay scroll horizontal en ninguna resolucion.

El texto del copyright se lee bien y no desborda.

Que se puede reutilizar directamente

El patron de grid de tres columnas con 2fr 1fr 1fr se puede adaptar para cualquier seccion de informacion: equipo, servicios, caracteristicas, testimonios o contacto.

La barra inferior con display: flex, justify-content: space-between y border-top se puede reutilizar para cualquier separador de contenido: copyright, terminos, version, estado o creditos.

El sistema de enlaces con hover que cambia de color usando var(--primary) es un patron universal que funciona en cualquier lista de navegacion.

El uso de variables CSS para colores, bordes y espacios permite que el footer se adapte automaticamente a cualquier tema sin escribir CSS adicional.

Variables, clases, ids y atributos que debes cambiar

- ****Elemento**** (Para que sirve): Que debes cambiar
- ****siteFooter**** (id del contenedor principal): Cambia si ya existe ese id
- ****site-footer**** (clase del contenedor principal): Cambia si colisiona con otra clase
- ****site-footer__grid** clase del grid de columnas**: Cambia si colisiona con otra clase
- ****site-footer__brand** clase de la columna de marca**: Cambia si colisiona con otra clase

- **site-footer__links** clase de la columna de enlaces: Cambia si colisiona con otra clase
- **site-footer__contact** clase de la columna de contacto: Cambia si colisiona con otra clase
- **site-footer__bottom** clase de la barra inferior: Cambia si colisiona con otra clase
- **brand__mark** (clase del logo circular): No cambies porque ya existe en el header
- **CW** (iniciales del logo): Cambia por las iniciales de tu proyecto
- **Componentes Web** (nombre del proyecto): Cambia por el nombre real
- **Banco de...** (descripcion de la marca): Cambia por la descripcion real
- **Secciones** (titulo de la columna de enlaces): Cambia por el titulo que necesites
- **Inicio** (texto del enlace interno): Cambia por las secciones de tu proyecto
- **Contacto** (titulo de la columna de contacto): Cambia por el titulo que necesites
- **Email / GitHub** (textos de los enlaces de contacto): Cambia por tus redes sociales reales
- **mailto** (href del correo): Cambia por tu correo real
- **https://github** (href de GitHub): Cambia por tu URL real
- **2026 Carlos Madero** (texto del copyright): Cambia por tu nombre y ano
- **55 funcionalidades** (etiqueta informativa): Cambia por el dato de tu proyecto

56. Tarjetas de precios Bootstrap

Que hace?

Es una actividad guiada para construir una seccion de tarifas o precios responsiva usando la logica de componentes de Bootstrap como referencia, pero adaptada al estilo visual de este proyecto. Cuenta con tres tarjetas de plan (Basico, Popular, Equipo) donde se resalta el plan Pro mediante bordes, sombras y degradados distintivos. Todo adaptado para pantallas moviles sin romperse.

Objetivo de la actividad

Aprender a maquetar multiples tarjetas alineadas en rejilla responsiva (Grid), implementar badges de categoria y botones de accion, diferenciar visualmente un elemento destacado (featured card) y utilizar variables de CSS para que el diseno se adapte a temas claro y oscuro de manera limpia.

Estructura general del ejercicio

El contenedor principal con clase pricing-lab agrupa las tarjetas. Cada tarjeta es un article con clase pricing-lab__card. La tarjeta destacada tiene la clase adicional pricing-lab__card--featured. Cada tarjeta contiene un badge pricing-lab__badge, un titulo de plan (h4), un precio pricing-lab__price y una lista de características pricing-lab__list.

HTML minimo que debes copiar

index.html

```
<div class="pricing-lab">
  <article class="pricing-lab__card">
    <span class="pricing-lab__badge">Basico</span>
    <h4>Starter</h4>
    <p class="pricing-lab__price">$19<span>/mes</span></p>
    <ul class="pricing-lab__list">
      <li>1 proyecto</li>
      <li>Soporte por correo</li>
      <li>Componentes base</li>
    </ul>
    <button type="button" class="secondary-button">Elegir plan</button>
  </article>

  <article class="pricing-lab__card pricing-lab__card--featured">
    <span class="pricing-lab__badge">Popular</span>
    <h4>Pro</h4>
    <p class="pricing-lab__price">$49<span>/mes</span></p>
    <ul class="pricing-lab__list">
      <li>10 proyectos</li>
      <li>Soporte prioritario</li>
      <li>Layouts premium</li>
    </ul>
    <button type="button" class="primary-button">Elegir plan</button>
  </article>

  <article class="pricing-lab__card">
    <span class="pricing-lab__badge">Equipo</span>
```

```

<h4>Scale</h4>
<p class="pricing-lab__price">$99<span>/mes</span></p>
<ul class="pricing-lab__list">
  <li>Proyectos ilimitados</li>
  <li>Equipo colaborativo</li>
  <li>Soporte dedicado</li>
</ul>
<button type="button" class="secondary-button">Elegir plan</button>
</article>
</div>

```

CSS del componente

styles.css

```

.pricing-lab {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
  gap: 16px;
}
.pricing-lab__card {
  display: grid;
  gap: 14px;
  padding: 20px;
  border: 1px solid var(--border);
  border-radius: var(--radius-md);
  background: rgba(255, 255, 255, 0.06);
}
.pricing-lab__card--featured {
  background: linear-gradient(180deg, rgba(37, 99, 235, 0.2), rgba(255, 255, 255,
0.08));
  border-color: rgba(37, 99, 235, 0.5);
  box-shadow: 0 18px 36px rgba(37, 99, 235, 0.16);
}
[data-theme="light"] .pricing-lab__card {
  background: rgba(255, 255, 255, 0.82);
}
.pricing-lab__badge {
  display: inline-flex;
  width: fit-content;
  padding: 6px 10px;
  border-radius: 999px;
  background: rgba(37, 99, 235, 0.16);
  color: #bfdbfe;
  font-size: 12px;
  font-weight: 700;
}
[data-theme="light"] .pricing-lab__badge {
  color: #1d4ed8;
  background: rgba(37, 99, 235, 0.1);
}
.pricing-lab__price {
  margin: 0;
  font-size: 2rem;
  font-weight: 800;
  color: var(--text);
}

```

```

.pricing-lab__price span {
  font-size: 0.9rem;
  font-weight: 600;
  color: var(--muted);
}
.pricing-lab__list {
  display: grid;
  gap: 8px;
  margin: 0;
  padding-left: 18px;
  color: var(--muted);
}

```

JavaScript

No requiere JS. Esta actividad es puramente de maquetación HTML y CSS. Si deseas agregar interacción, puedes implementar un toggle de facturación mensual/anual o animar la tarjeta al pasar el cursor (hover).

Variables, clases, ids y atributos que debes cambiar

- ****Elemento**** (Para que sirve): Que debes cambiar
- ****pricing-lab**** (clase del contenedor principal): Cambia si colisiona con otra clase
- ****pricing-lab__card** clase de cada tarjeta de plan**: Cambia si colisiona con otra clase
- ****pricing-lab__card--featured** clase para destacar el plan Pro**: Cambia el degradado y sombra si deseas otro estilo
- ****pricing-lab__badge** clase del badge del plan**: Cambia los colores de fondo y texto si usas otra paleta
- ****pricing-lab__price** clase del precio**: Ajusta el tamaño de fuente si es necesario
- ****pricing-lab__list** clase de la lista de beneficios**: Cambia el espaciado o agrega iconos tipo check

Paso a paso para implementarlo en otro proyecto

1. Copia la estructura HTML del contenedor pricing-lab en la sección correspondiente de tu index.html.
2. Copia los estilos CSS de pricing-lab, pricing-lab__card, featured, badge, price y list en tu styles.css.
3. Asegurate de tener configuradas las variables de tema (claro/oscuro) en tu :root o cambialas por colores planos fijos si tu proyecto no tiene temas dinámicos.
4. Ajusta la anchura mínima en el minmax de grid-template-columns según las dimensiones deseadas para tus tarjetas.
5. Cambia el contenido de los planes (nombres, precios, características) por los de tu propio servicio.

Errores comunes y como solucionarlos

- Las tarjetas se ven amontonadas en pantallas medianas: el breakpoint o el tamaño mínimo en minmax(220px, 1fr) es demasiado grande o pequeño. Ajusta el valor del minmax.
- El plan destacado no resalta lo suficiente en tema claro: ajusta el border-color y el

background para el selector [data-theme="light"] .pricing-lab__card--featured.

- La lista de beneficios tiene vietas por defecto (puntos): el navegador aplica estilos de lista por defecto. El CSS incluye reset o define list-style: square/disc si deseas conservarlos, o puedes quitarlos con list-style: none y padding: 0.

Checklist de prueba antes de reutilizar

- El layout es responsive y pasa de 3 columnas en escritorio a 1 columna en pantallas de movil.
- El plan destacado Pro es claramente distinguible de los demas planes.
- Los textos y precios son legibles en tema claro y oscuro.
- No hay scroll horizontal en resoluciones pequenas.

57. Barra de desplazamiento personalizada

Que hace

Aplica estilos personalizados y modernos a las barras de desplazamiento (scrollbars) de la pagina o de contenedores especificos. Reemplaza la barra nativa del sistema operativo por una barra mas delgada, elegante y con transiciones.

Donde se usa en industria

Paneles de control, sidebars de navegacion, bloques de codigo, tablas con scroll horizontal, areas de texto largas, chats y dashboards premium.

Como se personaliza

Ajusta el width (ancho), los border-radius (redondeado), el background-color del track (pista) y el thumb (deslizador). Puedes usar degradados y efectos hover.

Mapa rapido de reciclaje

1

Ubica el contenedor con desbordamiento (overflow: auto o scroll) que quieres estilizar.

2

Copia los selectores CSS especificos de scrollbar.

3

Ajusta las variables de color y el ancho segun el diseno de tu interfaz.

4

Prueba tanto en navegadores basados en Chromium (Chrome, Edge, Opera, Safari) como en Firefox.

HTML minimo que debes copiar

index.html

```
<div class="custom-scrollable-container">
  <div class="custom-scrollable-content">
    <p>Contenido largo que genera scroll...</p>
  </div>
</div>
```

CSS del componente

styles.css

```
/* Contenedor de la demo */
.custom-scrollable-container {
  border: 1px solid var(--border);
  border-radius: var(--radius-md);
  background: rgba(255, 255, 255, 0.03);
  padding: 16px;
}
.custom-scrollable-content {
  max-height: 160px;
  overflow-y: auto;
  padding-right: 8px;
```



```

    /* Soporte para Firefox (estandar moderno) */
    scrollbar-width: thin;
    scrollbar-color: rgba(255, 255, 255, 0.18) rgba(255, 255, 255, 0.02);
}

/* Soporte para Webkit (Chrome, Edge, Safari) */
.custom-scrollable-content::-webkit-scrollbar {
    width: 6px;
    height: 6px;
}
.custom-scrollable-content::-webkit-scrollbar-track {
    background: rgba(255, 255, 255, 0.02);
    border-radius: 999px;
}
.custom-scrollable-content::-webkit-scrollbar-thumb {
    background: rgba(255, 255, 255, 0.18);
    border-radius: 999px;
}
.custom-scrollable-content::-webkit-scrollbar-thumb:hover {
    background: linear-gradient(180deg, var(--primary), var(--secondary));
}

```

JavaScript modular con export/import

funciones.js - no requiere JS

// No requiere JS. Estilo de scroll puro con CSS.

JavaScript normal sin modulos

script-normal.js - no requiere JS

// No requiere JS. Estilo de scroll puro con CSS.

Variables, clases, ids y atributos que debes cambiar

Elemento

Para que sirve

Que debes cambiar

.custom-scrollable-content

Contenedor con scroll.

Reemplaza por tu selector, ej. .guide-sidebar o pre.

width / height

Dimensiones de la barra.

Cambia 6px a 8px o 4px segun el grosor deseado.

scrollbar-color

Estilos en Firefox.

El primer color es el thumb, el segundo es el track.

::-webkit-scrollbar-thumb

Deslizador en Chrome/Safari.

Cambia el color, agrega border o usa degradados en hover.

`::-webkit-scrollbar-track`

Pista de fondo de la barra.

Puedes poner un color solido o mantenerlo transparente.

Implementacion paso a paso

1

Identifica el contenedor en tu HTML que tiene scroll activo (por ejemplo, con `overflow-y: auto`).

2

Copia las propiedades de Firefox (`scrollbar-width` y `scrollbar-color`) dentro del selector de tu contenedor.

3

Copia las pseudoclases `::-webkit-scrollbar` para definir el ancho/alto de la barra en Chrome/Edge.

4

Copia `::-webkit-scrollbar-track` para el fondo y `::-webkit-scrollbar-thumb` para la barra que se mueve.

5

Personaliza los colores en estado normal y hover para que hagan juego con tu paleta visual.

6

Prueba la interaccion desplazando el contenido y pasando el mouse por encima de la barra.

Errores comunes y como solucionarlos

La barra no aparece personalizada: revisa que el contenedor tenga `overflow-y: auto` o `overflow-x: auto`. Si no hay desbordamiento de contenido, la barra no se muestra.

Diferencia entre Firefox y Chrome: Firefox no soporta `::-webkit-scrollbar`, usa `scrollbar-width` y `scrollbar-color`. Asegurate de incluir ambas soluciones.

`Scrollbar-color` no funciona: en Firefox, el valor debe ser de dos colores validos, ej: `scrollbar-color: thumb track`. No uses gradientes directos en esta propiedad.

El contenido se superpone a la barra: agrega un `padding-right` al contenedor scrollable para dejar espacio a la barra y evitar que el texto quede pegado.

Checklist de prueba antes de reutilizar

La barra de desplazamiento aparece mas delgada y estilizada que la nativa.

La barra reacciona al hover cambiando de color.

Funciona tanto en scroll vertical como en scroll horizontal (ej. bloques de codigo pre).

No hay desbordamientos raros de disenio en movil.

Es legible y visible tanto en tema claro como en tema oscuro.

Mejoras opcionales

Usa variables CSS globales para que todas las barras de tu proyecto cambien de color al

mismo tiempo.

Agrega un efecto de borde transparente al thumb usando background-clip: padding-box para simular espacio flotante.

Conecta con JS para ocultar la barra cuando el usuario no esta moviendo el cursor y mostrarla solo en scroll.

Aplica el scrollbar personalizado al body de la pagina principal (html, body) si quieres un look 100% homoganeo.